

ЭЛЕКТРОНДЫ ОҚУ ҚҰРАЛЫ

JAVA

тілінде объектіге бағытталған бағдарламалау

15 дәріс · практикалық жұмыстар
білімді бақылау · қазақша видеосабақтар

Найзағараева А. А.

АСТАНА / 2026

01 / БАСЫЛЫМ

Оқу құралы туралы

Найзағараева А. А. (дайындаған). Java тілінде объектіге бағытталған бағдарламалау. Электронды оқу құралы. Астана, 2026.

Оқу құралы 6B06101 «Бағдарламалық инженерия» және 6B06102 «Цифрлық агрожүйелер мен кешендер» білім беру бағдарламаларының студенттеріне арналған. Дәрістер, зертханалық жұмыстар, қазақша видеосабақтарға сілтемелер және өзіндік жұмысты бағалауға арналған материалдар бір кітапқа біріктірілген.

Оқу бағыты

Дәрісті оқыңыз, тақырып атауын басып видеосабақты ашыңыз, практикалық жұмысты орындаңыз, содан кейін «Білімді бақылау» бөлімінде біліміңізді тексеріңіз. Мазмұндағы әр жол тиісті бөлімге апарады.

Бұл басылымда бастапқы 14 дәріс пен практикалық бөлім қайта беттелді. №15 зертханалық жұмыс негізінде 15-дәріс қосылды. Жаңа оқу нәтижелері, бақылау сұрақтары, глоссарий, қорытынды және Java кодтарының қосымшалары дайындалды. Бастапқы мәтіндегі басқа тілдер мысалдары тиісті түсіндірмемен берілген.

02 / ОҚУ БАҒЫТЫ

Пәннің мақсаты мен міндеттері

Пәннің мақсаты — ақпарат өңдеуге арналған алгоритмдерді құру, оларды Java тілінде іске асыру және объектіге бағытталған бағдарламалаудың негізгі тәсілдерін меңгеру.

- Мәлімет типтерін, операторларды, әдістерді және деректер құрылымдарын орынды қолдану.
- Практикалық есепті кіріс деректер, алгоритм және тексерілетін нәтиже түрінде тұжырымдау.
- Объекттерді модельдеп, кодты түсінікті класстар мен пакеттерге бөлу.
- Есептеу және графикалық құралдарды пайдалану; ерекше жағдайларды талдау.
- Өз шешімін түсіндіру, шекаралық жағдайларды тексеру және жұмыс нәтижесін рәсімдеу.

03 / ҚОЛДАНУ ТӘСІЛІ

Кітаппен жұмыс істеу

01. Теория

Әр дәріс жаңа беттен басталады. Тақырып атауы видеосабаққа сілтеме ретінде қызмет етеді; мазмұн жолы дәрістің өзіне апарады.

02. Практика

Тапсырма бойынша алгоритм, бастапқы код және нәтиже ұсыныңыз. Тек дайын жауапты емес, оның алыну жолын түсіндіріңіз.

03. Білімді бақылау

Әр тақырыпта 3 оңай, 4 орташа және 3 қиын теориялық сұрақ, одан кейін 5 практикалық өзіндік тапсырма бар. Тест форматы қолданылмайды.

04. Навигация

Екі портреттік бет бір мезетте көрінеді. Бағыттау батырмалары, ← → пернелері немесе көлденең сырғыту арқылы парақтаңыз. Үлкейту кезінде кітапты жылжытып оқуға болады.

04 / БІЛІМ МЕН ДАҒДЫ

Оқу нәтижелері

Пәнді меңгерген студент:

01. Java бастапқы кодын компиляциялап, JVM ортасында іске қосады; JDK құралдарының қызметін түсіндіреді.
02. Мәлімет типтерін таңдайды, консольден дерек оқиды және нәтижені түсінікті форматта шығарады.
03. Арифметикалық өрнектер, шарттар, циклдер және әдістер арқылы қолданбалы есептерді шешеді.
04. Бір және екі өлшемді массивтерді, тізімдерді, жиындарды және сөздіктерді қолданады.
05. Рекурсивті алгоритмнің базалық жағдайын анықтап, оның орындалуын талдайды.
06. Класс, объект, конструктор, инкапсуляция, мұрагерлік және полиморфизм ұғымдарын практикада қолданады.
07. Математикалық және графикалық кітапханалар арқылы есептеу мен визуализацияны орындайды.
08. Ерекше жағдайларды өңдейді, шекаралық деректермен бағдарламаны тексереді және нәтижені хаттамаға түсіреді.

05 / КІТАП КАРТАСЫ

Мазмұны

Бөлім немесе тақырып атауын басып, қажетті бетке өтіңіз.

Оқу құралы туралы	2
Пәннің мақсаты мен міндеттері	3
Кітаппен жұмыс істеу	4
Оқу нәтижелері	5
Мазмұны	6
Теориялық бөлім	10
Дәріс 1. Кіріспе	11
Дәріс 2. Тілдің негіздері, мәліметтер түрлері және өңдеу ортасы	15
Дәріс 3. Мәліметтерді енгізу және шығару	22
Дәріс 4. Базалық математикалық операциялар және есептеулер	26
Дәріс 5. Шартты оператор: if	36
Дәріс 6. Циклдік операторлар: for және while	40
Дәріс 7. Тізімдер	47
Дәріс 8. Рекурсия және екі өлшемді массивтер	51
Дәріс 9. Функцияны хабарлау. Жиындар	56

Дәріс 10. Сөздіктер	61
Дәріс 11. Объектіге бағытталған бағдарламалауға кіріспе	65
Дәріс 12. Модульдер	70
Дәріс 13. Mathlab тәрізді және өзге кітапханаларды пайдалану	76
Дәріс 14. Қателерді талдай: throwExceptions	86
Дәріс 15. Ерекше жағдайларды өңдеу және бағдарламаның тұрақтылығы	93
Практикалық бөлім	96
Зертханалық жұмыс №1	98
Зертханалық жұмыс №2	103
Зертханалық жұмыс №3	105
Зертханалық жұмыс №4	108
Зертханалық жұмыс №5-6	113
Зертханалық жұмыс №7	120
Зертханалық жұмыс №8	126
Зертханалық жұмыс №9	130
Зертханалық жұмыс №10	136
Зертханалық жұмыс №11	140
Зертханалық жұмыс №12	142

Зертханалық жұмыс №13	157
Зертханалық жұмыс №14	162
Зертханалық жұмыс №15	167
Білімді бақылау	171
Дәріс 1. Білімді бақылау	172
Дәріс 2. Білімді бақылау	174
Дәріс 3. Білімді бақылау	176
Дәріс 4. Білімді бақылау	178
Дәріс 5. Білімді бақылау	180
Дәріс 6. Білімді бақылау	182
Дәріс 7. Білімді бақылау	184
Дәріс 8. Білімді бақылау	186
Дәріс 9. Білімді бақылау	188
Дәріс 10. Білімді бақылау	190
Дәріс 11. Білімді бақылау	192
Дәріс 12. Білімді бақылау	194
Дәріс 13. Білімді бақылау	196
Дәріс 14. Білімді бақылау	198
Дәріс 15. Білімді бақылау	200

Глоссарий	202
Қорытынды	206
Пайдаланған әдебиеттер	207
Қосымшалар	210
А. Консольдік енгізу және есептеу	211
В. Рекурсия және матрица	212
С. Жолдар және коллекциялар	213
D. Объект және конструктор	214
Е. №15 жұмысқа тексеру хаттамасы	215

06 / ДӘРІСТЕР

Теориялық бөлім

Негізгі бағдарламалау ұғымдарынан объектілер мен ерекше жағдайларды өңдеуге дейінгі 15 тақырып.

Тақырып атауы мен видеосабақ карточкасы сыртқы видеоға апарды. Кітаптың мазмұны дәрістің бастапқы бетіне апарды.

ДӘРІС 01 / ТЕОРИЯ

Дәріс 1. Кіріспе

Видеосабақ: Лекция 1 Объектіге бағытталған бағдарламалауға кіріспе

Қазақша видеосабақ дәрістің негізгі ұғымдарын толықтырады.

Java—Sun Microsystems компаниясының жасап шығарған объектіге бағытталған -бағдарламалау тілі. Java қосымшалары әдетте арнайы байт - кодта компиляцияланады, сол себепті олар кез келген виртуалды Java- машинасында (JVM) компьютерлік архитектурасына тәуелсіз орындалады. Ресми шығу күні — 23 мамыр 1995 жыл.

Программалау тілдері бүкіл әлемде мәліметтерді өңдеу үшін ғана емес, операциялық жүйелерді жасау үшін де қолданылатын, компьютерлердің программалық жабдықтамаларын жасайтын қуатты құрал болып саналады.

Бұл пәнде оқыту кезінде қазіргі жаңа программалау тілі мүмкіндіктері, жобалар жасау тәсілдері беріледі, ол өте өзекті тіл болып саналады. Пән негіздері мамандыққа сәйкес басқа да пәндерді игеруде жан-жақты қолданылады, сондықтан ол болашақ маманның ақпараттық технология саласындағы білімдерін жетілдіру кезінде кеңінен пайдаланылады. «Java тілінде объектіге-бағытталған бағдарламалау» пәні дәрістер жинағы және зертханалық жұмыстар, әр зертханалық жұмысқа сүйемелденген видео сабақтардан тұрады. Салыстырмалы түрдегі тұрақты негізгі теориялық білім (ядро) дәріс сабақтарда, ал кез келген маман білуге қажет динамикалық түрде өзгеретін білім (практикалық мәліметтер) зертханалық сабақтарда беріледі.

Пәнді оқыту мақсаты – ақпарат өңдеуге арналған алгоритмдер мен программалау тілдерін пайдалана отырып, оларды техника, технология, экономика және статистика салаларында қолдану үшін студенттерді дербес компьютерде жұмыс істеу мен өздерінің болашақ кәсіби

мамандығына сәйкес программалар құруды оқытып үйрету;

объектіге-бағытталған программалау тілдерінің ерекшеліктерін мәліметтер типтерін, операторларын, функцияларын оқып-үйрену, осы тілде программалаудың практикалық негіздерін меңгеру; программалар құруда, зерттеуде және оны іс жүзіне орындауда кездесетін қиыншылықтармен және

программаларды трансляциялау әдістерімен танысу. Бұл пәнді игеру нәтижесінде студенттердің программалық технологияларды үйрену ісін компьютер арқылы меңгеруі қамтамасыз етіледі.

Пәнді оқыту міндеттері. Бұл пәнді оқып үйренгеннен кейін, студент осы пәннен алған білімін басқа пәндердің теориясымен байланыстыра отырып, ақпараттық технологияларды игеріп, тиімді программаларды құра білуі тиіс.

Пәнді оқытудың негізгі мәселелері:

а) Java программаларының құрылымдарының негізгі ұғымдарын игеру, ақпаратты қазіргі қоғамның қажетті қоры ретінде бейнелеп, ақпараттық қорларды пайдалануда

студенттердің алгоритмдік ойлау қабілетін қалыптастыру;

ә) студенттерді объектіге бағытталған программалау тілдерінің негізгі сипаттамаларымен таныстырып, Java

тілінде жұмыс істеу негіздерін үйрету;

б) күнделікті өмірде кездесетін әртүрлі есептерді шығара отырып, әр түрлі мәліметтерді программаға енгізіп, олардың нәтижесін алу болып табылады.

Пәнді игерген әрбір студент пәннің негізгі анықтамалары мен түсініктерін біледі, Java тілінде программалар жаза алады, оларды компьютерде орындай біледі.

Java объектіге бағытталған бағдарламалау тілінің жаңартылған түрі бағдарлама құруға жеңіл. Java Интернетпен жұмыс істеуге бағытталған және онда апплеттер деп аталатын кішірек бағдарламаларды қолдануға болады, олар браузер арқылы көрсетіледі және HTML- беттің бөлігі болып саналады.

Деректер түрлері және оларға қолданылатын операциялар Java тіліндегі– бірінші бағдарламасы.

Бұрыннан келе жатқан әдіс бойынша C тілінде программалау сияқты "Hello, World!" сөзінен бастайық. Осы сөз Java тілінде былай жазылады.

Листинг 1.

Java тіліндегі бірінші бағдарлама; (Листинг дугеніміз басылып шыққан программа) `class Hello. World { public static void main(String[] args)`

```
{ System. out. Println ("Hello, XXI Century World!");  
}  
}
```

Кез келген бір немесе бірнеше класстан тұрады, біздің мысалымызда тек бір класс (class).

Класты жазу үшін қызметтік сөз class қолданылады, содан кейін класс аты, кез келген болуы мүмкін, біздің мысалда HelloWorld класқа қатысты сөздер фигуралық жақшаға алынып жазылады және класс денесін (class body) құрастырады.

Мәліметтерді өңдеу әдістер арқылы орындалады (method).

Бұл ат Java тілінде "функция" ретінде қолданылады Әдістер аттары арқылы ажыратылады, олардың біреуі міндетті түрде main деп аталуы керек және программаның орындалуы осыдан басталады. Біздің қарапайым программада бір әдіс, яғни оның аты main Орындалу нәтижесі (returns)) арқылы беріледі және алдында аты көрсетіледі. Орындалғаннан кейін мән болмауы да мүмкін, ол процедура түрінде болады. Онда нәтиже түрі орнына void сөзі жазылады.

Әдіс атынан кейін жақша ішіне үтір арқылы бөлініп аргументтер (arguments) немесе әдіс параметрлері көрсетіледі. Әрбір аргументтің түрі және бос орыннан кейін аты жазылады. Мысалда бір аргумент бар, оның түрі белгілер жолынан тұратын — массив. Белгілер жолы - string, ал квадрат жақшада массив белгілері.

Массив аты кез-келген болуы мүмкін, мысалда args аты алынған.

Қайтарылатын, яғни нәтиже мәнінің түрінің алдынан модификаторлар (modifiers) жазылуы мүмкін. Мысалда,

```
олар екеу: public сөзі әдіс барлығына мүмкін екенін;  
static сөзі программаның басында main () шақыруды
```

білдіреді. Модификаторлардың болуы міндетті емес, бірақ

```
main () үшін олар қажет.
```

Ескерту Әдісте қолданылатын мәндер әдіс денесі (method body) ішінде фигуралық жақшаға алынып жазылады. Мәтіндік тұрақтылар тырнақшаға алынып жазылады. Қосымша

```
System. out. println аты System класындағы Java API
```

кіретін out атты айнымалы анықтайды, ол Print. Stream классында болады.

`println()` әдіс. `println()` өз аргументін шығарады, ол экранға шығатын мәтіндік терминалмен байланысты және жүйеге байланысты мына терезелер қолданылады. MS-DOS Prompt немесе Command Prompt немесе Xterm. Java тілінде бас әріп пен үлкен әріптер әртүрлі қабылданады.

Түсініктеме

Программада түсініктемелер былай жазылады:

```
екі көлденең сызық //;
```

жұлдызшасы бар көлденең сызық `/*` бірнеше жолдардан тұруы мүмкін `*/` (белгілер арасынан бос орын жіберуге болмайды).

Java үшінші түр JDK — программа `javadoc` үшін енгізілген түсініктеме бар. HTML форматындағы жеке файл үшін қолданылады және гиперсілтеулер жасай алады:

көлденең сызық екі жұлдызша `/**` түсініктемелер `*/` жазылады және `javadoc` программасымен өңделеді. Бұндай түсініктемеге `javadoc` программасындағы `@` жазуға болады.

JDK құжаты осылай құрылады.

Листинг 1. 2.

Түсініктемесі бар бірінші программа `/**` Программа ерекшелігі және оны талдау. `@author` Аты Фамилиясы (автор) `@version 1.0` (программаның нұсқасы) `*/`

```
class Hello. World
{
// Hello. World – бұл тек ат
// Келесі әдіс программаның орындалуын бастайды public
static void main(String[] args) {
// args қолданылмайды /* Келесі әдіс өз аргументін
```

экранға шығарады `*/`

```
System. out. Println ("Hello, 21 st Century World!");
// Келесі әдіс орындалмайды //
System. out. Println ("Farewell, 20 th Century!");
}
}
```

ДӘРІС 02 / ТЕОРИЯ

Дәріс 2. Тілдің негіздері, мәліметтер түрлері және өңдеу ортасы

Видеосабақ: Java программалау тілі / Сабақ #1.

Java негіздері, айнымалылар және мәлімет типтері.

Жоспар

2.1. Java тілі (даму тарихы) 2.2. Java Script пен Jаваның ортақ қасиеттері 2.3. Java тілінің ерекшеліктері 2.4. Алғашқы программа 2.5. Лексикалық негіздер 2.6. Типтер

1. Java тілі (даму тарихы) Java жаңа объектілі бағытталған программалау тілі, сонымен бірге оқып үйрену кезінде жеңіл қабылданатын және кез -келген платформада артық жұмыс атқармай программаларды құруға мүмкіндік беретін тіл. Java беймәлім себептермен кофе тақырыбымен байланысты (логотипі, өнім аты) болып кеткен.

Сыншылардың айтуы бойынша, бұл тіл алғашқы қолдану кезінде өте ыңғайлы және қарапайым болғанымен, көптеген тамаша қасиеттері тек теория жүзінде беріліп, бірақ олар негізінен дұрыс істемеді. 1996 жылы бүкіл әлем Java тілінің құрастырушыларын біріктірген Sun арнайы программасы іске қосылды. Бастапқы кезде тіл ОаК ("емен") деп аталған, оны құру бойынша жұмыстар 1990 жылы басталды. Тілдің бес негізгі версиясы бар, 1995 жылы

1.0 версиясы және 2005 жылы 1.5 версиясы шықты.

Барлық

версияларды және оған арналған барлық құжаттарды арнайы веб-сайттан тегін алуға болады: <http://java.sun.com/>.

Осы сайт қазіргі күнге дейін Java тілі бойынша мәліметтерді алудың негізгі көзі болып отыр. Кеңінен таралған JavaScript тілін көбісі Java тілімен

байланыстырады, себебі аттары ұқсас. Бірақ олар әр түрлі программалар. Дегенмен, кейбір жалпы ұқсас қасиеттері бар.

2. Java Script пен Jаваның ортақ қасиеттері:

- жеңіл меңгерілуі. Осы параметр бойынша Java Script-ті Visual Basic-пен салыстырады. Бұл тілдерді қолдану үшін

программалау тәжірибесі талап етілмейді;

- кроссплатформалы. JavaScript коды браузермен орындалады. Браузерлер әр түрлі платформаларда сценарий тілін қолданушы веб- парақтар үшін бірдей функцияларды

қамтамасыз етуі тиіс;

- ашықтығы, тілдің спецификациясы (ережелері)

қолданушылар үшін ашық;

- JavaScript Интернет ортасы үшін программалауға жақсы

бейімделген;

- Java Script пен Java тілдерінің синтаксистері өте

ұқсас. Олар Java тілі негізінде жасалған;

- JavaScript тілі нағыз объектіге бағытталған тіл емес, бірақ ол браузер көмегімен әр түрлі объектілермен жұмыс істей алады. Сипаттамаларының ұқсастығына қарамастан, Java и Java Script – әр түрлі тілдер. Java – бұл толық программалау тілі, ал Java Script – HTML тілінің толықтырмасы.

Java тіліндегі кез келген программаның мәтінін жай редакторда немесе тілдің арнайы жабдықтамасында теруге болады және оның кеңейтілуі: *.java. Мысалы, azat.java, prog1.java Компилятор жұмысының нәтижесі class кеңейтілуі бар бинарлы файлдарда сақталады. Жавада қолданылатын мәліметтердің 8 типі бар, олар объект болып табылмайды. Бес бүтін санды типтер бвр, олар:

byte, short, int, long және символдық тип char. Екі нақты тип

float, double және бульдік тип boolean кіреді. Мұндай

типтерді қарапайым немесе примитивті типтер деп атайды (ағылшынша primitive). Басқа барлық типтер - объектілі немесе сілтемелі (ағылшынша, reference) тип деп аталады.

Java синтаксисі C/C++ тілдерінің негізінде құрылған.

3. Java тілінің ерекшеліктері:

- Объектіге бағытталған тіл, онда тиімді объектілі

модель құрылған;

- C/C++ синтаксисін қолдайды;

- Internet ортасында пайдалануға бағытталған, мұнда

желілік қосым-шаларды құруға да болады;

- динамикалық жаңа мүмкіндіктерді қосу және дамыту

мүмкіндігі бар;

- үйрену үшін қарапайым тіл деп саналады.

4. Алғашқы программа.

Java тілінің алғашқы файлы – кластардың бір немесе бірнеше сипаттамаларынан тұратын мәтіндік файл.

Программа файлы java кеңей-тілуі бар файлдарда сақталады (Hello.java). Файл аты мен класс аты бірдей болуы тиіс.

Сонымен, алғашқы Java-программа мәтіні:

```
class HelloWorld {  
public static void main (String args [ ])   
{System. out. println ("Hello World");  
}
```

Программаны трансляциядан өткізгенде, class типіндегі аралық файл пайда болады. Сонан кейін ол орындалып, программа нәти-жесі шығады. Жоғарыдағы файлды Hello.java орындағанда, оның нәтижесі мынадай сөз болады: Hello World!

Программа жолдарын талдап барып, тілдің төмендегі негізгі элементтерін қарастырамыз, олар: босорын, комментарий, түйінді сөздер, идентификаторлар, литералдар, операторлар және айыру таңбалары.

Енді әрбір жолға тоқталып өтейік:

```
class HelloWorld { ... } Мұнда class түйінді сөзі
```

қолданылады. Ол трансляторға жаңа класты сипаттайтынымызды хабарлайды.

```
public static void main (String args [ ]) { ... } public түйінді
```

сөзі— қатынасу модификаторы, ол кез келген айнымалы мен тәсілдің қолданылу (көріну) мүмкіндігін басқарады.

Мұнда public модификаторы main тәсілінің кез келген

класпен қатынаса алатынын көрсетеді;

- static түйінді сөзі жалпы осы класпен жұмыс істеу үшін қолданылатын класс айнымалылары мен тәсілдерін хабарлайды. static сөзі көрсетілген тәсілдер жергілікті және статикалық айнымалылармен ғана жұмыс істей алады.

void модификаторы тек мәтінді экранға шығару керектігін

хабарлайды, мұнда main тәсілінен мән қайта ру талап етілмейді. String args[] элементі args параметрін хабарлайды, мұндағы String типі -бұл класс. Тік жақша жиым, яғни массив қолданылатынын көрсетеді.

```
System.out.println("Hello World!");
```

мұнда out объектісінің println тәсілі орындалады.

out объектісі OutputStream класында хабарланған және System класында инициализацияланады.

5. Лексикалық негіздер.

Java тіліндегі программа— бұл бос орындардың, түсініктемелердің, түйінді сөздердің, идентификаторлардың, тұрақ-тылардың, операторлардың жиынтығы.

Түсініктемелер. Бір жолға сиятын комментарийлер сол жолдың соңында орналасады да,

```
// символдарынан бастап жазылады. Мысалы: a = 42; // 42 -
```

а айнымалысының мәні Бірнеше жолдан тұратын толық мағыналы түсініктемелер

```
/* және */ символдарының арасына орналасады. Осындай
```

түсініктемелер комментарий болып саналады, оларды кездестіргенде, транслятор ешқандай әрекет орындамайды.

```
/* Мұндай кодтар түсініктемелерді қажет етеді, сондықтан
```

осылай жазылады. Оларды былай түсіндіреміз

*/ Комментарилердің үшінші түрі javadoc сервистік программасы үшін жасалған, оларды Java трансляторы жинақтай отырып, автоматты түрде құжаттама жасау үшін қолданады, Сонымен, Java-дағы комметарийдің осы түрі және JDK құрамындағы javadoc программасы соларды жинақтап, жеке HTML форматындағы файл жасайды да, оларды гиперсілтемелермен толықтырады.

Түйінді сөздер. Түйінді сөздер (keywords –ключевые слова)—бұл арнайы идентификаторлар. Қазіргі кезде Java- да 59 түйінді сөздер бар. Төмендегі кестеде солардың тізімі берілген: abstract case const else float Int null protected

```
static throw var boolean cast continue extends for implements  
interface operator public if super throws void break byte  
byvalue
```

Идентификаторлар. Идентификаторлар кластарды, тәсілдерді және айнымалыларды атау үшін қолданылады.

Идентификатор ретінде әріптер, цифрлар және _ (төменгі сызық) және \$ символдары қолданылады.

Идентификаторлар цифрдан бастал-мауы керек, себебі транслятор оларды сандық литералдармен шатастыруы мүмкін. Java — әріптердің бас әріппен немесе кіші әріппен жазылуына сезімтал, сондықтан Value и VALUE — әр түрлі идентификаторлар болып табылады.

Тұрақтылар. Java –да әр түрлі типті тұрақтыларды қолдануға болады. Бүтін тұрақтылар. Бүтін сандар түріндегі тұрақтылар программада жиі қолданылады. Бүтін тұрақтылар int типінің мәні болып табылады, ол Java-да 32-биттік машиналық сөзде сақталады. Егер өте үлкен тұрақтылар қажет болса, long типіндегі тұрақтылар қолданылады (2 миллиардқа дейінгі сандар).

Қалқымалы нүктелі тұрақтылар. Қалқымалы нүктелі сандар бөлшегі бар ондық мәндерді сипаттайды. Java -да қалқымалы нүктелі сандар double типінің мәні ретінде қарастырылады. Егер float типінің тұрақтысы қажет болса, онда литералға F немесе f символдарын соңына қосып жазу керек. Бүтін тұрақтылар түрінде сегіздік және оналтылық сандар да қолданылады, сегіздік сандар алдында 0 орналасады, мысалы, 0725. Ал оналтылық сандар 0X символдарынан басталады, мысалы, 0X98A. Оналтылық сандар цифрлары— 0..15 аралығында болады, мұнда 10..15 цифрлары орнына А-дан F-ке дейінгі әріптер қолданылады (немесе а-дан f -ке дейін).

Логикалық тұрақтылар. Логикалық айнымалының тек екі ғана мәні болуы мүмкін: true (ақиқат) және false (жалған).

Символдар. Java-дағы символдар – UNICODE символдар кестесіндегі индекстер. Символдық тұрақтылар (' ') ішінде орналасады, мысалы, ASCII-код символдары:

'a', 'z', '@'. Символдар түрінде басқару тізбектері де қолданылады.

Символдарды басқару тізбегі

Басқару тізбегі Сипаттамасы \ddd Сегіздік символ (ddd) \uxxxx Он алтылық символ UNICODE (xxxx) \' Апостроф \" Қостырнақша \\ Кері көлбеу сызық \r Каретканы қайтару (carriage

return)

\n Келесі жолға көшу (line feed, new line) \f Келесі бетке көшу (form feed) \t Көлденең табуляция (tab) \b Бір символға кері қайту (backspace) Операторлар

Оператор бір немесе екі аргумент арқылы белгілі бір әрекет орындап, нәтиже беретін амалдардан тұрады.

Синтаксистік тұрғыдан алғанда, операторлар идентификаторлар мен литералдар арасында орналасады.

Операторлар тізбегі келесі кестеде көрсетілген.

Айнымалылар. Айнымалы — Java программасындағы ақпаратты сақтаудың негізгі элементі. Айнымалылар идентификаторлармен, типтермен және солардың әсер ету аймағымен сипатталады. Айнымалылар сипатталу орнына қарай жергілікті және ауқымды болуы мүмкін. Жергілікті айнымалылар көбінесе жүйелі жақшалармен қоршалған аймақта орналасады.

Айнымалының жариялану форматы: типі идентификатор [= мәні] Типтер — byte, short, int, long, char, float, double,

boolean. мысалы: int d = 3, e, f = 5;

Кейбір айнымалылардың бірден инициалданатынына назар аударыңдар. Алғашқы мәні берілмеген айнымалы мәні нөлге тең болып саналады. Пифагор теоремасы бойынша үшбұрыштың гипотенузасын есептеу:

```

class Variables
{ public static void main (String args [])
{
double a = 3, b = 4, c;
c = Math.sqrt (a* a + b* b);
System.out.println ("c = ", c);
}
}

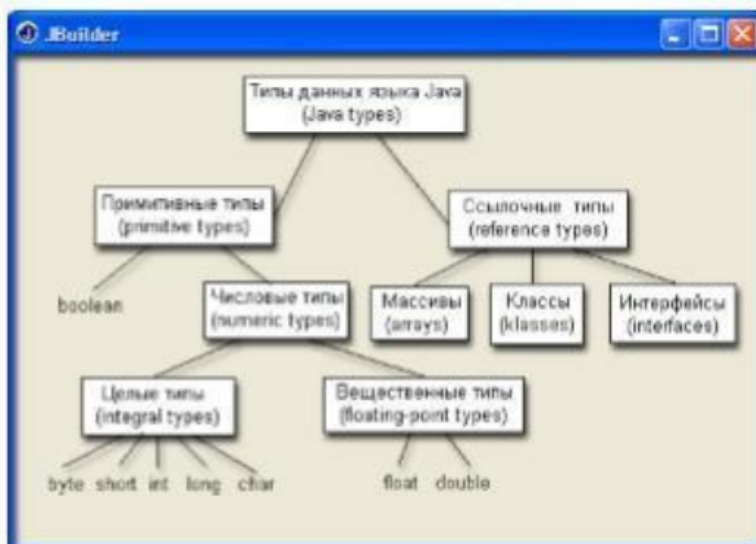
```

Типтер Типтер: примитивті (primitive types) және сілтемелі (reference types).
Сілтемелі типтер жиым (arrays), кластар (classes) және интерфейстер (interfaces) болып бөлінеді.

Примитивті типтердің 8 түрі бар, олар логикалық (boolean) және сандық типтер болып бөлінеді. Бүтін типтер: byte , short , int , long , char. Нақты типтер: float , double. Java тілінің типтері төмендегі суретте келтірілген:

Тақырып бойынша сұрақтар:

1. Идентификаторлар.



2. Тұрақтылар.

3. Қалқымалы нүктелі тұрақтылар

4. Айнымалылар.

5. Айнымалының жариялану форматы

ДӘРІС 03 / ТЕОРИЯ

Дәріс 3. Мәліметтерді енгізу және шығару

Видеосабақ: Лекция 8 Кіріс шығыс үрдістері

Қазақша видеосабақ дәрістің негізгі ұғымдарын толықтырады.

Жоспар

3.1. File класы 3.2. Енгізу-шығару ағындары 3.3. Алдын ала анықталған ағындар

File класы Java қосымшаларында файлдармен жұмыс жасау үшін java.io пакетінің кластары қолданылады.

File классы каталогтар мен файл аттарын объект ретінде сақтап, өңдеуге мүмкіндік береді. File классының тәсілдері файлдың жасалған уақыты, файлға қол жеткізу (доступ), жаңадан файл ашу, атын өзгерту, файлды жабу, өшіру, файл маршрутын анықтауға мүмкіндік береді.

File классының тәсілдері

```
Жаңадан файл ашу - File fp=new File("demo.txt");  
Файл атын анықтау - fp.getName();  
Файл маршрутын анықтау - fp.getPath();
```

Файлға баратын абсолютты жолды анықтау -

```
fp.getAbsolutePath();  
Файл мөлшерін анықтау - fp.length();
```

Файлдың соңғы өңделген уақытын анықтау -

```
fp.lastModified();  
Файлды оқу үшін ашу - fp.canRead();  
Файлды жазу үшін ашу - fp.canWrite();  
Файлды өшіру - fp.delete();  
Файлды жабу - fp.close();  
Осыларды қолданып мысал келтірейік. import java.io.*;  
import java.util.*;  
public class FileTest { public static void main(String[] args)
```

throws IOException /*отказ от обработки исключения в

```

main()*/
{ PrintWriter pw=new PrintWriter (new OutputStreamWriter
(System.out, "Cp866"), true);
File fp=new File("C:\\demo.txt");
if(fp.isFile()){//если объект дисковый файл pw.println ("Имя
файла:\t" + fp.getName());
pw.println ("Путь к файлу:\t" + fp.getPath());
pw.println ("Абсолютный путь:\t" + fp.getAbsolutePath ());
pw.println ("Размер файла:\t" + fp.length());

```

```

pw.println ("Последняя модификация файла:\t" +

```

```

fp.lastModified());
pw.println ("Файл доступен для чтения:\t" + fp.canRead());
pw.println ("Файл доступен для записи:\t" + fp.canWrite());
pw.println("Файл удален:\t" + fp.delete());

```

```

} if(fp.createNewFile()){ pw.println("Файл " + fp.getName() +

```

```

" создан");

```

```

} if(fp.exists()){ pw.println("файл "+ fp.getName() + "

```

```

существует");
}else
pw.println("файл"+ fp.getName() + " не существует");
//в объект типа File помещается каталог\директория
Filedir=new
File("f:\\myworks\\умкjava");

```

```

If (dir.isDirectory())/*если объект объявлен как каталог на

```

```

диске*/ pw.println ("Директория!");
if(dir.exists()){//если каталог существует pw.println ("Dir " +
dir.getName() + " существует");
File [] files = dir.listFiles();
pw.println ("");
for(int i=0; i<files.length; i++)
{ Date date = new Date(files[i].lastModified());
pw.println(files[i].getPath() + " \t| " + files[i].length() + "\t| " +
date.toString()); //toLocaleString());
//toGMTString()
}
}
}

```

Енгізу-шығару ағындары Байт тізбектерін енгізу ағындары абстрактты InputStream классының ішкі классы, байт тізбектерін шығару ағындары абстрактты OutputStream классының ішкі классы болып саналады. Файлдармен жұмыс жасағанда осы кластардың ағындарды ашып, оны сәйкес файлдармен

байланыстыратын `FileInputStream` және `FileOutputStream` ішкі класстарының құрастырушылары қолданылады. Байты немесе байттар массивін оқу үшін `FileInputStream` класының `read()/read(byte[] b)` тәсілдері қолданылады.

```
import java.io.*;
public class ReadBytes {
    public static void main(String[] args){ int b;
    try
    {
        PrintWriter pw=new PrintWriter (new
        OutputStreamWriter(System.out, "Cp866"), true);
        File f = new File("ReadBytes.java");
        FileInputStream is = new FileInputStream(f);
```

```
While ((b= is.read()) != -1) { //прочитанные данные
```

```
выводятся на консоль pw.println("прочитан байт = " + b);
}
```

```
is.close(); //закрытие потока ввода }
```

```
catch (IOException e)
{ System.out.println("ошибка файла: " + e);
}
}
```

Байтты немесе байт массивін файлға жазу үшін шығару ағыны – `FileOutputStream` класының `write()` тәсілі қолданылады.

```
import java.io.*;
public class WriteBytes {
    public static void main (String[] args){ int pArray[] = {1, 2, 3,
    5, 7, 11, 13, 17};
    try { FileOutputStream os = new FileOutputStream
    ("bytewrite.dat" );
    for (int i = 0; i < p Array.length; i++)
    os.write(pArray[i]);
    os.close();
    }
```

```
atch (IOException e) { System.out.println("ошибка файла: " +
e);
}
}
```

Енгізу-шығару ағындарын сүзбеден өткізуге арналған `DataInputStream` класының тәсілдері енгізілген мәндерді сәйкес базалық типтерге түрлендіреді: `readBoolean()`, `readByte()`, `readChar()`, `readInt()`, `readLong()`, `readFloat()`,

`readDouble()` `DataOutputStream` класының тәсілдері базалық типтегі мәндерді шығару ағындарына түрлендіруге мүмкіндік береді: `writeBoolean()`, `writeChar()`, `writeInt()`, `writeLong()`, `writeFloat()`, `writeLine()`.

Тақырып бойынша сұрақтар:

1. Файл классы
2. Идентификаторлар.
3. Тұрақтылар.
4. Айнымалылар.
5. Айнымалының жариялану форматы
6. Қалқымалы нүктелі тұрақтылар

ДӘРІС 04 / ТЕОРИЯ

Дәріс 4. Базалық математикалық операциялар және есептеулер

Видеосабақ: Java программалау тілі / Сабақ #1.

Арифметикалық операциялар бөлімі осы дәріске сәйкес келеді.

Жоспар

4.1 Операторлар: арифметикалық әрекеттер, меншіктеу, инкрементті, декрементті.

4.2 Лексикалық негіздер

4.3 Түсініктемелер

4.4 Жадыда сақталған Java сөздері

4.5 Идентификаторлар

Java тіліндегі бастапқы файл — ол өзінде бір немесе бірнеше класстардың сипаттамасын қамтитын мәтіндік файл. Java трансляторы, бағдарламаның бастапқы мәтіні кеңейтулері бар файлдарда сақталатынымен түсіндіріледі.

Трансляция барысында әр класс үшін алынатын код, сәйкес ат, класс атымен және class кеңейтілуімен жеке шығу файлдарында жазылады.

Ең алдымен, бұл тарауда, біз трансляция жасап және канондық «Hello World» деген бағдарламаны жазып жүргіземіз. Бұдан әрі біз Java- трансляторы қабылдайтын барлық лексикалық элементтерді қарастырамыз: аралықтар, түсініктемелер, негізгі сөздер, идентификаторлар, литералдарды, операторлар және бөлгіштер.

Тараудың соңына қарай сіздер Java- бағдарламасында өздігінен бағыттала алатындай жеткілікті мәлімет ала аласыздар.

Hello World Біздің бірінші Java-программамыз:

```
class HelloWorld {  
public static void main (String args []) { System. out. println  
(«Hello World»);  
}  
}
```

Қадамнан қадамға Әрине, HelloWorld — бұл тривиальды мысал. Алайда тіпті мынадай жай бағдарлама Java тіліндегі жаңа қолданушыға күрделі болып көрінеді, өйткені ол сендерді көптеген жаңа ұғымдар мен тіл синтаксисының бөлшегітерімен таныстырады.

```
class HelloWorld
```

1 қатар

```
class HelloWorld{
```

Бұл қатарда сақталынған class сөзі пайдаланылады. Ол трансляторға біздің жаңа классты сипаттайтынымызды хабарлайды. Класстың толық сипаттамасы бірінші қатарда ашылған фигуралы жақша мен оған қос 5 қатардағы жабылатын фигуралы жақшалардың арасында орналасқан.

Java-да қолданылатын фигуралы жақшалар C мен C++ тілдеріндегідей қолданылады.

2 қатар

```
public static void main (String args []) {
```

Бір көзге қарағанда мысал қатарының тым қиын болуы, Java тілін өңдеуде қаланған маңызды талаптардың салдары болып табылады. Java-да ғаламдық атқаратын қызметтер болмайды. Осындай қатарлар бірінші тараудың көп бөлігінде кездескендіктен, екінші қатардың әрбір элементін қарастырайық.

Public Бұл қатарларды бөлек лексемаларға бөле отырып, public сөзімен кездесеміз. Бұл — бағдарламашыға көрінетін кез келген әдіс пен айнымалыларды басқаратын модификатордың мүмкіндігі. Бұл жағдайдағы public модификатордың мүмкіндігі, main әдісі кез келген класста көрінетіндігін және жетімділігін білдіреді.

Мүмкіндіктің (доступ) тағы 2 сілтегіш деңгейі бар —

```
private және protected бар. static
```

Бұл сөз арқылы класстың әдістері мен айнымалылар жарияланып, класстармен толық жұмыс жасау үшін пайдаланылады. Static сөзі қолданылатын әдістер, тек локальды және статистикалық айнымалылармен жұмыс жасайды.

```
void
```

Сіздерде бір немесе басқа үлгінің мағынасын қайтаратын әдістерге жиі қажеттілік туады: мысалға, бағдарламашымен тағайындалатын `int` толық мағыналар үшін, `float` – заттай немесе деректер үлгілерінің класс атауы үшін. Біздің жағдайда экранға тек қатарды шығару керек, ал `main` әдісінен мағынасын қайтарудың керегі жоқ. Дәл осы себептен `void` модификаторы пайдаланылды.

`main` Енді `main` әдісін талқылайық. Бұл жерде ешқандай ерекшелік жоқ, тек барлығын Java- интерпретаторымен жүзеге асыру керек, классты интерпретациялау командасын алып, `main` әдісінің шақыруынан өзінің жұмысын бастайды. Java- транслятор `main` әдісі жоқ классты трансляторлай алады. Ал Java - интерпретатор классты `main` әдісінсіз жүргізе алмайды.

Әдіске жіберілетін барлық параметрлер, «;» символымен бөлінген элементтер тізбегі ретінде қос домалақ жақшада көрсетіледі. Параметрлер тізбегінің әрбір элементі, бөлшектенген қадам типі мен идентификатордан тұрады.

Тіпті әдісте параметрлер болмаған жағдайда, оның артынан міндетті түрде бірнеше домалақ жақша қою керек. Қазір талқылап отырған мысалда, `main` әдісінде күрделендірілген типтегі тек бір параметр бар.

```
String args элементі[] args атымен аталатын параметрді
```

жариялайды, ол `String` классты массивтер объектілерін жариялайды. `args` идентификаторынан кейін тұрған тік бұрышты жақшаларға назар аударыңыздар. Олар біздің көрсетілген типтің жеке элементімен емес, массивтермен жұмыс істеп жатқанымызды айтады.

3 қатар

```
System.out.println («Hello World!»);
```

Осы қатарда `out` нысанының `println` әдісі орындалады. `out` нысаны `OutputStream` классында жарияланған және статикалық `System` классында инициализацияланады. `Main` әдісін хабарлау 4 қатардағы жабылмалы ирек

жақшамен бітеді, ал HelloWorld классы 5 қатарда дәл осындай фигуралы жақшамен хабарлануымен аяқталады.

Лексикалық негіздер Java- классын тереңірек қарастырғаннан кейін, енді осы тілдің синтаксисының ортақ аспектітерін қарастырайық.

Java-ның бағдарламалары— қадамдар, түсініктемелердің, бұлақты сөздердің, идентификаторлардың, литералдық тұрақтылардың, операторлар мен бөлгіштердің жиынтығы.

Қадамдар Java – тілі, бағдарламаның мәтінін еркін өңдеуге мүмкіндік беретін тіл. Бағдарлама дұрыс жұмыс жасау үшін, мәтінді арнайы әдістермен түзетудің қажеті жоқ.

Мысалы, HelloWorld классын екі қатарда немесе кез келген басқа әдіспен жазуға болар еді. Тек бөлек лексамалардың арасында бір қадам, табуляция символы немесе қатар аудармасының символы болуы керек, сонда ол дұрыс жұмыс атқарады.

Түсініктемелер Түсініктемелер бағдарламаның кодына әсер етпесе де, оны дұрыс пайдалану, бастапқы мәтіннің байыпты бөлігі болып табылады.

Түсініктемелердің үш түрі: бір қатарға түсініктемелер, бірнеше қатардағы түсініктемелер және документтеудегі түсініктеме деп бөлінеді. Бір қатарды алатын түсініктемелер // деген символмен басталып, қатар соңында аяқталады. Бұл түрдегі түсініктемелер бөлек қатар кодтарына қысқаша түсініктеме бергенге пайдалы.

```
a = 42; // егер 42 – жауап болса, онда қандай сұрақ
```

болды?

Бірнеше қатарларда орналасқан және мәтін түсініктемелерін

```
/* мына символмен бастап, */ осы символмен аяқтау
```

арқылы, біз түсініктемелерді толық түсіну үшін пайдалана аламыз. Осы қос символдар арасында орналасқан мәтін, түсініктеме ретінде пайымдалады және оны транслятор ескерусіз қалдырады.

```
/*
```

* Этот код несколько замысловат... * түсіндіруге тырысайық:

*

*/

Үшінші түрдегі ерекше түсініктемелер javadoc сервисті бағдарламасына арналған, ол Java-транслятор компоненттерін автоматты түрде класс интерфейстері бойынша документтер генерациясына қолданылады.

Түсініктемелердің бұл түрін қолданудың келісімі мынада:

ашық (public) классын хабарлау алдында, /** түріндегі символдан бастау керек. Бұндай түсініктемелер, қарапайым түсініктемелер сияқты аяқталады. Javadoc бағдарламасы түсініктемелі құжаттардағы @ деген символдан басталатын кейбір арнайы айнымалыларды ажырата алады.

/**

* Осы класс керемет мүмкіндіктерді жасай алады.

* бұдан да жетілдірілген класс жазам деушілерге, осыны негіз етіп алуды ұсынамыз.

* @see Java. applet. Applet * ©author Patrick Naughton * @version 1. 2 */

class CoolApplet extends Applet { /**

* бұл әдістің екі параметрі:

* @param key – параметрдің аты.

* @param value — key есімді параметрдің мағынасы .

*/ void put (String key, Object value)

{

Жадыда сақталған ұйытқы сөздер Жадыда сақталған сөздер- Java тілінде кірістірілген типтерді, модификатор мен басқару құралдарын идентификаторлау үшін қолданылатын бағдарлама. Бүгінгі таңда Java тілінде 59 жадыда сақталған сөздер бар (2 кестені қараңыз). Жадыда сақталған сөздер операторлар және бөлгіштердің синтаксисымен Java тілінің сипаттамасына кіреді. Оларды айнымалылар, класстар мен әдістердің идентификаторы ретінде қолдануға болмайды.

2 кесте Жадыда сақталған Java сөздері

abstract boolean break byte byvalue case cast catch char class const continue default do double

```

else extends false final finally
float for future generic goto
if implements import inner instanceof
int interface long native new
null operator outer package private
protected public rest return short
static super switch synchronized this
throw throws transient true try
var void volatile while

```

byvalue, cast, const, future, generic, goto, inner, operator, outer, rest, var сөздері Java-ның жадысында сақталған, бірақ әлі қолданысқа кірмеген. Сонымен қатар Java да жадыда сақталған әдіс атаулары бар.

Java әдістерінің жадыда сақталған аттары clone equals finalize getClass hashCode notify notifyAll toString wait

Идентификаторлар Идентификатор класстарды, әдістер мен айнымалыларды атау үшін қолданылады. Идентификатор ретінде кез келген тізбектелген қатар, жазбаша әріптер, сандар және мына символдарды (_ \$) қолданылады. Транслятор оларды астында сипатталған сандық литералдармен, константамен салыстырмау үшін, идентификатор сандардан басталмауы керек. Java — әріптің регистріне сезімтал тіл. Мысалы, Value және VALUE — түрлі идентификаторлар дегенді білдіреді.

JavaScript-тегі арифметикалық әрекеттер операторлары дәл C және Java-дағы операторлар секілді: қосу "+", азайту "-", көбейту "*", бөлу"/", бүтін санды бөлгендегі қалдық"%".

Бұл операторлар кез-келген сандық өрнекте кездеседі.

(Назар аударыңыз! Олар сондай -ақ қатарлы өрнектерде сандарды автоматты түрде қатарға айналдыру жағдайларында кездеседі).

JavaScript-тегі меншіктеу операторлары C және Java -дағы секілді:

```
"=", "+=", "-=", "*=", "/=", "%="
```

```
x=y, басқа тілдердегі секілді "x" айнымалысына "y"
```

айнымалысының мәнін меншіктейміз.

Келесі операторлар C тілінің операторларының синтаксисіне ұқсас синтаксиске ие:

$y+=x$ эквивалентті $y=y+x$ $y-=x$ эквивалентті

```

y=y-x  y*=x  эквивалентті
y=y*x  y/=x  эквивалентті
y=y/x  y%=x  эквивалентті
y=y%x  – бөлгендегі қалдық y-ті x-ке.

```

Шартты өрнек келесі түрде болады:

(шарт)?мән1: мән2 Егер шарттың мәні true болса, онда шартты өрнектің мәні мән1, болмаса – мән2 болады. Шартты өрнекті қарапайым өрнекті қолдануға болатын барлық жерде қолдануға болады.

Мысал:

```
a=(b<1)?0:(x-1)+c
```

Инкрементті және декрементті операторлар C тілінен алынған синтаксиске ие:

"x++", "++x", "x--", "--x".

Өрнектер:

```

y=x++ екі меншіктеуге эквивалентті: y=x; y=y+1, y=++x
екі меншіктеуге эквивалентті: x=x+1;
y=x, y=x-- екі меншіктеуге эквивалентті: y=x; x=x-1, y=--x

```

екі меншіктеуге эквивалентті: x=x-1; y=x.

Қатарлы операциялар Қатарлармен жұмыс істеуге арналған бірнеше операторлар бар:

"+" - s1+s2 қатарларын қосу (конкатенация) s1 қатарының символдарының кезектілігінен одан кейін s2 қатарының символдарынан тұратын қатарды береді.

eval(s) – JavaScript-тің ішкі қосылған функциясы. Ол JavaScript-тің бір немесе бірнеше операторларынан (нуктелі үтір арқылы) тұратын аргументпен – s қатарымен берілген кодты орындайды. Бұл функцияны тек операторды орындауға ғана емес, сондай-ақ өрнекті есептеуге де қолдануға болады. Ол берілген кодтағы соңғы есептелген өрнектің мәнін береді. eval(s) функциясы қолданушымен енгізу пункттеріне енгізілген мәндерді есептеуге, сонымен қатар JavaScript-бағдарламада орындалатын кодты динамикалық модификациялауға мүмкіндік береді.

parseInt және parseFloat функцияларынан көрі жалпы.

`parseFloat(s)` – JavaScript-тің ішкі қосылған функциясы. `S` қатарында орналасқан қатардың басынан санға жатпайтын бірінші символға дейінгі заттық санды (`Float` типті) табады. Егер сан табылмаған болса, `NaN` (“Not a Number”) мәнін береді.

`parseInt(s)` – бұл да бүтін сандар үшін (`Integer`). Бұл жағдайда негізі автоматты табылады.

`parseInt(s,n)` – бұл да бүтін негізі `n` (от 2 до 36) сандар үшін.

`n=0` болған кезде `-parseInt(s)` секілді. Бұл жағдайда негізі

автоматты табылады.

Меншіктеудің биттік операциялары Биттік меншіктеудің бірнеше операторлары бар:

$x \ll n$ эквивалентті $x = (x \ll n)$ — x бүтін санының екілік

түрінің оңға қарай n битке жылжуы;

$x \gg n$ эквивалентті $x = (x \gg n)$ — x бүтін санының белгілік битінің сақталуы бойынша екілік түрінің оңға қарай n битке жылжуы (қосымша кодта теріс сандарда бірінші бит бір болып табылады. Жылжудан кейін бірінші биттің

орнына 1 жазылады және сан теріс болып қала береді);

$x \ggg n$ эквивалентті $x = x \ggg n$ — x бүтін санының алдыңғы 0 бар екілік түрінің оңға қарай n битке жылжуы (Жылжудан кейін бірінші биттің орнына 0 жазылады және сан оң болады);

Барлық өрнек (бізде — “ x ”) сол жақта 32-биттік бүтін санға айналады, осыдан кейін керекті жылжу орындалып, алынған сан өрнек– шешімнің типіне (бізде бұл тағы да “ x ”) келтіріледі және меншіктеу орындалады.

Мысалдар:

$9 \ll 2$ 36 береді, себебі 1001 жылжуы (екілік жүйедегі 9 саны) 2 битке солға жылжиды береді 100100, яғни 36.

Жетпей жатқан биттер 0-мен толтырылады. $9 \gg 2$ береді 2, себебі 1001 жылжуы (екілік жүйедегі 9 саны) 2 битке оңға жылжиды береді 10, яғни 2 Жетпей жатқан биттер 0-мен толтырылады.

“&” – биттік AND – “ЖӘНЕ”;
“|” – биттік OR – “НЕМЕСЕ”;

"^" — биттік XOR — "ИСКЛЮЧАЮЩЕЕ НЕМЕСЕ".

Барлық операциялар сандардың екілік берілуімен орындалады, алайда нәтиже ондық түрде беріледі.

Мысалдыр:

```
15&9 9 қайтарады, себебі (1111) AND (1001) 1001-ге тең;  
15|9 15 қайтарады, себебі (1111) OR (1001) 1111-ге тең;
```

15^9 6 қайтарады, себебі (1111) XOR (1001) 0110-ге тең.

Логикалық өрнектер:

```
"&&" — логикалық AND — "ЖӘНЕ";  
"||" — логикалық OR — "НЕМЕСЕ";
```

"!" — логикалық NOT — "ЕМЕС".

Мысал:

$(a > b) \&\& (b \leq 10) || (a > 10)$ JavaScript-те әрқашан логикалық өрнектердің қысқартылған тексерілуі орындалады: $B1 = \text{false}$ болғандағы $B1 \&\& B2$ операндасында $B2$ бағасы орындалмайды және false мәні қайтарылады. $B1 = \text{true}$ болғанда $B1 || B2$ true болып табылады. Бұл кезде логикалық өрнектің анализі солдан оңға қарай болады және барлық өрнектер бағаланғаннан кейін нәтиже қайтарылады. Сол себепті $B2$ ретінде функция болса, ол шақырылмайды және егер ол қосымша эффектілер беру керек болса онда бұл қателікке алып келуі мүмкін.

Салыстыру операторлары

```
"=" - "тең";  
">" - "артық";  
"<" - "кем"; ">=" - "артық немесе тең";
```

"<=" - "кем немесе тең"; "!=" - "тең емес".

Салыстыру операторлары тек сандық емес, сондай-ақ қатарлы өрнектерге де қолданылады. Бұл жағдайда қатарлар тең болып есептеледі, егер олардың барлық символдары сәйкес келсе және бірдей реттілікте болса (бос орын символ ретінде есептеледі). Егер қатарлар түрлі ұзындықта болса, онда ұзындығы үлкен қатар үлкені болып табылады. Егер олардың ұзындықтары сәйкес келсе, онда солдан оңға қарай көру кезінде номері үлкен символ үлкені болып табылады ($a < b < c < . < z < A < . < Z$).

Қатарларды қаттауға болады, егер $S1 = \text{"это"}$, $S2 = \text{"моя строка"}$, онда $S1 + S2$ тең болады `"это моя строка"`.

Операторлардың приоритеті (кішілерден бастап; бір қатарда үлкендік бірдей):

`"+"`, `"-"`, `"*"`, `"/"`, `"%"`, `"<<="`, `">>="`, `">>>="`, `"&="`, `"^="`, `"|="`.

Операциялардың үлкендігі

```
Шартты өрнек: "?:";  
Логикалық "НЕМЕСЕ": "||";  
Логикалық "ЖӘНЕ": "&&";  
Биттік "НЕМЕСЕ": "|";  
Биттік "XOR": "^";  
Биттік "И": "&";
```

Тендікке салыстыру `"="`, `"!="`; Салыстыру : `"<"`, `"<="`, `">"`,

```
">=";  
Биттік жылжу: "<<", ">>", ">>>";  
қосу, азайту: "+", "-"; көбейту, бөлу: "*", "/";  
терістеу, инкремент, декремент: "!", "~", "++", "--";
```

жақшалар: `"( )"`, `"[ ]"`.

Тақырып бойынша сұрақтар:

1. Арифметикалық әрекеттер.
2. Меншіктеу.
3. Жадыда сақталған сөздер
4. Айнымалылар.
5. Айнымалының жариялану форматы

ДӘРІС 05 / ТЕОРИЯ

Дәріс 5. Шартты оператор: if

Видеосабақ: Java программалау тілі / Сабақ #2. Шартты оператор

Қазақша видеосабақ дәрістің негізгі ұғымдарын толықтырады.

Жоспар

5.1. Салыстыру операциялары. Биттік операциялар.

Меншіктеу операциялары. Шартты операция. Өрнектер.

5.2. Операторлар. Меншіктеу операторы. Шартты оператор.

Цикл операторлары. Continue, break операторы. Вариант операторы.

5.3. Java функциялары.

1. Java тілінің операциялары. Java тілінде шамалар бойынша бүтін сандарды салыстырудың 6 операциясы бар:

- үлкен > ;
- кіші < ;
- үлкен немесе тең >= ;
- кіші немесе тең <= ;
- тең == ;
- тең емес != .

Қосарланған символдар бос орынсыз жазылады, бірақ олардың орнын ауыстыруға болмайды, мынадай => жазба дұрыс емес, >= болып жазылуы керек.

Салыстыру нәтижесі — true немесе false мәні болады, мысалы, мынадай салыстыру 3 != 5 мәні – true, ал мынадай салыстыру 3 == 5 нәтижесі false болады.

Күрделі салыстыруларды жазу үшін логикалық операцияларды қолдану керек. Мысалы, $a < x < b$ сияқты теңсіздіктерді тексеру кезінде төмендегідей түрде $(a < x) \&\& (x < b)$ немесе $a < x \&\& x < b$ жазу керек.

Меншіктеу операциялары. Қарапайым меншіктеу операциялары (simple assignment operator) = теңдік таңбасы арқылы жазылады, оның сол жағында айнымалы, ал оң жақта осы айнымалыға сәйкес мән тұрады:

```
x = 3.5, y = 2 * (x - 0.567) / (x + 2),
b = x < y,
bb = x >= y && b. x = 3.5,
y = 2 * (x - 0.567) / (x + 2), b = x < y,
bb = x >= y && b.
```

Шартты операция. Бұл операция үш операндтан тұрады.

Алдымен кез-келген логикалық өрнек жазылады, оның нәтижесі true немесе false болуы тиіс, содан кейін сұрақ белгісі, ал содан кейін қос нүктемен бөлінген екі өрнек жазылады.

Мысалы, $x < 0 ? 0 : x$ $x > y ? x - y : x + y$ Осыны

пайдаланып, былай жаза аламыз: $n == 0 ? m : m / n$;

Шартты операция шағын тармақтарды жазуға өте ыңғайлы болып келеді.

Өрнектер (expressions) тұрақтылардан, айнымалылардан және соларға қолданылатын операциялардан, жай жақшалардан тұрады.

Өрнектерді есептеу кезінде 4 ереже орындалады:

1. Бір приоритетті операциялар солдан оңға қарай есептеледі: $x + y + z$ өрнегі есептеу былай орындалады $(x + y) + z$.

2. Меншіктеу операциясы оңнан солға қарай есептелінеді: $x = y = z$ өрнегін есептеу былай орындалады $x = (y = z)$.

2. Сол жақтағы операнд оң жақтағыдан ерте есептеледі.

3. Операндтар операцияға дейін алдын ала есептеледі.

Құрамалы меншіктеу операциясының орындалуы алдында сол жақтың мәні оң жақта қолдану үшін сақталады. Төмендегі мысалдар бастапқы үш ереженің қолдану ерекшеліктерін көрсетеді: `int a = 3, b = 5; b + (b`

`= 3)` өрнегінің нәтижесі 8 болады;

`(b = 3) + b` өрнегінің нәтижесі 6 болады.

Төртінші ережені арқылы `b += a += b += 7` өрнегінің мәні 20-ға тең болады.

Тіл операторлары. Операторлар (statements) – алгоритмдерді жазу құралы. Java тілінде goto операторы жоқ. Барлық операторлар нүктелі үтірмен аяқталады. Блок.

Операторлар блогы программаның мәтінінің оқылуын жақсарту үшін және айнымалылардың әсер ету аймағын шектеу үшін жиі қолданылады.

Мысалы, $\{x = 5; y = ?;\}$. Бос блоқты да жазуға болады: $\{ \}$.

Шартты оператор Шартты оператор (if-then-else statement) Java тілінде былай жазылады:

```
if (логикалық өрнек) оператор1;  
else оператор2;
```

Шартты оператор қысқартылған түрде болуы да мүмкін (if- then statement):

```
if (логикалық өрнек) оператор1;
```

Логикалық өрнек false болған жағдайда еш нәрсе орындалмайды.

Java функциялары. Math класы математикалық есептеулерді орындайтын статикалық әдістер жиынтығынан тұрады : Math.PI ($\pi = 3.14159$) Трансцендентті функциялар. Төменде double (радианмен берілген бұрыш) типті параметрлерді қабылдайтын және

```
double типті параметрлерді қайтаратын үш әдіс берілген:  
Math.sin(x) Math.cos(x) Math.tan(x)
```

Экспоненциалды функциялар. Math класында экспоненциалды әдістер анықталған (double типті аргументтер және нәтиже):

Math.exp(x)- e^x Math.log(x,y)- $\ln x$ Math.pow(x)- x^y Math.sqrt(x) Дөңгелектеу функциялары. Math класында келесідей дөңгелектеу әдістері анықталған: Math.abs(x)- $|x|$

```
Math.max(x,y)- x пен y мәнінің ең үлкенін қайтарады;  
Math.min(x,y)- x пен y мәнінің ең үлкенін қайтарады;
```

Math.ceil(x)- x-ке тең немесе x-тен үлкен бүтін кіші санды

қайтарады;

Math.floor(x)- x-ке тең немесе x-тен кіші бүтін үлкен санды

қайтарады;

Math rint(x) – x-ке жақын бүтін мәнді қайтарады.

Math.round(x)-жақын бүтін int мәніне дейін дөңгелектелген x-ті қайтарады (аргументі float, нәтижесі

```
int)
```

Қосымша келесі әдістер анықталған:

`Math.random()`-0 мен 1 аралығындағы кездейсоқ санды қайтарады (нәтижесі `double`) `Math.toRadians(x)`-градусты радианға ауыстырады (аргументі де, нәтижесі де `double`) `Math.toDegrees(x)`-радианды градусқа ауыстырады (аргументі де, нәтижесі де `double`)

ДӘРІС 06 / ТЕОРИЯ

Дәріс 6. Циклдік операторлар: for және while

Видеосабақ: Алгоритмдер қазақша | #2 — Сұрыптау алгоритмдері

Сұрыптау алгоритмдері арқылы циклдердің практикалық қолданылуы көрсетіледі.

Жоспар

6.1 while операторы:

6.2 for операторы:

while операторы:

```
while (өрнек) оператор;
```

Екінші циклдік оператор do-while do оператор while

```
(өрнек);
```

Алдымен оператор орындалады, содан кейін логикалық өрнек тексеріледі.

Логикалық өрнек true болғанша цикл қайталана береді.

for операторы:

```
for ( өрнек1; өрнек; өрнек2) оператор;
```

Ауыстыру операторы бірнеше бағыт бойынша тармақталуды орындайды:

```
switch (өрнек) { case өрн1:
```

```
оператор1 case өрн1:
```

```
оператор2 case өрнN:
```

```
операторN default:
```

```
операторDef }
```

Мысалы:

Switch (dayOfWeek)

```
{  
case 1: case 2: case 3: case 4: case 5:  
System.out.println("Week-day");  
break;
```

```
case 6: case 7:
```

```
System.out.println("Week-end");  
break;  
default:  
System.out.println("Unknown day");  
}
```

Әрбір жолдағы әрекеттерді break операторымен аяқтауды ұмытпау керек.

for циклы

```
for (инициализация секциясы; шарт секциясы;
```

```
есептегіштердің өзгеру секциясы)
```

```
{
```

```
бекіту
```

```
}
```

Секциялардың әрбірі бос болуы мүмкін. Инициализация секцияларында және есептегіштердің өзгеруінде өрнектердің кезектілігін үтірлермен бөліп жазуға болады.

Циклдың орындалуы келесі түрде болады. Бірінші болып инициализация секциясы орындалады. Содан кейін шарт тексеріледі. егер шарт true болса, онда цикл денесі орындалады, ал одан кейін есептегіш өзгеру секциясы.

Егер шарт false болса, онда циклдың орындалуы тоқтатылады.

Мысал

```
function HowMany(SelectObject)
{
var number Selected=0
for (i=0; i< SelectObject.options.length; i++)
{
if (SelectObject.options[i].selected==true) number Selected++;
}
return number Selected;
}
```

for операторы объектінің барлық өрістерін жинау үшін қолданылады (объектілік модель туралы бөлімді қараңыз) Синтаксис:

For (in объект айнымалысы)

```
{
```

өрнек

```
}
```

Бұл жағдайда объектегі көрсетілген айнымалының барлық мүмкін мәндері жиналады және олардың әрқайсысы үшін бекіту орындалады.

Мысал: function student(name, age, group)

```
{
this.name=name; this.age=age;
this.group=group;
}
function for_test(myObject)
{
for(var i in myObject)
{
document.write("i="+i+" => "+myObject[i]+"  
");
}
}
Helen=new student("Helen K.", 21, 409);
for_test(Helen);
```

Экранға шығару: i=0 => Helen K. i=1 => 21 i=2 => 409 while циклы

```
while (шарт)
{
```

өрнек

```
}
```

While циклының орындалуы шартты тексеруден басталады.

Егер ол true болса, онда цикл денесі орындалады, болмаса басқару циклдан кейінгі операторға беріледі.

While операторын қолдану мысалы:

```
n1=10
n=0 x=0
while(n<n1)
{
n=n+1;
x=x+n;
}
```

Циклдардың орындалуын ұзу операторлары for немесе while операторларының ағымдық циклының орындалуын тоқтату үшін break операторы қолданылады.

Break операторын қолдану мысалы:

```
function test(x)
{
var j=0; var sum=0;
while (n<n1)
{
If (n==x)
{
sum=x;
break;
}
sum=sum+n;
n=n+1;
}
return sum;
}
```

Continue операторы ағымдық итерацияның орындалуын for немесе while ішінде тоқтатады және келесі итерацияның басына өтуді шақырады.

Continue операторын қолдану мысалы:

```
function test1(x)
{
var j=0;
while (n<n1)
{ if (n==x)
{ sum=x;
continue;
}
sum=sum+n;
n=n+1;
}
return sum;
}
```

Мысал. Квадратты теңдеудің түбірлерін табу

```
class Kwur1
{
public static void main (String[] args) { double
a=1,b=2,c=3,d,d1, a1;
d=b*b-4*a*c;
if (d<0) { d1=0.5*Math.sqrt(-d)/a;
a1=-0.5*b/a;
System.out.println ("x1="+a1+"+i"+d1+", x2="+a1+"-i"+d1);
}
else
{ d=0.5*Math.sqrt(d)/a;
a=-0.5*b/a;
System.out.println ("x1="+a+d+", x2="+a-d));
}
}
```

Циклдің негізгі операторы– while операторы келесі түрде болады: while (логӨрнек) оператор.

Алдымен, логӨрнек логикалық өрнегі есептелінеді; егер оның мәні true болса, онда циклді құрайтын оператор орындалады. Одан кейін қайтадан логӨрнек есептеледі және оператор әрекет етеді, яғни ол false мәні шыққанша орындалады. Егер лог Өрнек басында false-қа тең болса, онда оператор бір рет те орындалмайды. Алдын ала тексеру циклдің орындалуының қауіпсіздігін қамтамасыз етеді, толып кетуден, нольге бөлуден және басқа да қолайсыздықтардан сақтайды. Сондықтан while операторы негізгі болып табылады, ал кейбір тілдерде ол тіптен жалғыз циклдік оператор болып табылады.

Циклдегі оператор бос болуы мүмкін, мысалы, келесі код бөлігі:

```
int i = 0;
double s = 0.0;
while ((s += 1.0 / ++i) < 10);
```

s гармоникалық қосындысы 10 мәніне жететіндей i көбейтіндінің санын есептейді. Шексіз цикл ұйымдастыруға да болады : while (true) оператор Егер циклге бірнеше оператор жазу керек болса, онда операторлар блогін {} қолдануға болады.

Циклдің екінші операторы - do-while операторы келесі түрде болады: do оператор while (логӨрн). Мұнда ең алдымен оператор орындалып, содан кейін логикалық өрнек логөрнек есептеледі. Цикл логөрнек true-ге тең болғанға дейін орындалады. Continue операторы тек цикл операторларында қолданылады. Ол екі түрге ие. Бірінші түрі тек қана continue сөзінен тұрады және жедел түрде келесі цикл итерациясына өтуді жүзеге асырады.

Код үзіндісінде continue операторы нольге бөлуден сақтандырады:

```
for (int i = 0; i < N; i++)
{ if (i == j) continue;
  s += 1.0 / (i - j);
}
```

Break операторы цикл операторларында және осы конструкциялардан жылдам шығу үшін нұсқа операторында қолданылады. Нұсқа операторы switch бірнеше бағыт бойынша тармақталуды қамтамасыз етеді. Әр тармақ тұрақтымен немесе қандай да бір бүтін типтің (long типінен басқа) тұрақтылық өрнегімен белгіленеді және таңдалады.

Барлық конструкция мына түрде болады:

switch (БүтінӨрн)

```
{
```

case констӨрн1:

оператор1 case констӨрн2:

оператор2 case констӨрнN:

операторN default:

операторDef

```
}
```

Тақырып бойынша сұрақтар:

1. Саластыру операциялары
2. Шартты операциялар
3. Циклдік операциялар
4. while циклы
5. Идентификаторлар.
6. Тұрақтылар.
7. Қалқымалы нүктелі тұрақтылар
8. Айнымалылар.

ДӘРІС 07 / ТЕОРИЯ

Дәріс 7. Тізімдер

Видеосабақ: Лекция 6 Жолдармен жұмыс

Бастапқы дәріс String және StringBuffer кластарына арналған; видео жолдармен жұмысты толықтырады.

Дәрістің бастапқы «Тізімдер» атауы сақталған. Негізгі мәтін String және StringBuffer арқылы сөз тіркестерімен жұмыс істеуге арналған; білімді бақылау осы мазмұн бойынша құрастырылған.

Жоспар

7.1. String класы 7.2. String Buffer класы 7.3. String класы

Java жүйелік кітапханасының java. lang пакетінде анықталған, сөз тіркестерімен жұмыс істеуге арналған String және StringBuffer кластары бар. Бұл кластар final модификаторымен жарияланған, сондықтан олар ары қарай пайдаланылатын сөз тіркестерінің қасиеттері бар өз кластарын жасай алмайды.

String класы бірнеше конструкторларды сүйемелдей алады,

олар: String(), String(String str), String(char[] c), String(byte asciii char[]).

Бұлар класс объектілерін инициалдау үшін пайдаланылады.

String класы объектісін класс нұсқаушына бір тіркестің

мәнін меншіктеу арқылы немесе конструктор мен new сөзін пайдалану жолымен жасауға болады, мысалы:

```
String s1 = "Строка";  
немесе String s2 = new String("Строка");  
String класының сөз тіркестерімен жұмыс істеуге арналған
```

тәсілдері

```
String s=new String(); - бос тіркес құрастыру;  
char chars[]={‘K’,‘A’, ‘З’, ‘H’,‘Y’};  
String s1=new String(chars); - String типті объект  
құрастырып, оған мән беру;
```

Немесе String s2="KNU"; int length(); - сөз тіркестерінің ұзындығын анықтау

```
String s1 = "Академия " + age + " лет."; - жолдарды  
біріктіру;
```

```

Мысал: import java.io.*;
class lengthDemo { public static void main(String args[])
throws IOException
{
PrintWriter pw=new PrintWriter (new OutputStreamWriter
(System.out,"Cp866"), true);
int age=20;
String s = "Алматы";
pw.println(s.length());
String s1 = "Академия " + age + " жыл.";
pw.println (s1);
}
}

```

Нәтижесі: 6 Академия 20 жыл.

Сөз тіркестерінен символды қиып алу тәсілдері char

```

charAt(int n);
тіркестен нөмірі көрсетілген символды қиып алу;

```

VoidgetChars (int start, intend, char buf, int n); -бірнеше қатар орналасқан символдарды қиып алу Start – бірінші символ нөмірі End– соңғы символ нөмірі Buf –қиып алынған символдарды меншіктейтін жиым (массив) n – қиып алынған символдардың жиым элементтерінің қандай нөмірінен бастап жазылатындығын көрсетеді

```

Символдарды қиып алу мысалдары import java.io.*;
class getCharsDemo {
public static void main(String args[]) throws IOException {
PrintWriter pw=new PrintWriter (new OutputStreamWriter (
System.out, "Cp866"), true);
String s = "Демонстрация метода get Chars на примере.";
// заданная строка pw.println(s);
pw.println(s.charAt(3));
//печать 3-символа int start = 13;
int end = 18; char buf[] = new char[end - start];
s.getChars (start, end, buf, 0);
pw.println (buf);
}
}

```

Мысал нәтижесі : Демонстрация метода get Chars на примере. (3 орындағы символ – тіркестің 4-символы) метод (тіркестің 13 -18- символдары, яғни 14-19-орындағы символдар)

Сөз тіркестерін салыстыру Boolean equals(object s); - екі тіркесті салыстыру, салыстыру кезінде бас және кіші

әріптер әр түрлі болып саналады;

`Boolean equalsIgnoreCase(object s);` - екі тіркесті салыстыру, салыстыру кезінде кіші және бас әріптердің

айырмашылығы жоқ;

Сөз тіркестерін реттеу (сортировка) `Int compareTo(String str);` - `str` тіркесін регистрді есепке ала отырып, алфавит бойынша (лексикографикалық) реттейді.

`Int compareToIgnoreCase (String str);` - `str` тіркесін регистрді есепке алмай – бас әріп пен кіші әріп бірдей), алфавит бойынша (лексикографикалық) реттейді. Егер тәсілдің қайтарған мәні теріс болса, онда салыстырылған тіркес параметр-тіркестен кіші, оң болса үлкен, 0 болса тең болғаны.

Сөз тіркестеріндегі символды іздеу `int indexOf(char ch);` - ізделінетін символдың тіркестегі бірінші кездескен орнын

анықтайды (басынан бастап іздеу);

`Int lastIndexOf(char ch);` - ізделінетін символдың тіркестегі

соңғы кездескен орнын анықтайды (артынан іздеу);

Осы тәсілдердің көмегімен бір символ емес сол тіркестегі ішкі тіркестің бірінші және соңғы символдарының орнын да анықтауға болады. Сонымен бірге ізделінетін символды немесе ішкі тіркесті нешінші позициядан бастап іздеу керек екендігін де көрсетуге болады.

Сөз тіркестерін түрлендіру

`String substring (int startindex, int endindex);` - берілген сөз тіркесінен бірнеше символды немесе ішкі тіркесті қиып алу;
`String concat(string str);` - екі сөз тіркесін біріктіру;
`String replace(char original, char replacement);` - берілген тіркестері бір символды көрсетілген символмен алмастыру;
`String toLowerCase();` - бас әріптерді кіші әріптермен алмастыру;
`String toUpperCase();` - кіші әріптерді бас әріптермен алмастыру;
`String trim();` - берілген сөз тіркесінің алдындағы және соңындағы бос орындарды алып тастайды;

`StringBuffer` класы `StringBuffer` класының `String` класынан айырмашылығы бастапқы берілген жолдың ұзындығын өзгертуге болады.

`StringBuffer` класының тәсілдері `length();` - `StringBuffer`

ағымдағы ұзындығын анықтау;

`setLength(int len);` - буфердің ұзындығын алдын ала

орнату;

`capacity();` - `StringBuffer`-ге бөлінген жады көлемін анықтау;

`ensureCapacity(int capacity );` - буферге бөлінетін жады

көлемін алдын ала анықтау;

`charAt(int n);` - `n`-ші орындағы символды анықтау;

`setCharAt(int n, char ch);` - `n`-ші орындағы символды `ch`

символына алмастыру;

`insert(int index, string str);` - берілген сөз тіркесі ішіне `str` тіркесін көрсетілген `index` -тен бастап енгізу `reverse();` -

берілген тіркесті кері оқу;

`delete (int startindex, int endindex );` - берілген тіркестері `startindex` және `endindex` аралығындағы символдарды алып тастау `deleteCharAt(int k);` - `k` позициясындағы символды

өшіру;

`replace (int startindex, int endindex, string str );` - берілген тіркестері `startindex` және `endindex` аралығындағы символдарды `str` ішкі тіркесімен алмастыру.

Тақырып бойынша сұрақтар:

1. `String` класы
2. `StringBuffer` класы
3. `String` класы
4. Шартты операциялар
5. Циклдік операциялар
6. `while` циклы

ДӘРІС 08 / ТЕОРИЯ

Дәріс 8. Рекурсия және екі өлшемді массивтер

Видеосабақ: Java. ООП. Матрицаны құру және көбейту программасы

Java тілінде матрица құру және көбейту. Рекурсияға қосымша видео төменде берілген.

Қосымша: функциялар және рекурсия

Жоспар

8.1 Массивтер.

8.2 Массивтерді өңдеу.

Программалауда массив дегеніміз — оперативті жадының аралас ұяшықтарында сақталатын біртекті айнымалылар жиынтығы. Java тілінде массивтер сілтемелі типтерге жатады және өзгеше сипатталады, бірақ сілтемелі типтерге тән. Сипаттау үш кезеңге бөлінеді.

Бірінші кезең — жариялау (declaration). Бұл кезеңде массив типінен тұратын, массивке сілтеме (reference) типті айнымалы ғана анықталады. Ол үшін массив элементтері типінің аты жазылады, тік жақшамен жай айнымалы емес, массивке сілтеме жарияланатыны көрсетіледі, мысалы,

```
double[] a, b;
```

Мұнда екі айнымалы анықталған — double типті массивтерге a және b сілтемелері. Тікелей атынан кейін тік жақша қоюға болады. Мұны жай айнымалылар анықтамаларының арасында жасауға болады: `int I = 0, ar[], k = -1;` Мұнда i және k бүтін типті екі айнымалы анықталған, және ar бүтін типті массивке сілтеме жарияланған.

Екінші кезең — анықтама (installation). Бұл кезеңде массивтің ұзындығы деп аталатын элементтер саны көрсетіледі, жедел жадыда массив үшін орын бөлінеді, айнымалы-сілтеме массив адресін алады. Осы әрекеттердің бәрі Java тілінің тағы бір операциясы — тип операциясында көрсетілген объект үшін

жедел жадыда аймақ бөлетін және нәтиже ретінде осы аймақтың адресін қайтаратын `new` типті операциясымен орындалады. Мысалы, `a = new double[5]; b = new double[100]; ar = new int[50];` Массив индекстері әрқашан 0-ден басталады. А массиві бес айнымалыдан тұрады `a[0], a[1], ..., a[4]`. `A[5]` элементі массивте жоқ.

Индекстерді `long` типінен басқа кез келген бүтінсанды өрнектермен беруге болады, мысалы, `a[i+j]`, `a[i%5]`, `a[++i]`.

Java-ның орындаушы жүйесі осы өрнектердің мәндері массив ұзындығының шекарасынан аспауын бақылайды.

Үшінші кезең —инициализация. Бұл кезеңде массив элементтері бастапқы мәндерді қабылдайды.

Мысалы,

```
a[0] = 0.01;
a[1] = -3.4;
a[2] = 2.89;
a[3] = 4.5;
a[4] = -6.7;
for (int i = 0;
i < 100; i++) b[i] = 1.0 / i;
for (int i = 0; i < 50; i++) ar[i] = 2 * i + 1;
```

Бірінші екі кезеңді біріктіруге болады: `doublet] a = new`

```
double[5], b = new double[100];
int i = 0, ar[] = new int[50], k = -1;
```

Бастапқы мәндерді де бірден констант немесе константалық өрнектер түрінде үтір арқылы ирек жақшаға алып жазуға болады. Бұл жағдайда массив элементтерінің санын көрсету

```
міндетті емес, ол бастапқы мәндер санына тең болады;
double[] a = {0.01, -3.4, 2.89, 4.5, -6.7};
```

Екінші және үшінші кезеңді де біріктіруге болады:

```
a = new doublet] {0.1, 0.2, -0.3, 0.45, -0.02};
```

Бірден `new` операциясының нәтижесін қолдана отырып, аты жоқ массив құруға да болады, мысалы, мынадай:

```
System.out.println(new char[] {'H', 'e', 'l', 'l', 'o'});
```

Массивке сілтеме сипатталған массивтің бөлігі болмайды, оны сондай типті меншіктеу операциясы бар басқа массивке айналдыруға болады.

Мысалы, $a = b$ меншіктегеннен кейін a және b екі сілтеме де

double типті 100 сандық айнымалыдан бір массивке ғана

көрсетеді және бір адрестен ғана тұрады. Сілтеме жедел жадының ешбір адресін көрсетпейтін null "бос" мәнін меншіктей алады: $ar = null$; Содан кейін берілген сілтеме көрсеткен массив оған басқа сілтемелер болмаса жоғалады.

Қарапайым меншіктеу операциясынан басқа, сілтемелермен тағы тек теңдікке салыстыруға болады, мысалы, $a = b$, және теңсіздік, $a != b$. Бұл жағдайда сілтемелердегі адрестер салыстырылып қаралады, біз олардың тек бір ғана массивке жіберілгенжіберілмегенін білуімізге болады. Java-да массивтер оларға сілтемелер статикалық берілсе де, әрқашан динамикалық анықталады.

Массивке сілтемеден басқа, әр массив үшін автоматты түрде `length` атты бүтін тұрақты анықталады. Ол массив ұзындығына тең. Әр массив үшін бұл тұрақтының аты массив атымен нүкте арқылы айқындалады. Сонымен, біздің анықтамалардан кейін, тұрақты `a.length` 5 -ке тең, тұрақты `b.length` 100 -ге тең, ал `ar.length` 50-ге тең. А массивінің соңғы элементін мына түрде жазуға болады: `a[a.length - 1]`, `a[a.length - 2]` және т. б.

Массив элементтері әдетте мына цикл түрінде жинақталады:

```
double aMin = a[0], aMax = aMin;
for (int i = 1;
i < a.length; i++) {
if (a[i] < aMin) aMin = a[i]; if (a[i] > aMax) aMax = a[i];
}
double range = aMax - aMin;
```

Мұнда массив мәндерінің диапазоны есептеледі. Java-да массив элементтері қайтадан массивтер болуы мүмкін:

```
char[] [] c;
баламалы char c[] c[]; немесе char c[][];
```

Одан кейін сыртқы массивті анықтаймыз:

```
c = new char[3][];
```

Мұнда `c` – үш элемент-массивтерден тұратын массив екені белгілі болады. Енді оның элементмассивтерін анықтаймыз:

```
c[0] = new char[2];
c[1] = new char[4];
c[2] = new char[3];
```

Осы анықтамалардан кейін айнымалы `c.length` 3-ке тең, `c[0].length` 2-ге, `c[1].length` 4-ке және `c[2].length` 3-ке тең.

Ақырында бастапқы мәндерді береміз `c [0] [0] = 'a'`,

```
c[0][1] = 'r',
c[1][0] = 'r',
c[1][1] = 'a',
c[1][2] = 'y' және т.б.
```

Мысал 1. Паскаль үшбұрышы `class PascalTriangle{`

```
public static final int LINES = 10;
// Тұрақтылар осылай анықталады public static void
main(String[] args) {
    int[][] p, = new int [LINES] [];
    p[0] = new int[1];
    System.out.println (p [0] [0] = 1);
    p[1] = new int[2];
    p[1][0] = p[1][1] = 1;
    System.out.println (p[1][0] + " " + p[1][1]);
    for (int i = 2; i < LINES; i++)
    {
        p[i] = new int[i+1];
        System.out.print ((p[i][0] = 1) + " ");
        for (int j = 1; j < i; j++)
            System.out. print ( (p[i] [j] =p[i-1][j-1] +p[i-1][j]) + " ");
        System.out. println (p [ i] [i] = 1) }
    }
}
```

```
Мысал 2. Массивтерді реттеу import javagently.*;
import java.io.*;
class Mas1
{
    final int n=3;
    Mas1() throws IOException
    {
        String f[],pr[];
        int gr[], rez[], i, k, c;
        f =new String[n];
        pr=new String[n];
        gr=new int[n];
        rez=new int[n];
        Stream in=new Stream (System.in);
        For (i=0; i<n; i++)
        {
            f[i]=in.readString();
            pr[i]=in.readString();
            gr[i]=in.readInt();
            rez[i]=in.readInt();
        }
        For (i=0; i<n-1; i++) for(k=i; k<n; k++) if (rez[k]>rez[i])
        {
            c=rez[i];
            rez[i]=rez[k];
            rez[k]=c;
        }
        System.out.println();
        For (i=0; i<n; i++) System.out.println(rez[i]);
    }
    public static void main(String[] arg) throws IOException { new
    Mas1();
}
}
```

Тақырып бойынша сұрақтар:

1. Массивтер
2. Бір өлшемді массивтер
3. Екі өлшемді массивтер
4. String класы
5. StringBuffer класы
6. String класы

ДӘРІС 09 / ТЕОРИЯ

Дәріс 9. Функцияны хабарлау. Жиындар

Видеосабақ: Пайдаланушы функциялары мен процедуралары. Рекурсия

Қазақша видео функциялар мен рекурсия идеясын түсіндіреді; оның синтаксисін Java әдістеріне бейімдеңіз.

Мәтінде JavaScript функциялары салыстырмалы мысал ретінде кездеседі. Java мен JavaScript — бөлек тілдер. Java әдістерінің дұрыс баламалары мен факториалдың тоқтау шарты қосымшада берілген. «Жиым» — массив; ол Set жиынымен бір ұғым емес.

Жоспар

9.1 Функциялар

9.2 Java функциялары.

9.3 Жиымдар.

Функциялар JavaScript-те C және Java-дағы секілді процедуралар және процедура функциялар функциялар деп аталады. Функция декларациясы:

```
Function резервтелген сөзінен;  
Функция атынан;
```

Домалақ жақшада үтірлермен бөлінген аргументтер

```
тізімінен;
```

Фигуралық жақшадағы функция денесінен тұрады.

```
function myFunction(arg1, arg2, ...)  
{
```

Операторлардың кезектілігі

```
}
```

myFunction — функция аты, arg1, arg2 — формальді параметрлердің тізімі.

Мысал:

```
function Factorial(n)
{
  If ((n<0)|| (round (n)!=n))
  {
```

Alert ("Factorial функциясы "+n аргументінде

```
анықталмаған);
return NaN;
}
else
{
  result= (n*Factorial (n-1));
  return result;
}
}
return резервтелген сөзінің көмегімен функция мәнді
```

қайтармауына болады.

Мысал:

```
function Greeting(s)
{
  document.write ("Hello,"+s+"!");
}
```

Функцияны шақыру нақты параметрлермен орындалады.

Мысал:

```
Factorial (2+1);
```

— қандай да бір өрнектің ішінде 6 қайтарады, ал Greeting

```
("world");
```

—экранға "Hello, world!" қатарын шығарады..

Әрбір функция, мысалы myFunction, аты myFunction болатын объект болып табылады, оның аргументтері аты arguments массиві ретінде сақталады, аргументтерге былай қол жеткізуге: myFunction.arguments[i], i—аргумента номері (нумерация 0-ден басталады).

Нақты параметрлердің саны тең немесе функцияның сипатталуындағы формальді параметрлердің санынан көп болуы керек. Функцияны шақыру

кезінде берілген аргументтердің шынайы саны `myFunction.arguments.length` өрісінде сақталады және осы өрістің меншіктелуі арқылы динамикалық өзгеруі мүмкін.

Мысалы:

Құжатта HTML форматындағы тізімді шығару. Бұл жерде бірінші аргумент (`ListType`) реттелген тізім үшін "о" немесе "О" мәні, ал реттелмеген үшін "u" или "U" мәнге ие болады.

```
function myList (ListType)
{
document.write("<" + ListType + "L");
for (var i=1; i < myList.arguments.length; i=i+1)
{
document.write("<LI>" + myList.arguments[i]);
}
document.write("</" + ListType + "L>");
}
```

Текстте осы функцияның HTML құжатын шақыру:

```
<script>
```

```
myList("0", "один", 2, "3")
```

```
</script>
```

Тексті шығаруға алып келеді:

бір

Java функциялары.

`Math` класы математикалық есептеулерді орындайтын статикалық әдістер жиынтығынан тұрады : `Math.PI` ($\pi=3.14159$) Трансцендентті функциялар. Төменде `double` (радианмен берілген бұрыш) типті параметрлерді қабылдайтын және

```
double типті параметрлерді қайтаратын үш әдіс берілген:
Math.sin(x) Math.cos(x) Math.tan(x)
```

Экспоненциалды функциялар. `Math` класында экспоненциалды әдістер анықталған (`double` типті аргументтер және нәтиже): `Math.exp(x)` - `ex` `Math.log(x,y)`-`lnx` `Math.pow(x)-xy` `Math.sqrt(x)`- Дөңгелектеу функциялары. `Math` класында келесідей дөңгелектеу әдістері анықталған: `Math.abs(x)`-`|x|`

`Math.max(x,y)` - x пен y мәнінің ең үлкенін қайтарады;
`Math.min(x,y)` - x пен y мәнінің ең үлкенін қайтарады;

`Math.ceil(x)` - x -ке тең немесе x -тен үлкен бүтін кіші

санды қайтарады;

`Math.floor(x)` - x -ке тең немесе x -тен кіші бүтін үлкен

санды қайтарады;

`Math rint(x)` – x -ке жақын бүтін мәнді қайтарады.

`Math.round(x)` - жақын бүтін `int` мәніне дейін дөңгелектелген x -ті қайтарады (аргументі `float`, нәтижесі `int`).

Қосымша келесі әдістер анықталған:

`Math.random()` – 0 мен 1 аралығындағы кездейсоқ санды қайтарады (нәтижесі `double`) `Math.toRadians(x)`- градусты радианға ауыстырады (аргументі де, нәтижесі де `double`) `Math.toDegrees(x)` - радианды градусқа ауыстырады (аргументі де, нәтижесі де `double`) Жиымдар. Жиым — жедел жадыдағы көршілес ұяшықта сақталынатын бір типті айнымалылар жиынтығы. Java тіліндегі жиымдар сілтемелі типке жатады.

Сипаттама үш кезеңде жүргізіледі. Жиымды хабарлау.

Бірінші кезең — хабарлау (`declaration`). Бұл кезеңде тек жиымға сілтеме типіндегі айнымалылар анықталады.

Мысалы, `double[] a, b`; екі айнымалы анықталған — `double` типті a мен b жиымдары.

Екінші кезең — анықтау (`installation`). Бұл кезеңде жиым элементтерінің саны көрсетіледі, айнымалы жиым адресін

алады. Мысалы, `a = new double[5];`

Элементтері: `a[0]`, `a[1]`, `a[2]`, `a[3]`, `a[4]`.

Үшінші кезең — инициалдау (`initialization`). Бұл кезеңде жиым элементтері бастапқы мәндерді қабылдайды.

```
Мысалы, x 28 a[0] = 0.01;  
a[1] = -3.4;  
a[2] = 2.89;  
a[3] = 4.5;  
a[4] = -6.7;  
for (int i = 0; i < 100; i++) b[i] = 1.0 / i;  
for (int i = 0; i < 50; i++) t[i] = 2 * i + 1;
```

Жиым элементтері — бұл кез келген бір типтегі айнымалылар, осы типке қатысты әр түрлі операцияларды орындауға болады: $(a[2] + a[4]) / a[0]$; т.с.с.

Көпөлшемді жиымдар Java –да жиым элементтері ретінде

жиымның өзі бола алады. Хабарлауға болады: `char[] [] c;`

бұған эквивалентті түрде былай да жазуға болады:

```
char c[] c[]; немесе char c[][];
```

Сыртқы жиымды да анықтай анықтаймыз: `c = new`

```
char[3][]; c жиымның үш жиым элементтерінен тұратын
```

жиым. Енді оның жиым-элементтерін анықтай аламыз:

```
c[0] = new char[2]; c[1] = new char[4]; c[2] = new char[3];
```

Тақырып бойынша сұрақтар:

1. Функциялар
2. Жиындар
3. Массивтер
4. String класы
5. StringBuffer класы
6. Циклдік операциялар
7. while циклы

ДӘРІС 10 / ТЕОРИЯ

Дәріс 10. Сөздіктер

Видеосабақ: Лекция 10 Объектіге бағытталған бағдарламалау принциптерін жолдарға қолдану

Жолдарды өңдеу бастапқы дәрістің негізгі мазмұнына сәйкес келеді.

Бұл дәрістің негізгі мазмұны — сөз тіркестерін өңдеу. `<string>`, `assign()`, `c_str()`, `string` және `cout` жазылымдары C++ мысалдарына жатады. Java-да `String`, `substring()`, `indexOf()`, `equals()` және `StringBuffer` пайдаланылады. Қосымшада Java баламалары берілген.

Жоспар

10.1. Конструкторлар және тіркестерді меншіктеу 10.2. Тіркестердің бөлігін қосу және меніктеу 10.3. Тіркестерді түрлендіру 10.4. Тіркестердің құрамын іздеу 10.5. Тіркестердің бөліктерін салыстыру 10.6. Тіркестердің сипаттамаларын алу

1. Java ортасындағы сөз тіркестері Java тілінде арнайы сөз тіркесіне арналған тип қарастырылмаған. Бірақ Java ортасында тіркес-термен жұмыс істеудің екі мүмкіндігі бар: C кітапханасынан қалған (`<string.h>` не `<cstring>`) тақырып айлында сипатталған функциялар және Java-тің `string` кітапханалық класы. C-дан мұрагерлікке қалған функцияларды қолдану ыңғайсыз және қауіпті, себебі олар сөз тіркестерінің шекарадан шығып кетуін қадағалай алмайды. Стандартты кітапхананың `string` типі тіркестердің ақпараттың шекарасынан шығып кетуінен қорғалған және олар операциялар мен тәсілдер көмегімен жұмыс істей алады. Енді тіркестермен жұмыс істеудің негізгі ерекшеліктері мен тәсілдерін қарастырайық. Сөз тіркесі немесе тіркестер – нөл-символмен аяқталатын символдар жиымы (массиві). Олардың ұзындығы қажеттілікке қарай динамикалық түрде өзгертіліп отырады. Тіркестермен жұмыс істеу класын пайдалану үшін программаға `<string>` тақырыптық файлын енгізу керек. Бұл тақырыптық файл бұрынғы шыққан компиляторларда `<cstring.h>` (`<bstring.h>`, `<string.h>`) түрлерінде де кездесуі мүмкін.

2. Конструкторлар және тіркестерді меншіктеу `String` класында бірнеше конструкторлар анықталған, олардың жиі қолданылатындары:

```
string (); string (const char *); string (const char *, int n); string  
(string &);
```

Бірінші конструктор string типті бос объект құрады.

Екінші конструктор ескі стильдегі тіркестер негізіндегі string типті объект құрады, ал үшіншісі string типті объект құрады да, бірінші параметрде көрсетілген тіркестен n символды алып осыған жазады.

Төртінші конструктор көшіру конструкторы болып табылады, ол параметр ретінде берілген объектінің көшірмесі арқылы жаңа объектіні құрады.

string класында үш меншіктеу операциясы анықталған:

```
string&operator= (const string&str);  
string&operator= (const char*s);  
string&operator= (char c);
```

Мұнан бір тіркеске string типтегі басқа тіркесті немесе жеке символды да меншіктеуге болатыны көрініп тұр.

```
Мысалы: string s1; string s2("Azat"); string s3(s2); s1='X';  
s1="Azat"; s2=s3;
```

string класының функциялары: тіркестің бөлігін қосу және меншіктеу, тіркестерді түрлендіру, ішкі тіркесті іздеу, тіркес бөліктерін салыстыру. Тіркес элементін анықтау үшін бұрынғы индекстеу операциясы қолданылады және at функциясы анықталған.

```
Мысалы: string s("Azat"); cout << s.at(1);
```

Егер индекс тіркес ұзындығынан артып кетсе, онда out_of_range аластау (исключение) оқиғасы туындайды.

2. Тіркес бөлігін қосу және меншіктеу. Бір тіркестің ішкі бөлігін екінші бір тіркеске меншіктеу үшін assign

```
функциясы қолданылады: assign (const string &str);  
assign (const string &str, size_type pos size_type n);  
assign (const char *s, size_type n);
```

Алғашқы форма str тіркесін оны шақырған тіркеске меншіктейді, бұл функция әрекеті меншіктеу операциясына эквивалентті болып саналады.

Мысалы: string s1("Azat"), s2 s2.assign (s1); // бұл s2=s1 дегенмен бірдей
Жоғарыдағы функциялардың екінші формасы шақырушы тіркеске str тіркесінің pos позициясынан басталған n символдардан тұратын бөлігін меншіктейді. Егер pos+n мәні тіркес ұзындығынан артық болса, онда str тіркесі толық меншіктеледі.

Функциялардың үшінші формасы шақырушы тіркеске s тіркесінің n символын меншіктейді.

Мысалы: `string s1("telefon"), s2("olar"); s1.assign (s2);` // нәтижесі:

`s1=s2=olars1.assign (s2, 1, 3);` // нәтижесі: lar Бір тіркестің бөлігін екінші тіркеске қосып біріктіру үшін

```
append функциясы қолданылады: append (const string&str);
append (const string&str, size_type pos, size_type n);
append (const char*s, size_type n);
```

Бірінші форма str тіркесін шақырушы тіркестің соңына қосады, оның әрекеті конкатенация амалына (біріктіруге) ұқсас болып келеді.

Екінші форма шақырушы тіркеске pos позициясынан басталған str тіркесі бөлігін меншіктейді.

Мысалы: `string s1("fon"), s2("dar"); s1.append (s2);` // нәтижесі: fondar `s1.append (s2, 1, 2);` // нәтижесі: fonar

3. Тіркестерді түрлендіру. Бір тіркес бөлігін екінші тіркеске кірістіріп қою (вставка) үшін `insert` функциясы қолданылады: `insert (size_type pos1, const string &str); insert (size_type pos1, const string &str, size_type pos2,`

```
size_type n); insert (size_type pos, const char *s, size_type n);
```

4. Бірінші форма str тіркесін оны шақырған тіркеске pos1 позициясынан бастап кірістіріп орналастырады да, одан кейін шақырған тіркестің қалған символдары жалғасады.

Егер pos1 тіркес ұзындығынан артық болса, онда `out_of_range` оқиғасы орын алады.

Екінші формада оны шақырған тіркестің pos1 позициясы сақталады, сонан соң оның pos2 позициясынан бастап str тіркесінің n символы кірістіріледі де, одан кейін шақырған тіркестің қалған символдары жалғасады. Функцияның үшінші формасында оны шақырған тіркестің pos1 позициясынан бастап бұрынғы типтегі s тіркесінің n элементін кірістіріп орналастырады. Тіркестің ішкі бөлігін жою үшін `erase` функциясы қолданылады: `erase (size_type`

```
pos=0, size_type n=npos);
```

Ол шақырылған тіркестің pos позициясынан бастап n элементін жояды. Егер позиция көрсетілмесе, онда ол тіркес басынан жойылады. Бүкіл тіркесті тазалау қызметін clear

функциясы атқарады: `void clear();`

ДӘРІС 11 / ТЕОРИЯ

Дәріс 11. Объектіге бағытталған бағдарламалауға кіріспе

Видеосабақ: Лекция 2 Объектіге бағытталған әдіс

Қазақша видеосабақ дәрістің негізгі ұғымдарын толықтырады.

Жоспар

11.1. Мәліметтерді енгізу операциялары 11.2. Объектіге бағытталған программалау әдістемесі.

11.3. Объектілер.

11.4. Кластар 11.5. Конструкторлар 11.6. Инкапсуляция, мұралау және полиморфизм.

1. Мәліметтерді енгізу операциялары. Пернетақтадан айнымалылар мәнін енгізу үшін `System.in.read (byte [ ] bt)` әдісі қолданылады. Бұл әдіс енгізілген цифрларды `bt` байттар жиымына жазады, мұнда әрбір байтқа бір цифрдан жазылады. Мұнан кейін байттар жиымын `Double(new`

`String(bt)).doubleValue0` әдісімен нақты сандарға түрлендіру

қажет. Оған қоса туындауы мүмкін ерекше жағдайларды да өңдеу керек болады. Төмендегі мысалда консольмен байланысқан кіріс ағымына сөз тіркестерін енгізу қарастырылады. // тіркес енгізу мысалы:

```
InputStr.java import java.io.*;
public class InputStr {
    public static void main(String[] args)
```

```
{ /* System.in байттық кіріс ағымы InputStreamReader класс объектісіін құру
кезінде конструкторға беріледі
```

```
*/ InputStreamReader is = new InputStreamReader (System.in);  
/*мәліметтерді буферлеу жүзеге асады */ BufferedReader bis  
= new BufferedReader(is);  
try  
{  
System.out.println ("Өз атыңызды енгізіңіз және <Enter>  
пернесін басыңыз:"); /*Буферден қатарды оқу;
```

readLine() әдісі мүмкін болатын қателерді өңдейді */String

```
name = bis.readLine();  
System.out.println ("Привет, " + name);  
}  
catch (IOException e)  
{ System.out.print ("Енгізу қатесі " + e);  
}  
}
```

Азат деген тіркесті енгізсеңіз, онда келесі ақпарат шығады: Привет, Азат!

Бұл мысалда java.io кітапханасынан алынған InputStreamReader және BufferedReader класс әдістері мен конструкторлары, жеке жағдайда System.in кіріс ағымымен байланысты readLine() буферден қатарды оқу әдісі де қолданылған.

2. Объектіге бағытталған программалау әдістемесі.

Мәліметтерді өңдеу программал. Объектіге бағытталған программалау (ОБП) — программалық кодтар жасаудың қазіргі тәсілі, ол құрылымдық программалаудан кейін келді.

ОБП құрылымдық программалауды ауыстырған жоқ, ол оны ары қарай логикалық жетілдіру кезеңінен өткізді.

Құрылымдық программалау негізі — есепті шешудің сатылы бұтақ тәріздес құрылымын жасау және оның программалық кодын жекелеп жазып шығу болып табылады. Құрылымдық программалаушылар бір логикалық бірлікке қатысты процедуралар мен мәліметтерді жеке файлға құрылым (Си-де struct) арқылы жинақтай отырып жасап шығады. Дәстүрлі процедуралы программалаудың әдістері күрделі программалар құруға жауап бере алмайды.

Сондықтан объектіге бағытталған программалау (ОБП) пайда болды. Оның ерекшеліктері:

- Программалық жабдықтамалардың күрделілік дәрежесін

төмендету;

- Программалық жабдықтамалардың сенімділігін

жоғарылату;

- Программалық жабдықтамалардың кейбір компоненттерін қайтадан қолдану мүмкіндігін қамтамасыз ету. ОБП қолданбалы программалауда жедел дамушы бағыттардың бірі болып табылады.

3. Объектілер. ОБП-да объектілер негізгі элемент болып табылады. Объект — бұл мәліметтер мен функциялар жиынынан тұратын бірыңғай конструкция. Әрбір объект үшін оларға қолданылатын нақты операциялар жиынтығы бар. Мысалы, дербес компьютердің операциялық жүйесіндегі файлдарға қолданылатын операциялар:

- құру; ашу;
- файлдан оқу; файлға жазу;

- жабу; жою.

Объект интерфейсі – бұлар пайдалануға болатын объектінің айнымалылары мен функциялары. Объектінің ішкі құрылымы — программалау тілінде объектінің ішкі айнымалылары мен функцияларын сипаттау.

4. Кластар. Кластарда мәліметтер және программалық кодтар болады. Ондағы кодтар класс тәсіл -дері ішінде орналасады. Басты бір тәсілмен – main() тәсілімен таныстындар. Енді толығырақ программалар жазу үшін, объектілік көзқарасты мысал арқылы қарастырайық. Класс — объект типінің спецификациясы, ал объект — кластың бір нақты экземпляры (данасы). Негізінде, класс дегеніміз — абстрактылық ұғым, ол ешбір мәліметтерді өңдемейді. Солардың негізінде объектілер жасай отырып, солар арқылы керекті мәліметтермен жұмыс істеуге болады.

Класса сипатталған мәліметтер (айнымалылар мен константалар) класс мүшелері, ал функциялар — класс тәсілдері деп аталады. Класс данасында сақталатын мәліметтер объект айнымалылары (instance variables) деп аталады. Кластардан объектілер жасау синтаксисін қарастыралық. Мысалдарда келесі синтаксистік конструкция қолданылған:

```
Vehicle car = new Vehicle();
```

Бұл өрнек екі әрекет атқарады: мұнда Vehicle типіндегі car айнымалысы жарияланады және Vehicle класының данасын жедел жадыға орналастыру үшін орын бөлінеді де, car айнымалысына сол жады адресі меншіктеледі.

Сонымен, car айнымалысында объектінің өзі емес, соған сілтейтін нұсқауыш сақталады. Сол объектіні жасауды екі командамен былай да жазуға болар еді:

```
Vehicle car; // айнымалыны жариялау —// объектіге нұсқауыш car=new Vehicle();  
объект құру, айнымалыға оның адресін меншіктеу */ car айнымалысын жариялау  
нәтижесінде ол мән қабылдамай, тек сілтеме ғана алады.
```

Объект new командасы арқылы жасалып, оның адресі car айнымалысына меншіктелуі тиіс. Сонан соң ғана ол объектімен байланысады.

6. Конструкторлар. Алдыңғы мысалдарда Vehicle типті әрбір объектінің ішкі айнымалылары мәндерін код ішінде бердік. Бірақ мұндай әрекет кәдімгі программалауда жиі қолданылмайды. Мұнда бір мәнді меншіктеуді ұмытып, қате жіберіп алуға болады. Объектіге бастапқы мән берудің жақсы бір мүмкіндігі - бұл конструкторды пайдалану.

Конструктор—бұл объектіні жасау кезінде шақырылатын арнайы тәсіл. Бұл тәсілдің аты класс атымен бірдей болып келеді. Ол ешқандай мән қайтармайды да, тек объект айнымалыларына бастапқы мән беру үшін қолданылады.

Java тілінде әрбір класта кем дегенде бір конструктор болады. Ол айқын түрде берілмеуі де мүмкін, мұндайда компилятор класқа конструкторды келісім бойынша қосады да, объект айнымалыларына мән меншіктейді. Белгілі бір класс үшін өз конструкторыңызды жарияласаңыз, автоматты түрде іске қосылатын конструкторға қол жеткізуге болмайды.

7. Инкапсуляция (encapsulation) термині «қара жәшік» ретінде қарастырылатын объектінің ішкі құрылымын жасыру болып табылады. Инкапсуляция - кластың ішкі құрылымын жасырып, сыртқы құрылымнан бөлу деген сөз.

Объектінің қасиеттері белгілі болып саналады, яғни олар - сырттан қол жеткізуге болатын айнымалылар. Бірақ бұл функциялар қалай құрылған, олар қандай алгоритммен жұмыс істейді, ол туралы программалаушыға айтылмайды.

Программалаушы немесе объектіні тұтынушы адам объектінің қосымша ішкі функциялары мен айнымалылары бар ма, олар қол жеткізуге болатын қасиеттер мен тәсілдермен қалай байланысқан, ол жағын білмейді.

8. Мұралау. Мұралау (inheritance) - бұл кластар ара қатынасы, мұнда бір класс екінші бір кластың құрылымын немесе тәсілдерін пайдаланады. Мұралау "жалпы/жалқы" иерархиясын енгізеді, мұнда ішкі класс сыртқы кластың қасиеттерін өзіне мұра ретінде қабылдап алады. Ішкі кластар әдетте мұралап алған құрылымды немесе әдісті толықтырады немесе басқаша анықтайды. Полиморфизм.

Полиморфизм (polymorphism) ОБП-ның іргелі ұғымы.

"Полиморфизм" сөзі грек тілінен аударғанда "көптеген формасы бар" дегенді білдіреді. Мысалы, біз векторлық графика редакторын жасамақшы болдық делік, оның құрамындағы қарапайым графикалық фигураларды (притивтерді) бірнеше кластар - Point, Line, Circle, Box и т.б.

түрінде сипаттап алуымыз керек. Олардың әрқайсысында экранға белгілі бір фигура шығаратын draw әдісін анықтаймыз. Бізге осы примитивтерді қарастыра отырып, ішіндегі керектісін таңдайтын draw тәсілін анықтайтын код жазуымыз керек. Полиморфизмді білмейтін маман осылардың әрқайсысы үшін жиым (массив) жазар еді (әр фигура үшін), ал полиморфизмдегі draw тәсілі мұны әжептеуір жеңілдетеді.

Тақырып бойынша сұрақтар:

1. ОБП
2. Конструкторлар
3. Объектілер
4. Функциялар
5. Жиындар
6. Массивтер
7. String класы

ДӘРІС 12 / ТЕОРИЯ

Дәріс 12. Модульдер

Видеосабақ: Лекция 3 Объектілер мен класстар туралы түсінік

Класстар мен объекттер: дәрістегі класс және конструктор бөлімдеріне сәйкес.

Бастапқы дәріс класс, объект және конструктор ұғымдарын қамтиды. Пакеттер мен компиляция модулі туралы материал №13 зертханалық жұмыста берілген.

Жоспар

12.1 Java-да объектілі-бағытталған программалау

12.2 Класстар

12.3 Конструктор

Java-да объектілі-бағытталған программалау Объектілі-бағытталған программалаудың қасиеттері:

- мұрагерлік (жаңа класстарды ескілердің негізінде құру

мүмкіндіктері);

- инкапсуляция (мәліметтер мен әдістерді программаның

қалған бөлігінен ажырату мүмкіндігі);

- полиморфизм (бір операцияны, егер олардың ортақ қасиеті болса, әртүрлі типтегі класстармен орындау мүмкіндігі).

Класстар – объектілі-бағытталған программалаудың негізі болып табылады. Жалпы айтқанда, класстар – бұл мәліметтерге айналу, оларды қолдану мен модификациялауға қажетті барлық әдістер мен мәліметтерді біріктіру тәсілі.

Класстар екі негізгі компоненттен тұрады: жағдайы мен әдістері.

Жағдай – бұл оның барлық айнымалыларының мәндері.

Әдістер оның функционалды мүмкіндіктерін анықтайды.

Класты сипаттау `class` сөзінен басталады да, одан кейін кластың аты жазылады. Кластың аты бас әріппен жазылуы тиіс.

`Class` сөзінің алдына класс модификаторларын жазуға болады.

Бұл `public`, `abstract`, `final`, `strictfp` сөздерінің бірі. Алынған класс атының алдына `protected`, `private`, `static` модификаторларын қоюға болады. Алынған кластар мен интерфейстер, әдістер, өрістер кез келген ретпен жазылған кластың денесі ирек жақшаға алынады. Өрісті сипаттау барысында оның типі, одан кейін бос орын, аты және теңдік белгісінен кейін тұрақты мәнермен жазуға болатын бастапқы мәні көрсетіледі.

Өрісті сипаттау бір немесе бірнеше міндетті емес `public`,

`protected`, `private`, `static`, `final`, `transient`, `volatile`

модификаторларынан басталуы мүмкін. Әдісті сипаттауда оларға қайтарылған мәnnің типі немесе `void` сөзі, одан кейін, бос орын арқылы әдістің аты, содан соң жақшаға алынып параметрлер тізбегі көрсетіледі. Ары қарай ирек жақшада орындалатын әдіс жазылады.

Әдісті сипаттау `public`, `protected`, `private`, `abstract`, `static`, `final`, `synchronized`, `native`, `strictfp` модификаторларынан басталуы мүмкін. Параметрлер тізімінде үтір арқылы әр параметрдің аты мен типі келтіріледі. Қандай да бір параметрдің типінің алдында `final` модификаторы тұруы мүмкін. Мұндай параметрді әдістің ішінде өзгертуге болмайды. Параметрлер тізімі жоқ болса да жақшалар сақталады. Әдіс жұмысы басталмас бұрын әр параметр үшін әдіске айналғанда берілетін параметрдің мәні көшірілетін, жедел жады ұяшығы бөлінеді. Мұндай тәсіл параметрлерді мәні бойынша беру деп аталады.

Егер класты сипаттау барысында ешқандай кеңейтілу көрсетілмесе, яғни `extends` сөзі және кластың аты жазылмаса, онда Java бұл класты `Object` класының кеңейтілуі деп есептейді және компилятор бұл кеңейтілуді үнсіз келісім бойынша ары қарай жазады: `class Pet extends Object{ . . . }` Бұл кеңейтілуді бұдан да ашып жазуға болады.

`Object` класының өзі ештеменің мұрагері емес, одан кез келген Java класының иерархиясы басталады.

Демек, барлық массивтер - `Object` класының тікелей мұрагерлері. `Object` класы барлық кластардың жалпы қасиеттерінен тұратын болғандықтан, оған бірнеше жалпы әдістер ғана енгізілген, мысалы, берілген объектіні логикалық мәнді

қайтаратын және аргументте берілген объектімен теңдікке салыстыратын `equals()` әдісі. Оны былай қолдануға болады:

```
Object obj1 = new Dog(), obj 2 = new Cat(); if (obj1.equals(obj2)) ...
```

Конструктор ерекшеліктері:

Конструктор кез-келген класта болады. Ол жазылмағанның өзінде Java компиляторы келісілген шартпен конструкторды өзі құрады, бірақ ол суперкласс конструкторын шақырудан басқа ешқандай іс- әрекет орындай алмайды. Жады мүмкіндігінше бөлініп, өрістер бос күйге келгеннен кейін, бірақ құрылатын объект қолданылғанға дейін класс көшірмесін құру кезінде конструктор автоматты түрде орындалады.

Конструктор ешқандай мән қайтармайды. Сондықтан оның сипаттамасында `void` сөзі жазылмайды, бірақ `public`,

`protected` немесе `private` тәрізді үш модификатордың бірін

беруге болады. Конструктор әдіс емес, ол класс мүшесі болып та табылмайды. Сондықтан оны мұрагерленуге немесе ішкі класта қайта анықтауға болмайды.

Конструктор денесі мына түрде басталуы мүмкін: о суперкласс конструкторларының бірін шақыру кезінде, мұнда жақшада параметрлерімен (қажет жағдайда) `super()` сөзі жазылады; о сол кластың басқа конструкторын шақыру кезінде, мұнда жақшада параметрлерімен (қажет жағдайда) `this()` сөзі жазылады. Егер `super()` конструктордың басында көрсетілмесе, онда алдымен аргументсіз супер класс конструкторы орындалады, содан кейін оларды жариялауда көрсетілген мәндері бар өрістердің инициализациясы орындалады, одан кейін ғана конструкторда жазылғандар орындалады. Қалған барлық жағдайларда конструкторды қарапайым әдіс деп есептеуге болады, онда кез-келген операторды жазуға болады. Бір класта бірнеше конструкторлар болуы мүмкін. Оларда класс атымен сәйкес келетін бірдей аттар болатын болса, онда олар параметрлер типімен не/немесе санымен ерекшеленуі керек. `New` операциясы массив пен объектілердің жадыларын белгілеу үшін қолданылады. Егер конструктор класта болмаса, онда үнсіз келісім бойынша конструктор шақырылады. Кластың сандық өрістері нольдік мәндерді, логикалық өрістер — `false` мәнін, сілтемелер — `null` мәнін алады.

`New` операциясының нәтижесі құрылған объектіге сілтеме болып табылады. Бұл сілтеме берілген типке сілтеме типті айнамағымен меншіктелуі мүмкін:

Қосымша түрінде безендірілген кез-келген программа main атты әдістен тұруы керек. Ол барлық қосымшалар үшін біреу болуы мүмкін немесе осы қосымшалардың кейбір кластарында, сонымен қатар әрбір класта болуы мүмкін.

Main() әдісі қарапайым әдіс тәрізді жазылады, кез-келген іс- әрекеттер мен сипаттамалардан тұруы мүмкін, бірақ ол міндетті түрде ашық (public), статикалық (static) болуы керек, қайтарылатын мәні (void) болмайды. Оның аргументі болып міндетті түрде командалық жолдың аргументтері берілетін (string[])- args жолдар массиві болуы керек. Бұл ерекшеліктер қосымшаның ең алғаш орындалуы кезінде Java орындаушы жүйесі арқылы main() әдісінің автоматты түрде шақырылуынан пайда болады.

Java интерпретаторын шақыру кезінде орындлу басталатын

```
main() әдісі жазылған класс көрсетіледі. Main() әдісі бар
```

кластардың бірнеше түрі болғандықтан, әртүрлі кластардың әртүрлі жағдайларында қосымшаларды орындаудан бастап қосымша кіріс нүктелері бар қосымша құруға болады.

Көбіне main() әдісін әр класқа қалдырып отыру мақсатымен енгізеді. Бұл жағдайда main() әдісіне кластың барлық әдістерінің жұмысын тексеретін тест енгізіледі.

Java интерпретаторын шақыру кезінде main() әдісіне интерпретатордың жолдар массивіне енгізетін бірнеше параметрлерін беруге болады. Бұл параметрлер java шақыру жолында класс атынан соң бос орыннан кейін бірден келтіріледі. Егер параметрде бос орын қолданылса, онда оны жақшаға алу қажет. Жақшалар параметрге кірмейді, олар тек қана шектеулер ретінде қолданылады.

Кластарды қолдану мысалы:

```

import java.awt.*;
class avto4 { avto av1,av2;
public avto4() { av1=new avto(300,"opel","disel",7); av2=new
avto(400,"audi","disel",17);
av1.show();
av2.show();
v a1 = new av (300,"opel","disel",7,100,"aaaaa");
a1.prodan();
System.out.println (a1.pokyp);
av1 a11 =new av1 (300,"opel","disel",7,100,"aaaaa",10000);
av1 a22=new av1 (400,"audi","disel",17,100,"",9000);
a11.beg();
a22.beg(); }
public static void main (String[] arg) { ew avto4();
}
}
class avto {
int zena;
String marka;
String tip;
int srok; avto ( int s,String m,String t,int sr) { z ena=s;
marka=m; tip=t; srok=sr;
} void show() {
System.out.println("marka " + marka);
System.out.println("zena " + zena); }
}
class av extends avto { private int zenapr;
private String pokyp;
av(int s,String m,String t,int sr,int pc, String pok) { super( s, m,
t, sr);
zenapr=pc;
pokyp=pok; }

void prodan() { if (pokyp != "" )
{
System.out.println ("prodan");
System.out.println (pokyp);
} else
System.out.println ("noo");
}
} class av1 extends av
{
nt probeg;
av1(int sa, String ma,String t,int asr,int pca,String pok,int pr) {
super( sa, ma,t, asr,pca,pok);
probeg=pr; }
void beg()
{ System.out.println(probeg);
}
}
}

```

Тақырып бойынша сұрақтар:

1. Объектілі-бағытталған программалаудың принциптерін атаңыз.
2. Класс пен ішкі класты қалай сипаттауға болады?
3. Конструктор не үшін қажет?
4. New операнды не үшін қолданылады?
5. Айнымалылардың көріну аймағы қандай?

ДӘРІС 13 / ТЕОРИЯ

Дәріс 13. Matlab тәрізді және өзге кітапханаларды пайдалану

Видеосабақ: Лекция 7 Java тілінің стандартты кітапханалары және пакеттер

Java стандартты кітапханалары мен пакеттері туралы қосымша сабақ. MATLAB ортасына арналған ресми материал пайдаланылған әдебиеттерде берілген.

MATLAB — матрицалық және ғылыми есептеулерге арналған бөлек орта; Simulink — модельдеу пакеті. Java Math класы MATLAB ортасымен бірдей емес. Практикалық салыстыру тапсырмалары осы айырманы ескереді.

Жоспар

13.1 MatLab орталығы туралы жалпы мәлімет;

13.2 Simulink кітапхана блоктарының жалпы

сипаттамалары;

13.3 NCD Blockset кітапханасы блогының жалпы

сипаттамасы;

MatLab орталығы туралы жалпы мәлімет MatLab жүйесі инженерлік есептерді есептеуге арналған пакет. Бұл пакет қазіргі заманғы инженерлі және ғылыми математикалық аппаратына жақын, онда матрицалармен, векторлармен және комплексті сандармен жұмыс істеуге болады.

MatLab жүйесінің бірінші версиясы 70-жылдың соңында Нью-Мексико және Станфорд университетінде жазылған.

Бұл жүйе матрица теориясы, сызықты алгебра және санды талдау курстарын оқытуға арналған жүйе болатын. MatLab жүйесі FORTRAN, C және C++ бағдарламаларында жазылған функцияларды қабылдай алады. MatLab жүйесінің бағдарламалау тілі өте қарапайым, ол бірнеше операторлардан тұрады. Көп

процедураларымен функциялары бар және оларды қолданып қана қоймай, оларды модификациялауға болады.

Қазіргі замандағы MatLab жүйесі—ол инженерлік және ғылыми есептеулерде жоғарғы тиімді тіл болып табылады. Онда математикалық есептерді есептеу, ғылыми графиканың визуализациясын алуға және бағдарламалау мүмкіндіктері бар.

MatLab жүйесінің негізгі объектісі – ол тік бұрышты массив саны (матрица). Матрицада комплексті элементтерді қолдануға болады.

MatLab жүйесі екі негізгі режимде жұмыс істей алады:

– ғылыми калькулятор;

– программалық режим.

MatLab-ты қосқаннан соң 8.1-суретте көрсетілген негізгі терезесі ашылады.

Онда бірнеше терезелер көрсетіледі, оның негізісі Команда терезесі (Command Window). MatLab жүйесінің командаларын қабылдауға дайын екенін, команда жолындағы ">>" белгі арқылы білуге болады.

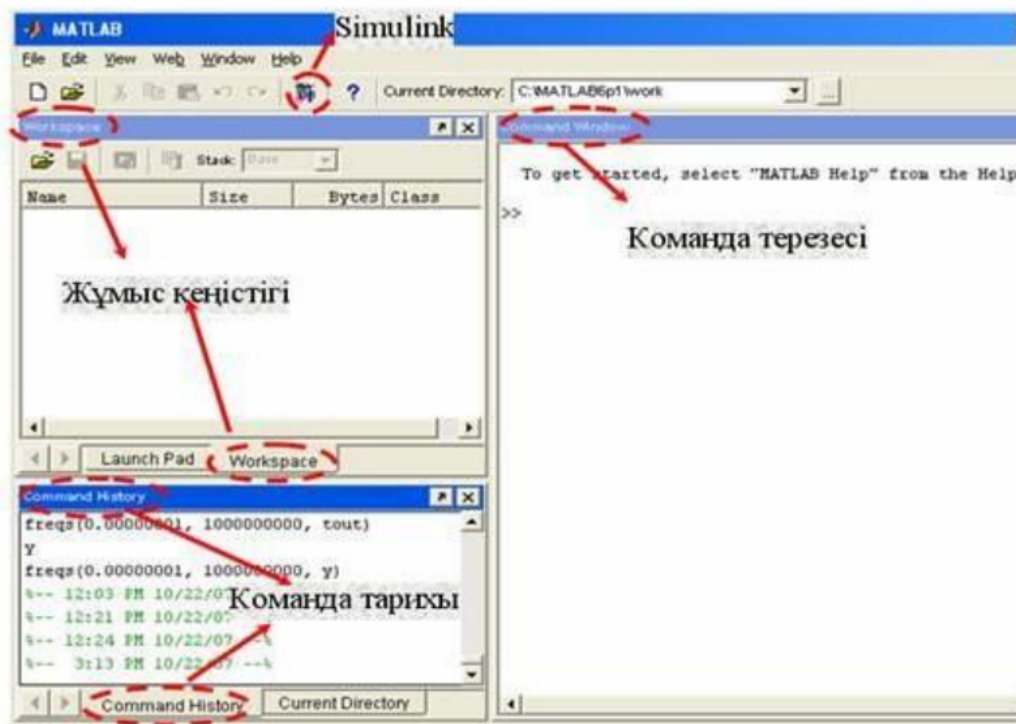
Ғылыми калькулятор режимінде жұмыс істеу үшін Команда терезесі қолданылады.

MatLab жүйесінде программалауы өзінің М-тілі атты бағдарламалық тілінде орындалады. Программалар Редактор М- файлов атты редакторында жазылады және олар М-файл деп аталады. Оларға *.m кеңейтілуі беріледі.

MatLab жүйесінің негізгі ұғымына – Жұмыс кеңістік (Рабочее пространство) ұғымы жатады. Жұмыс кеңістігі– MatLab бағдарламада қолданылатын айнymалылар сақталынатын жад аймағы. Ол арқылы MatLab жүйесінің түрлі компоненттер арасында мәліметтерді алмастыруға болады.

8.1 - сурет – MatLab жүйесінің негізгі терезесі

Simulink – динамикалық жүйелерді визуалды модельдеуге арналған пакет
Simulink динамикалық сызықты және сызықты емес



жүйелердің (уақыт арқылы модельдеу) тәртіптерін зерттеуге арналған пакет.

Программаны құру және жүйенің, экранда элементтердің байланыстарын сұлба арқылы құрып, олардың сипаттамаларын диалог режимінде енгізуге болады. Құру нәтижесінде жүйенің моделі жасалады (оларды S - модель деп атайды) және оларға *.mdl кеңейтілуі беріледі. Бұндай есептеу бағдарламасының құруын визуалды бағдарламалау деп атайды.

Simulink бағдарламасымен жұмыс істеу үшін MatLab командалық терезесі арқылы шығуға болады. Ол үшін саймандар тақтасында арнайы батырмасын басу қажет.

Нәтижесінде 8.2-суретте көрсетілген Simulink Library Browser (Браузер библиотек Simulink) терезесі пайда болады.

Терезенің сол жақ бөлігінде Simulink құрамына кіретін кітапханасының тізімі көрсетілген, терезенің оң жақ бөлігінде таңдалған кітапханасының тарауы немесе тараудың блок бейнелері көрсетіледі. Simulink пакетінің ядросы ол Simulink кітапханасы.

8.2 - сурет – Simulink Library Browser терезесі

Модельдеу жүйесінің блок-схемасын құру үшін Simulink браузер терезесінде File/New/Model (Файл/Новый/Модель) командасын алу керек. Нәтижесінде S-модель құрылатын 8.3-суретінде көрсетілген терезе пайда болады.

8.3 - сурет – Модельдің терезесі

Терезе меню жолынан, саймандар тақтасынан және жұмыс ай мағынан тұрады.

Меню File (Файл) - MDL-файлдармен жұмыс істейтін командаларынан тұрады.

Меню Edit (Правка) – блок-схемаларды өзгерту командаларынынан тұрады.

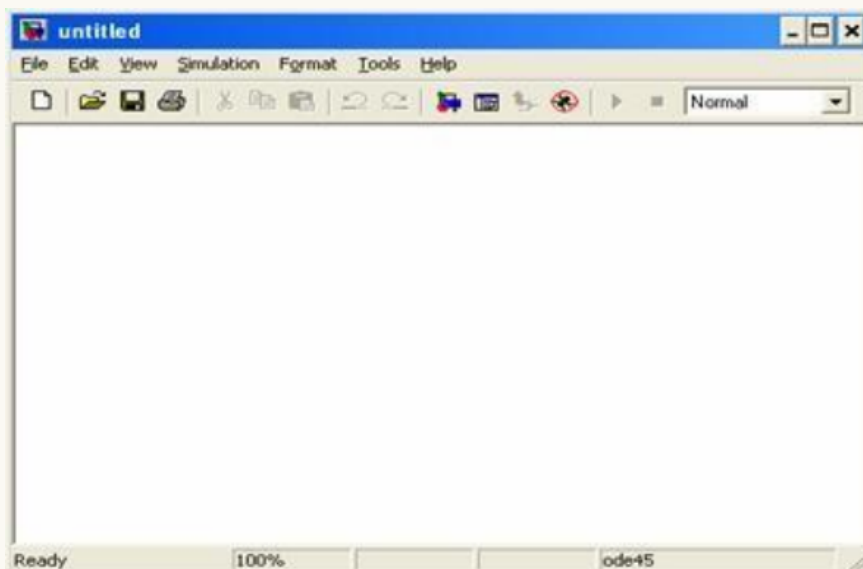
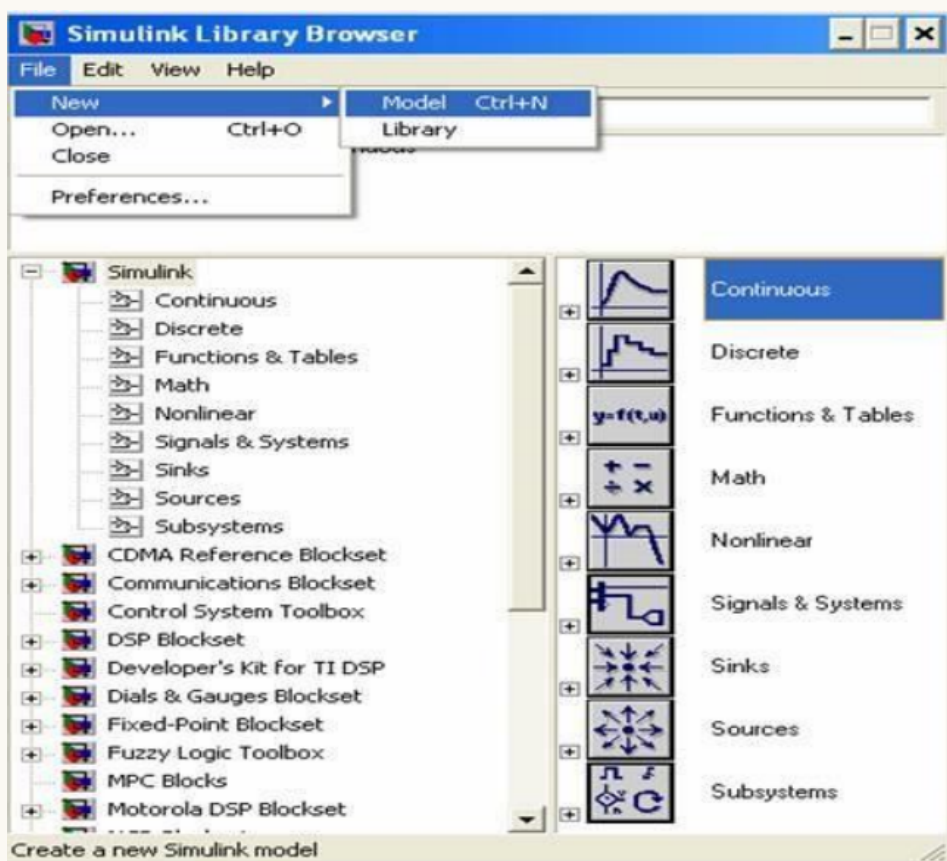
Меню View (Вид) –сыртқы терезенін түрін өзгерту командаларынынан тұрады.

Меню Simulation (Моделирование)– модельдеу процесін басқаратын командаларынан тұрады.

меню Format (Формат) – жеке блоктарының және жалпы блок- схемасының сыртқы түрін өзгерту командаларынан тұрады.

Меню Tools (Инструменты)–S-модельдермен жұмыс істейтін қосымша сервисты құралдары бар.

Блок-схемалар кітапханасының тарауында белгіленген



блокты модель терезесіне тарту арқыры құрылады. Содан соң, блоктардың шығыстары келесі блоктардың кірістеріне созу арқылы жалғастырылады. Егер қосу дұрыс болса, онда блоктың кірісінде қара стрелка пайда болады.

Кез келген блок-схемасы сигналды генерациялайтын, бір немесе бірнеше блок-көздерінен және жүйенің шығыс сигналдары туралы ақпараты алатан бір немесе бірнеше блок-қабылдағыштардан тұру керек.

Simulink пакетінің көпшілік кітапханасынан біз келесі кітапханаларын қолданамыз: Simulink, NCD Blockset және SimPower Systems [1].

Simulink кітапхана блоктарының жалпы сипаттамалары Sinks (приёмники) тарауы – осы блоктар арқылы модельдеуді өткізген кездегі визуалды нәтижелерін алуға болады және блоктардың дұрыс істеу жұмысын тексеру үшін арналған.

Осы блоктардың шығыстары жоқ, тек кірістері бар және олар 3 топқа бөлінеді:

1) Data Viewers – шолу терезесінің ролін орындайды;

2) Model & Subsystems Outputs– нәтижелерін сақтауға

және жіберуге арналған блоктар;

3) Stop Simulation–кіріске нөл емес сигнал берілген кездегі, модельдеуді тоқтататын басқару блогы.

Sources (источники) тарауы – S-моделдің немесе оның бөлек бөлшектерінің жұмыс істеуіне қамтамасыз ететін, сигналдарды түрлендіретін блоктардан тұрады.

Осы блоктарда кірістері жоқ, тек бір шығыстары бар.

Осы тарауда 2 топ блоктары бар:

1. Model & Subsystem Inputs - S-моделге немесе Simulink жүйелі бөлшектерден, басқа модельдерден, жұмыс кеңістіктерінен немесе М-файлдардан келген сигналдардың

кірісін қамтамасыз етеді;

2. Signal Generators – шығыс шамасын берілген уақыт функция ретінде формалайды. Яғни берілген типті сигналдардың генераторы ретінде қарастыруға болады.

Осы кітапханасында уақыт сигналын генерациялайтын Clock (Часы) блогінан басқа, барлық блоктарының параметрлерін өзгертуге болады.

Continuous (непрерывные) тарауы – тұрақты коэффициенттері бар сызықты дифференциалды теңдеулермен сипатталатын блоктарынан тұрады.

Олар 3 топқа бөлінеді:

1) жалпы тағайындалатын блоктары (интеграторлар,

туынды алатын блок);

2) сигналдарды ұстау блоктары;

3) сызықты стационарлы буынды блоктары.

Discrete (дискретные) тарауы – дискретті уақыт жүйелерінің тәртібін сипаттайтын блоктардан тұрады, сонымен қоса үздіксіз жүйелерді дискреттіге түрлендіретін блоктары бар.

Math (математические операции) тарауы – MatLab жүйесінің математикалық функцияларынан тұратын блоктар. Олар 4 топқа бөлінеді:

1) Math Operation – кіріс шамаларын математикалық түрлендіру блоктарынан тұрады (сумматор, күшейткіш, көбейткіш/бөлгіш), сонымен қоса математикалық стандартты блоктар.

2) Vector Operations – векторлық операцияларды орындайтын блоктар.

3) Complex Vector Conversions – векторлық комплексты шамаларды түрлендіретін блоктар.

Nonlinear (нелинейные) тарауы – кейбір сызықты емес кіріс сигналдарынан шығысқа тәуелсіздігін орындайтын блоктардан тұрады.

Signals and Systems (сигналдар және жүйелер) тарауы – әр түрлі сигналдарды жіберуін қамтамасыз ететін блоктардан тұрады (ауыстыру, бірнеше сигналдардың бір шинаға біріктіру немесе оларды ажырату және т.б).

Сонымен қоса сигналдардың кейбір атрибуттарын өзгертетін немесе қайта анықтайтын блоктары бар.

Function and Tables (функциялар және кестелер) тарауы

– сәйкестілік кесте арқылы берілген кіріс сигналдарын шығыс сигналдарына қатынасын көрсететін блоктардан тұрады.

Subsystems (подсистемы) тарауы – құрлымында төменгі деңгейі (жүйе бөлшек) моделі бар, күрдері S -модельдерін құру үшін және өз ара байланысын қамтамасыз етуіне арналған блоктардан тұрады.

Жүйе бөлшек – оның құрамында түрлі деңгейдегі жүйе бөлшектері бар, төменгі деңгей моделі болып табылады.

Жүйе бөлшектер негізгі S-моделінің құрамында ғана бола алады және олар (In) кіріс және (Out) шығыстар арқылы байланысады.

NCD Blockset кітапханасы блогының жалпы сипаттамасы MATLAB + Simulink жүйесінде Toolbox және Blockset атты құрал-саймандар жиынтығы бар. Toolbox жиынтығы MATLAB жүйесіне қатысты, ал Blockset жиынтығы - Simulink – ке.

BlockSet құрымына кіретін Nonlinear Control Design (NCD) сызықты емес жүйе басқаруын құру қолданбалы пакет бағдарламасы, басқару жүйесінің динамикалық оптимизациясын жасауға арналған. Ол арқылы айнымалыларды өзгертуге, жүйенің анықталмаған параметрлерін, интерактивты оптимизациялауын, Монте- Карло әдісімен модельдеу, басқару жүйесінің бір және көп өлшемді жобалауын қолдау, объектілерді кешіктіру арқылы модельдеу және басқарудың тағы да басқа есептерін шешуге болады.

MATLAB 7 + Simulink 6 версиясында Nonlinear Control Design блогы Response Optimization атты блогқа ауысты.

NCD пакетінің блок жиынтығы 3 блоктардан тұрады: CRMS (Continue RMS), DRMS (Discrete RMS) және NCD Output.

Қарастырылған блоктар, модельдеу жүйелерінің функционалдау сапасын квадратталған интегралды критерияларын немесе стандартты қатенің ауытқуын бағалау үшін қолданылады.

NCD блогы қарастырылатын блок жиынтығының негізі болып табылады. Оның өз жұмыс терезесі бар және интерактивті режимде келесі операцияларын орындайды:

1) Оптимизациялау жүйесінің сигналдарына уақытты

- аймағында керекті шектеулерін беруге болады;
- 2) оптимизациялайтын параметрлерін көрсетуге болады;
- 3) анықталмаған параметрлерін көрсетуге болады;

4) жүйенің берілген шектеулер арқылы параметрлі оптимизациясын өткізуге болады.

Оптимизация процесінің параметрлерін белгісіз, нақты емес және өзгеруі мүмкін шамаларын күрделетуге болады.

Start батырмасы арқылы процестің іздеу шешімін қосуға болады. Қорытынды нәтижесін NCD терезесінде өтпелі процесін бастапқы және соңғы формасын көруге болады, сонымен қоса MATLAB терезесінде оптимизация процесінің мәліметтерін көрсетеді. Осындай белгілерді беру үшін NCD Output терезесінің Optimization менюында Uncertainty... командасын таңдау керек. Ол Uncertain Variables атты терезені ашады.

Үнсіздік параметрдің номиналды мәні қолданылады (Constrain nominal simulation). Белгісіздікті енгізу үшін оның бастапқы (Constrain upper simulation) және/ немесе соңғы (Constrain lower simulation) шектуін көрсету керек.

Constrain Monte Carlo simulations белгісі көрсетілген зонасының (Монте-Карло әдісі) бірнеше кездейсоқ параметрлерінің мәнін модельдеуге болады.

Number of Monte Carlo simulations өрісінде сондай мәндерінің саны көрсетіледі, онда үлкен сандарды қолданбаған дұрыс.

NCD Output блогының графиктерін жою үшін Edit Delete plots меню пункті қолданылады (немесі Ctrl + X).

NCD Output блогының терезе менюін қарастырайық:

Меню File (Файл) – стандартты командаларынан тұрады Load (Загрузить - қосу), Close (Закреть - жабу), Save (Сохранить - сақтау) және Print (Печатать – баспаға шығару).

Меню Edit (Редактировать) – қарастырылған Edit constraint және Delete plots командаларынан тұрады, сонымен қоса соңғы команданы артқа қайтару Undo командасы бар.

Меню Options (Параметры) келесі пунктерден тұрады:

- Initial response (Начальный отклик). Бұл пункт зерттелетін жүйенің бастапқы параметрлерінің мәндерін шығарады.

- Reference input (Задающий вход). Бұл пункт Reference signal for output терезесін ашады. Ол арқылы керекті шығыс графигін алу үшін жүйенің кіріс параметірлері

көрсетіледі. Олар модельдеу және оптимизация процестерінде ешқандай әсер етпейді.

Step response (Характеристики переходного процесса).

Бұл пункт арқылы Step response атты диалог терезесін ашуға болады. Ол арқылы ауыспалы процестің келесі параметрлерін беруге болады:

- ұзақтылығын (Settling time):

- өсу уақытын (Rise time);
- қайта реттеу максималды мәнін (Percent overshoot);

- «қайта реттеу емес» максималды мәнін (Percent undershoot);

- берілген сипаттамаларының деңгейлерін (Percent settling, Percent rise, пайыз түрінде);

- модельдеудің бастапқы және соңғы мәндерін (Step time и Final time);

- шығыс сигналының бастапқы және керекті соңғы мәндерін (Initial output және Final output).

Time range (Временной диапазон). Осы пунктты таңдау арқылы Time Axis Limits атты диалог терезесі ашылады. Ол арқылы модельдеудің уақыт диапазонын және уақыт осінің белгісін беруге немесе өзгертуге болады, яғни абсцисс осінің параметрлерін Y-Axis (Ось Y). Ординат осінің параметрлерін. Refresh (Обновить) – барлық уақытша шектеулерін қайта құру.

Меню Optimization (Оптимизация) қарастырылған Parameters және Uncertainty пунктерінен тұрады, ал Start (Старт) пункті жүйенің оптимизация модельдеу процесін іске қосады. Stop (Стоп) пункті модельдеу процесін тоқтатады.

ДӘРІС 14 / ТЕОРИЯ

Дәріс 14. Қателерді талдау: `throwExceptions`

Видеосабақ: Лекция 9 Ерекшеліктерді өңдеу

Қазақша видеосабақ дәрістің негізгі ұғымдарын толықтырады.

Жоспар

Ескерту-бағдарламаның орындалу барысындағы қателер.

Ескерту- әртүрлі жағдайларда туындайды, мысалы

1. Қолданушы қате деректерлерді енгізді.
2. Бағдарлама, бағытталған файлды таппайды.
3. Деректердің жіберілуінің барысында сервермен желілік байланыс жоғалды.

Ескертулерді (exception handling) өңдеу - қателармен жұмыс кезіндегі бағдарламаның құбылуын сипаттауға және де бағдарламаның алгоритмін өңдеу мүлдем мүмкін емес жағдайдағы ескертулердің басқа да түрлеріне арналған бағдарламалау тілдерінің механизмі.

Синтаксис. Java да ескертулермен жұмыс жасайтын 5 кілттік сөз

1. `try` - айтылмыш кілттік сөз қатеге келтіретін блоктың кодының басын белгілеу үшін пайдаланылады.
2. `catch` - ескертулер табылған бөлікті өңдеу үшін блоктың кодының басын белгілеу үшін арналған кілттік сөз .
3. `finally` - қосымша қатеге келтіретін блоктың кодының басын белгілеу үшін пайдаланылады. Ол `catch` кілттік сөзінен кейін жүреді. Басқару `finally` блогына кез-келген жағдайда беріледі.
4. `throw` – ескертуді түрлендіру үшін қызмет етеді.
5. `throws` – көрсетілген типті ескертуге арналған әдістің сигнатурасына жататын кілттік сөз.

Қарапайым бағдарламадан бастаймыз. Үшбұрыштың ауданның есептеуден бастайық

Мұнда `getAreaValue` әдісінде біз `IllegalArgumentException`–ді `throw` кілтті сөзінің көмегімен ескертеміз. Бұл жағдайда әдістің сигнатурасында `throws IllegalArgumentException` болмайды, себебі `IllegalArgumentException` тексерілмейді олтуралы біз тағы айтамыз.

Конструкцияның ортақ көрінісі ерекше "аулау" жағдайлары үшін келесі бейнемен көрінеді.

```
public static void main(String[] args) {
    System.out.println(getAreaValue(-1, 100));
}

public static int getAreaValue(int x, int y){
    if(x < 0 || y < 0) throw new IllegalArgumentException("value of 'x' or 'y' is negative: x="+x+", y="+y);
    return x*y;
}
try{
    //здесь код, который потенциально может привести к ошибке
}
catch(SomeException e ){ //в скобках указывается класс конкретной о
```

жидаемой ошибки

```
//здесь описываются действия, направленные на обработку исключен
```

ий

```
}
finally{
    //выполняется в любом случае ( блок finally не обязателен)
}
```

үшбұрыштың ауданы үшін

```
public static void main(String[] args) {

    int result = 0;

    try{

        result = getAreaValue(-1, 100); кейде кодтың
```

```

public static void main(String[] args) {

} catch (IllegalArgumentException e) орындалуы кере

қолданылады. fi {

әдістің басында Logger.getLogger(NewClass.class.getName()).log(new LogRecord босатуға қолдан

(Level.WARNING, "В метод вычисления площади был передан аргум шығарылған кла

ент с негативным значением!"));

throw e;

System.out.println("Result is : "+result);

public static int getAreaValue(int x, int y){

if(x < 0 || y < 0) throw new IllegalArgumentException("value of 'x' o

r 'y' is negative: x="+x+", y="+y);

return x*y;

```

Мұнда біз `IllegalArgumentException` ді анықтап, жағдайды блоктайды.

Мұндай қателіктерді біз "жөндей" алмаймыз.

Қолданушының не ойлағанын біз біле алмаймыз.

Сондықтан бұл `throw e` ескертуден аттап өтеміз. `Finally` бөлігі табылған қателерге қарамай сондай аймақтарға `finally` кілттік сөзі жабуға және уақытша қолдануға алынған бөліктерді өте ыңғайлы. Төменде екі түрлі әдіспен мысал келтіреміз:

```

public static void main(String[] args) {

    int result = 0;

    try{

```

```

        result = getAreaValue(-1, 100);
    } catch (IllegalArgumentException e)
    {
        Logger.getLogger(NewClass.class.getName()).log(new LogRecord
(Level.WARNING, "В метод вычисления площади был передан аргум
ент с негативным значением!"));
        throw e;
    }
    System.out.println("Result is : "+result);
}
public static int getAreaValue(int x, int y){
    if(x < 0 || y < 0) throw new IllegalArgumentException("value of 'x' o
r 'y' is negative: x="+x+", y="+y);
    return x*y;
}

```

```

class FinallyDemo {
static void procA() {
try {
System.out.println("inside procA");
throw new RuntimeException("demo");
} finally {
System.out.println("procA's finally");
}

static void procB() {
try {
System.out.println("inside procB");
return;
} finally {
System.out.println("procB's finally");
}

public static void main(String args[]) {
try {
procA();
} catch (Exception e) {
procB();
}
}
}

```

```
class FinallyDemo {  
    static void procA() {  
        try {  
            System.out.println("inside procA");  
            throw new RuntimeException("demo");  
        } finally {  
            System.out.println("procA's finally");  
        }  
    }  
}
```

```
    static void procB() {  
        try {  
            System.out.println("inside procB");  
            return;  
        } finally {  
            System.out.println("procB's finally");  
        }  
    }  
    public static void main(String args[]) {  
        try {  
            procA();  
        } catch (Exception e) {  
        }  
        procB();  
    }  
}
```

Бағдарлама нәтижесі:

inside procA procA's finally inside procB procB's finally

Ескерту иерархиясы Қателерді өңдеуші барлық класстар java.lang класының мұрагерлері. Throwable класының тікелей мұрагерлері Error және Exception болып табылады.

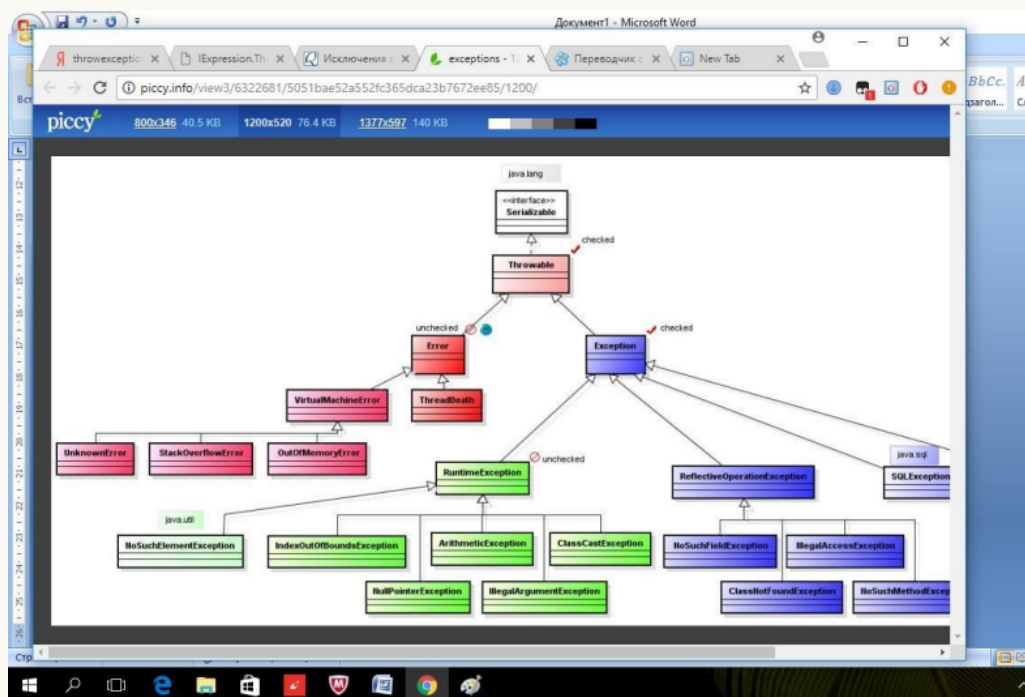
Error - бұл бағдарламаның орындалу барысындағы қиындықтарды көрсететін бағыныңқы класс.

Бағдарламалау кезінде Exception ге көп мән беру керек. Ол екіге бөлінеді: Runtime Exception және басқалар.

RuntimeException программадағы қатеден басқалары болжанбаған жағдайларға байланысты, мысалы енгізу- шығару қателері.

Тақырып бойынша сұрақтар:

1. try
2. catch
3. finally
4. throw
5. throws
6. Объектілі-бағытталған программалаудың принциптерін атаңыз.
7. Класс пен ішкі класты қалай сипаттауға болады?
8. Конструктор не үшін қажет?
9. New операнды не үшін қолданылады?
10. Айнымалылардың көріну аймағы қандай?



ДӘРІС 15 / ТЕОРИЯ

Дәріс 15. Ерекше жағдайларды өңдеу және бағдарламаның тұрақтылығы

Видеосабақ: Лекция 9 Ерекшеліктерді өңдеу

№15 зертханалық жұмыстың try/catch және ерекше жағдайлар тақырыбын толықтырады.

№15 зертханалық жұмыстан теорияға

Бұл дәріс практикалық бөлімдегі «Қателерді талдау (throwExceptions)» жұмысына сүйенеді. Мақсаты — қате туралы хабарламаны талдау, ерекше жағдайдың себебін ажырату және қате енгізу кезінде де жұмысты басқарылатын түрде жалғастыратын бағдарлама құру.

15.1. Ерекше жағдайлар иерархиясы

Java-да ерекше жағдайлар объект түрінде беріледі. Throwable — осы иерархияның түбірі. Оның негізгі тармақтары — Exception және Error. Exception тармағы қолданба өңдей алатын жағдайларды қамтиды. Error тармағы JVM деңгейіндегі күрделі ақауларды, мысалы жады жетіспеуін сипаттайды; оларды қарапайым енгізу қатесімен теңестіруге болмайды.

Checked ерекше жағдайлар компиляция кезінде өңдеуді немесе throws арқылы хабарлауды талап етеді. RuntimeException және оның ұрпақтары — unchecked ерекше жағдайлар. Оларды ұстауға болады, бірақ компилятор мұны міндеттемейді. ArithmeticException, ArrayIndexOutOfBoundsException және InputMismatchException осы дәрістергі негізгі мысалдар болып табылады.

15.2. Лабораториядағы өрнекті талдау

```
int[] m = {-1, 0, 1};  
int a = scanner.nextInt();  
m[a] = 4 / a;  
System.out.println(m[a]);
```

$a = 1$ кезінде $m[1]$ мәні 4 болады; $a = 2$ кезінде $m[2]$ мәні 2 болады. $a = 0$ кезінде бүтін санды нөлге бөлу `ArithmeticException` тудырады. $a = 3$ кезінде индекс массив шекарасынан шығады. «abc» енгізілсе, `nextInt()` `InputMismatchException` тастайды. Бұл төрт жағдай тексеру жоспарының негізін құрайды.

15.3. try, catch және finally

Қате тууы мүмкін амал try блогында орындалады. Сәйкес catch ерекше жағдайды ұстап, пайдаланушыға түсінікті хабарлама береді. Бір try үшін бірнеше catch қолдануға болады. Нақты түрлер жалпы `Exception` түрінен бұрын жазылады. `finally` блогы басқару осы құрылымнан шыққанда қорытынды әрекетті орындауға арналған; JVM мәжбүрлі тоқтатылса оның орындалуына кепіл берілмейді.

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class SafeArray {
    public static void main(String[] args) {
        int[] m = {-1, 0, 1};
        try (Scanner sc = new Scanner(System.in)) {
            System.out.print("Индекс енгізіңіз: ");
            try {
                int a = sc.nextInt();
                m[a] = 4 / a;
                System.out.println(m[a]);
            } catch (ArithmeticException e) {
                System.out.println("Нөлге бөлуге болмайды.");
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Индекс 0-2 аралығында болсын.");
            } catch (InputMismatchException e) {
                System.out.println("Бүтін сан енгізіңіз.");
            } finally {
                System.out.println("Өңдеу аяқталды.");
            }
        }
    }
}
```

15.4. Алдын ала тексеру және қайта енгізу

Ерекше жағдайды ұстау логикалық шартты алмастырмайды. $m[a] = 4/a$ есебінде a мәні массив шекарасында және нөлден өзге болуы қажет. Сондықтан $a \geq 1 \ \&\& \ a < m.length$ шартымен тексеруге болады. Енгізу қайта сұралса, `InputMismatchException` ұсталған соң `sc.next()` арқылы жарамсыз токенді алып тастау қажет; әйтпесе келесі оқу сол токенге қайта келеді.

Лабораториядағы $m[a-1] = 4/a$ нұсқасында есептеу нәтижесі $a-1$ индексіне жазылады. Оны көрсету кезінде де $m[a-1]$ қолданылуы тиіс. $m[a]$ шығару басқа элементті көрсетеді, ал $a=3$ болса массив шекарасынан шығуы мүмкін. Индексті бір рет есептеп, жазу мен оқуда бірдей қолдану кодтың түсініктілігін арттырады.

```
int index = a - 1;
if (a != 0 && index >= 0 && index < m.length) {
    m[index] = 4 / a;
    System.out.println(m[index]);
} else {
    System.out.println("Мән есептеу шартына сай емес.");
}
```

15.5. Нәтижені тексеру

Тексеру хаттамасында енгізу, күтілетін нәтиже және нақты нәтиже жазылады. Бастапқы $m[a]$ мысалы үшін $1 \rightarrow 4$; $2 \rightarrow 2$; $0 \rightarrow$ нөлге бөлу туралы хабарлама; $3 \rightarrow$ индекс туралы хабарлама; `abc` $\rightarrow$ бүтін сан туралы хабарлама. Әр тексеруде бағдарлама басқарылатын түрде аяқталуы және `finally` хабарламасы көрінуі тиіс.

Өзіндік жұмыс нәтижесі ретінде бастапқы және түзетілген кодты, бес тексерудің хаттамасын және әр ерекше жағдайдың себебін ұсыныңыз. Бағалау кезінде нәтиженің дұрыстығы, нақты қате түрлерінің өңделуі және хабарламалардың түсініктілігі ескеріледі.

07 / ЗЕРТХАНАЛЫҚ ЖҰМЫСТАР

Практикалық бөлім

Әр жұмыс үшін теориялық дайындық, алгоритм, бағдарлама және орындалу нәтижесі ұсынылады. Бастапқы жұмыс нөмірлері сақталған; №5–6 біріккен жұмыс ретінде беріледі.

Бастапқы практикалық бөлімде C/C++ және JavaScript тілдерінің салыстырмалы мысалдары бар. Java тіліндегі өзіндік тапсырмалар «Білімді бақылау» бөлімінде, орындалатын үлгілер қосымшада берілген.

ӘДІСТЕМЕЛІК НҰСҚАУЫ Жұмыс жасау кезіндегі қауіпсіздік техникасы және еңбекті қорғау

Еңбекті қорғау түсінігі адамның жұмыс үрдісінде қауіпсіздігін, денсаулығының сақталуын және жұмыс әрекеттілігін қамтамасыз ететін, әлеуметті-экономикалық, ұйымдастырушылық, техникалық, гигиеналық және емдеу профилактикалық әрекеттерді, заңнамалық актілер жүйесін білдіреді.

Еңбекті қорғау әдейі бір құқықтық және нормативті актілер негізінде құрастырылған әлеуметтік – экономикалық, кәсіпкерлік, техникалық, гигиеналық және емдеу– профилактикалық істер мен құралдар, жұмыс кезіндегі қауіпсіздік пен адамның денсаулығын сақтау жүйелер жинағы.

Еңбекті қорғаудың жалпы сұрақтары зертханада жұмыс жасайтын қызметкерлер үшін қауіпсіздік техникасының салалық ережелері негізінде технологиялық жабдықтарды қауіпсіз пайдаланудың құсқаулықтары өңделу қажет. Әр студент қауіпсіздік техникасы ережелерін қатаң сақтау мен апат, жаоақатқа әкелетін механизмдер, жабдықтар, аспаптар ақаулықтары жайлы тікелей бастығына айтуға міндетті.

Қауіпсіздік техникасы ережелеріне сәйкес қолданылатын қорғаныс құралдары мемлекеттік стандарттарына сай болуы керек және де электрқондырғыларда қолданылатын қорғаныстарды қолдану мен зерттеулер ережелеріне сай болуы керек.

Жұмыс кезінде қолданылатын механизмдер және жабдықтар зерттелген болу керек және құралмен жұмыс істеген кездегі қорғаныс ережелері, және де істеп шығарған зауыттың инструкциясына сәйкес қолдану керек.

Қауіпсіздік техника ережелерінің орындалуы міндетті болып келеді. Қауіпсіздік техникасына сай еңбекті қорғау нұсқаулары. Қауіпсіздік техникасының ережелеріне қарсы шараларды орындауға болмайды.

ПРАКТИКА / №1

Зертханалық жұмыс №1

Тақырыбы: Тілдің негіздері, мәліметтер түрлері және өңдеу ортасы.

Программалау тілінің интерпретаторын, IDE ортасын орнату, веб қабықшамен жұмыс жасау
Мақсаты: Бағдарламалау ортасымен танысу, бағдарлама құру және оны жүргізу.

Жұмысты қорғауға арналған сұрақтар:

1. Java тілінің негізі?
2. Java тілі қандай тілге жатады?
3. Java тілінде программалаудың құрылымы қандай?
4. Hello World программасын қатар бойынша талдау жасау?

Java тіліндегі бастапқы файл — ол өзінде бір немесе бірнеше класстардың сипаттамасын қамтитын мәтіндік файл. Java трансляторы, бағдарламаның бастапқы мәтіні кеңейтулері бар файлдарда сақталатынымен түсіндіріледі.

Трансляция барысында әр класс үшін алынатын код, сәйкес ат, класс атымен және class кеңейтілуімен жеке шығу файлдарында жазылады.

Ең алдымен, бұл тарауда, біз трансляция жасап және канондық «Hello World» деген бағдарламаны жазып жүргіземіз. Бұдан әрі біз Java- трансляторы қабылдайтын барлық лексикалық элементтерді қарастырамыз: аралықтар, түсініктемелер, негізгі сөздер, идентификаторлар, литералдарды, операторлар және бөлгіштер. Тараудың соңына қарай сіздер Java- бағдарламасында өздігінен бағыттала алатындай жеткілікті мәлімет ала аласыздар.

Hello World Біздің бірінші Java-программамыз:

```
class HelloWorld {  
    public static void main (String args []) { System. out. println  
        («Hello World»);  
    }  
}
```

Қадамнан қадамға Әрине, HelloWorld — бұл тривиальды мысал. Алайда тіпті мынадай жай бағдарлама Java тіліндегі жаңа қолданушыға күрделі болып

көрінеді, өйткені ол сендерді көптеген жаңа ұғымдар мен тіл синтаксисының бөлшегтерімен таныстырады.

```
class HelloWorld
```

1 қатар

```
class HelloWorld{
```

Бұл қатарда сақталынған `class` сөзі пайдаланылады. Ол трансляторға біздің жаңа классты сипаттайтынымызды хабарлайды. Класстың толық сипаттамасы бірінші қатарда ашылған фигуралы жақша мен оған қос 5 қатардағы жабылатын фигуралы жақшалардың арасында орналасқан.

Java-да қолданылатын фигуралы жақшалар C мен C++ тілдеріндегідей қолданылады.

2 қатар

```
public static void main (String args []) {
```

Бір көзге қарағанда мысал қатарының тым қиын болуы, Java тілін өңдеуде қаланған маңызды талаптардың салдары болып табылады. Java-да ғаламдық атқаратын қызметтер болмайды. Осындай қатарлар бірінші тараудың көп бөлігінде кездескендіктен, екінші қатардың әрбір элементін қарастырайық.

Public. Бұл қатарларды бөлек лексемаларға бөле отырып,

```
public сөзімен кездесеміз. Бұл—бағдарламашыға көрінетін
```

кез келген әдіс пен айнымалыларды басқаратын модификатордың мүмкіндігі. Бұл жағдайдағы `public` модификатордың мүмкіндігі, `main` әдісі кез келген класста көрінетіндігін және жетімділігін білдіреді.

Мүмкіндіктің (доступ) тағы 2 сілтегіш деңгейі бар —

```
private және protected бар.
```

Static. Бұл сөз арқылы класстың әдістері мен айнымалылар жарияланып, класстармен толық жұмыс жасау үшін пайдаланылады. `Static` сөзі қолданылатын әдістер, тек локальды және статистикалық айнымалылармен жұмыс жасайды.

Void. Сіздерде бір немесе басқа үлгінің мағынасын қайтаратын әдістерге жиі қажеттілік туады: мысалға, бағдарламашымен тағайындалатын `int` толық мағыналар үшін, `float` — заттай немесе деректер үлгілерінің класс атауы үшін.

Біздің жағдайда экранға тек қатарды шығару керек, ал main әдісінен мағынасын қайтарудың керегі жоқ.

Дәл осы себептен void модификаторы пайдаланылды.

Main. Енді main әдісін талқылайық. Бұл жерде ешқандай ерекшелік жоқ, тек барлығын Java- интерпретаторымен жүзеге асыру керек, классты интерпретациялау командасын алып, main әдісінің шақыруынан өзінің жұмысын бастайды. Java- транслятор main әдісі жоқ классты трансляторлай алады. Ал Java - интерпретатор классты main әдіссіз жүргізе алмайды.

Әдіске жіберілетін барлық параметрлер, «;» символымен бөлінген элементтер тізбегі ретінде қос домалақ жақшада көрсетіледі. Параметрлер тізбегінің әрбір элементі, бөлшектенген қадам типі мен идентификатордан тұрады.

Тіпті әдісте параметрлер болмаған жағдайда, оның артынан міндетті түрде бірнеше домалақ жақша қою керек. Қазір талқылап отырған мысалда, main әдісінде күрделендірілген типтегі тек бір параметр бар.

```
String args элементі[] args атымен аталатын параметрді
```

жариялайды, ол String классты массивтер объектілерін жариялайды. args идентификаторынан кейін тұрған тік бұрышты жақшаларға назар аударыңыздар. Олар біздің көрсетілген типтің жеке элементімен емес, массивтермен жұмыс істеп жатқанымызды айтады.

3 қатар

```
System.out.println(«Hello World!»);
```

Осы қатарда out нысанының println әдісі орындалады. out нысаны OutputStream классында жарияланған және статикалық System классында инициализацияланады. Main әдісін хабарлау 4 қатардағы жабылмалы ирек жақшамен бітеді, ал HelloWorld классы 5 қатарда дәл осындай фигуралы жақшамен хабарлануымен аяқталады.

Лексикалық негіздер Java- классын тереңірек қарастырғаннан кейін, енді осы тілдің синтаксисының ортақ аспектітерін қарастырайық.

Java-ның бағдарламалары— қадамдар, түсініктемелердің, бұлақты сөздердің, идентификаторлардың, литералдық тұрақтылардың, операторлар мен бөлгіштердің жиынтығы.

Қадамдар Java – тілі, бағдарламаның мәтінін еркін өңдеуге мүмкіндік беретін тіл. Бағдарлама дұрыс жұмыс жасау үшін, мәтінді арнайы әдістермен түзетудің қажеті жоқ.

Мысалы, HelloWorld классын екі қатарда немесе кез келген басқа әдіспен жазуға болар еді. Тек бөлек лексамалардың арасында бір қадам, табуляция символы немесе қатар аудармасының символы болуы керек, сонда ол дұрыс жұмыс атқарады.

Түсініктемелер Түсініктемелер бағдарламаның кодына әсер етпесе де, оны дұрыс пайдалану, бастапқы мәтіннің байыпты бөлігі болып табылады.

Түсініктемелердің үш түрі: бір қатарға түсініктемелер, бірнеше қатардағы түсініктемелер және документтеудегі түсініктеме деп бөлінеді. Бір қатарды алатын түсініктемелер // деген символмен басталып, қатар соңында аяқталады. Бұл түрдегі түсініктемелер бөлек қатар кодтарына қысқаша түсініктеме бергенге пайдалы.

```
а = 42; // егер 42 – жауап болса, онда қандай сұрақ
```

болды?

Бірнеше қатарларда орналасқан және мәтін түсініктемелерін

```
/* мына символмен бастап, */ осы символмен аяқтау
```

арқылы, біз түсініктемелерді толық түсіну үшін пайдалана аламыз. Осы қос символдар арасында орналасқан мәтін, түсініктеме ретінде пайымдалады және оны транслятор ескерусіз қалдырады.

```
/*  
*/
```

Үшінші түрдегі ерекше түсініктемелер javadoc сервисті бағдарламасына арналған, ол Java-транслятор компоненттерін автоматты түрде класс интерфейстері бойынша документтер генерациясына қолданылады.

Түсініктемелердің бұл түрін қолданудың келісімі мынада:

ашық (public) классын хабарлау алдында, /** түріндегі символдан бастау керек. Бұндай түсініктемелер, қарапайым түсініктемелер сияқты аяқталады. Javadoc бағдарламасы түсініктемелі құжаттардағы @ деген символдан басталатын кейбір арнайы айнымалыларды ажырата алады.

```
/**  
 * @see Java. applet. Applet  
 * @author Patrick Naughton  
 * @version 1. 2  
 */  
class CoolApplet extends Applet { /**
```

* бұл әдістің екі параметрі:

* @param key – параметрдің аты.

* @param value — key есімді параметрдің мағынасы .

*/ void put (String key, Object value) { Жадыда сақталған ұйытқы сөздер



ПРАКТИКА / №2

Зертханалық жұмыс №2

Тақырыбы: Мәліметтерді енгізу және шығару, input/output функцияларды

Мақсаты: Енгізу және шығару операторларына бірнеше мысалдар шығару

Жұмысты қорғау нәтижесі:

1. Тапсырмалар бойынша есеп беру;
2. Java тілінде құрылған программаларды талдау;
3. Енгізу және шығару операторларына тоқталу;

Тапсырмалар

1. // Программа создана для обучения. class MyProgram

```
{  
/** Выводим на экран строку "Это Ваша первая
```

```
программа на языке Java". */
```

```
public static void main(String args[]) { System.out.println("Эта  
Ваша первая программа на Java");  
}  
}  
/*Конец программы */
```

2. class Square1 {

```
public static void main(String args[])  
{  
int a, b = 4, s;  
a = 5;  
s = a * b;  
System.out.println ("Площадь прямоугольника со сторонами  
5 см и 4 см равна " + s + " квадратных сантиметров.");  
}  
}
```

```
// Айнымалы бола алатын мәндер  
_book new_look go_home Russia1 pi  
_Table_ My_1_1 My1_Home
```

Close_Your_Book Spartak_Moscow

```
// Айнымалы бола алмайтын мәндер do
```

Go Home lRussia Banan;Banan п "my" lshow_2 TV* TV abstract lpower yes or no

Жеке орындауға арналған тапсырмалар

1. $20^{\frac{1}{3}} + 158^2$ - өрнегін есептейтін және шешімін баспаға шығаратын бағдарлама құру.
2. $14/209 + 14 \times (29 - 13^2 + 14/3)$ - өрнегін есептейтін және шешімін баспаға шығаратын бағдарлама құру. Бұл жерде 14 санын бөлек айнымалы ретінде сипаттау міндетті
3. n айнымалысы екі таңбалы сан. n санының таңбалараының қосындысын табатын бағдарлама құру.
4. нақты n санының 0- ге тең емес бөлшегі бар. Жуық n ге дейін дөңгелектейтін бағдарлама құру



ПРАКТИКА / №3

Зертханалық жұмыс №3

Тақырыбы: Базалық математикалық операциялар және есептеулер, объектілер. Базалық операциялармен жұмыс, көбейту, бөлу, жақша.

Мақсаты: математикалық операциялар мен есептеулерді java тілінде жазу, бірнеше мысалдар шығару

Жұмысты қорғау нәтижесі:

1. Тапсырмалар бойынша есеп беру;
2. Java тіліндегі бағдарламасын құру;

Деректер типі	Жадтан алатын орны	% Қал Мәндер аралығы ++ 1қ	Үдінсқітзы есеп оксеул ісім бойынша мәні
byte	1 байт	бүтін [-128;127] эквив [-27;27]	0
short	2 байта	бүтін [-32768; 32767] немесе [- 15 15 2 ;2]	0
int	4 байта	бүтін [-2147483648; 2147483647] немесе [-231;231]	0
long	8 байт	бүтін [-263;263]	0L
float	4 байта	Нақты оң және теріс ~1,4*10-45дейін ~3,4*1038 стандарт IEEE 754	0.0f
double	8 байт	Нақты оң және теріс ~4,9*10-324до ~1,8*10308 по стандарту IEEE 754	0.0d

char 2 байта Натурал [0;65535], Unicode кестесінен

boolean

1 байт немесе әр түрлі

false немесе true

Арифметикалық операторлар + Қосу — Азайту * Көбейту — 1 алу += қосумен меншіктеу -= азайтумен меншіктеу *= көбейтумен меншіктеу /= бөлумен меншіктеу %= қалдықты меншіктеу
 Функциялар тізімі $\text{pow}(x,n)$ — x^n $\text{sqrt}(x)$ — $\sqrt{x}$ $\text{cbrt}(x)$ — $\sqrt[3]{x}$ $\text{abs}(x)$ — x тің абсолютті мәні (модуль x) $\text{exp}(x)$ — e^x $\text{log}(x)$ — натурал логарифм x $\text{log10}(x)$ — ондық логарифм x $\text{log2}(x)$ — $\log_2 x$ $\text{sin}(x)$ — $\sin x$ (x радианда) $\text{cos}(x)$ — $\cos x$ (x радианда) $\text{tan}(x)$ — $\tan x$ (x радианда) $\text{asin}(x)$ — $\arcsin x$ (x радианда) $\text{acos}(x)$ — $\arccos x$ (x радианда) $\text{atan}(x)$ — $\arctan x$ (x радианда) $\text{ceil}(x)$ — x жоғары дөңгелектеу, x тен кем емес ең кіші санға $\text{floor}(x)$ — x төменге дөңгелектеу, x тен үлкен емес ең үлкені санға $\text{round}(x)$ — ең жақын бүтін санға дөңгелектеу $\text{trunc}(x)$ — ең жақын бүтін санға дөңгелектеу (бөлшек бөлігін тастау арқылы) $\text{fmod}(a,b)$ — a/b қалдық

Жұмысты орындауға арналған тапсырмалар

1. Кубтың қыры берілген. Кубтың көлемі мен бүйір бетінің ауданын анықтаңдар.
2. Екі санның арифметикалық және геометриялық ортасын есептеп, нәтижесін шығаратын операторлар тізбегін жазыңыздар (оң сандар берілген деп есептейміз).
3. Екі санның қосындысын, айырмасын және көбейтіндісін есептейтін программа жазыңыз.
4. Берілген екі катеті бойынша тікбұрышты үшбұрыштың гипотенузасы мен ауданын есептейтін программа жазыңыз.
5. Координаттары x_1, y_1 және x_2, y_2 болып келген екі нүкте ара қашықтығын табыңдар.
6. x саны берілген. $2x^4 - 3x^3 + 4x^2 - 5x + 6$, $1 - 2x + 3x^2 - 4x^3$ және $1 + 2x + 3x^2 + 4x^3$ өрнектерін есептейтін программа құрыңыздар. Мұнда көбейту, қосу және алу амалдарын ғана қолдануға болады. Орындалатын операциялар саны мүмкіндігінше аз болатын болсын.
7. Тең қабырғалы үшбұрыштың қабырғасы берілген. Осы үшбұрыштың ауданын табатын программа құрыңыздар.
8. x, y, z берілген. a, b -ны есептеу керек:
- 9.
10. x, y, z берілген. a, b -ны есептеу керек:

x, y, z берілген. a, b-ны есептеу керек:

$$a = (1 + y) \frac{x + y / (x^2 + 4)}{e^{-x-2} + 1 / (x^2 + 4)}, \quad b = \frac{1 + \cos(y - 2)}{x^4 / 2 + \sin^2 z}$$

$$a = \frac{\sqrt{|x-1|} - \sqrt{|y|}}{1 + \frac{x^2}{2} + \frac{y^2}{4}}$$

$$b = x * (\operatorname{arctg}(z) + e^{-(x+3)})$$

$$a = \frac{3 + e^{y-1}}{1 + x^2 * |y - \operatorname{tg}(z)|}, \quad b = 1 + |y - x| + \frac{(y - x)^2}{2} + \frac{|y - x|^3}{3}$$

11. x, y, z берілген. a, b-ны есептеу керек:

12. x, y, z берілген. a, b-ны есептеу керек:

13. x, y, z берілген. a, b-ны есептеу керек:

14. x, y, z берілген. a, b-ны есептеу керек:

15. x және y сандары берілген, төмендегі мәнді табу керек.

16. Екі нақты сан берілген. Солардың арифметикалық ортасын және модульдерінің геометриялық ортасын табыңдар.

17. Үшбұрыш төбелерінің координаттары берілген. Сол үшбұрыштың периметрі мен ауданын анықтаңдар.

18. Арифметикалық прогрессия мүшелерінің, $a+d, \dots, a+(n-1)d$ қосындысын берілген a, d, n мәндері арқылы анықтаңдар.

19. Тікбұрышты үшбұрыштың гипотенузасы мен бір катеті берілген. Соның екінші катеті мен іштей сызылған шеңбер радиусын табыңдар.

20. Массалары m_1 және m_2 , ара қашықтықтары r болып келген денелердің бір бірін тарту күшін F анықтаңдар.

ПРАКТИКА / №4

Зертханалық жұмыс №4

Тақырыбы: If шартты операторы.

Мақсаты: Шартты алгоритмді программалауды меңгеру

Жұмысты қорғау нәтижесі:

1. Берілген есептерді шығару програмаларын құрыңдар.
2. Орындалған жұмыс бойынша есеп беру алгоритм блок-схемасы, программа мәтіні және есептеу нәтижелерінен тұрады.

Жұмысты қорғауға арналған сұрақтар:

1. Тармақты алгоритмдер ұғымы, оларды бейнелеу жолдары.
2. Шартты операторды жазу тәсілдері.
3. Меншіктеу, өрнек жазу операторлары.
4. Құрама операторлар.
5. Логикалық операциялар.

Жұмысты орындауға арналған тапсырмалар

1. Нақты x, y ($x \neq y$) берілген. Кішісін олардың жарты

$$a = \frac{2 * \cos(x - \pi / 6)}{1 / 2 + \sin^2 y}, \quad b = 1 + \frac{z^2}{3 + z^2 / 5}$$

$$a = \frac{1 + \sin^2(x + y)}{2 + \left| x - 2 * x / (1 + x^2 * y^2) \right|}, \quad b = \cos^2 \left(\arctg \frac{1}{z} \right)$$

$$\frac{|x| + |y|}{1 + |xy|}$$



$$a = y + \frac{x}{y^2 + \left| \frac{x^2}{y + x^3 / 3} \right|}, \quad b = \left(1 + \operatorname{tg}^2 \frac{z}{2} \right)$$

$$a = \ln \left| (y - \sqrt{|x|}) * \left(x - \frac{y}{z + x^2 / 4} \right) \right|, \quad b = x - \frac{x^3}{3!} - \frac{x^5}{5!}$$

қосындысымен, ал үлкенін - екі еселенген көбейтіндісімен алмастырыңыз.

2. Үш нақты сан берілген. Теріс емес сандарды квадраттаңыздар.

3. Егер берілген нақты x , y , z сандарының қосындысы 1-ден кем болса, онда бұл үш санның ең кішісін қалған екі санның жарты қосындысымен алмастырыңыз, кері жағдайда x және y -тің кішісін қалған екеуінің жарты қосындысымен алмастырыңыз.

4. Нақты a , b , c , d сандары берілген. Егер $a \leq b \leq c \leq d$ болса, онда әр санды ең үлкен санмен алмастырыңыз, егер $a > b > c > d$ болса, сандарды өзгеріссіз қалдырыңыз, кері жағдайда барлық сандарды олардың квадратымен алмастырыңыз.

5. Нақты x , y , z сандары берілген. Егер xy -ке қалдықсыз бөлінсе және yz -ке қалдықсыз бөлінсе, онда барлық сандарға бірді қосыңыз, кері жағдайда барлық сандарды нөлге теңестіріңіз.

6. Нақты a саны берілген. $f(a)$ –ны есептеңіз.

7. Нақты a саны берілген. $f(a)$ мәнін төмендегі формулалар бойынша есептеңіз:

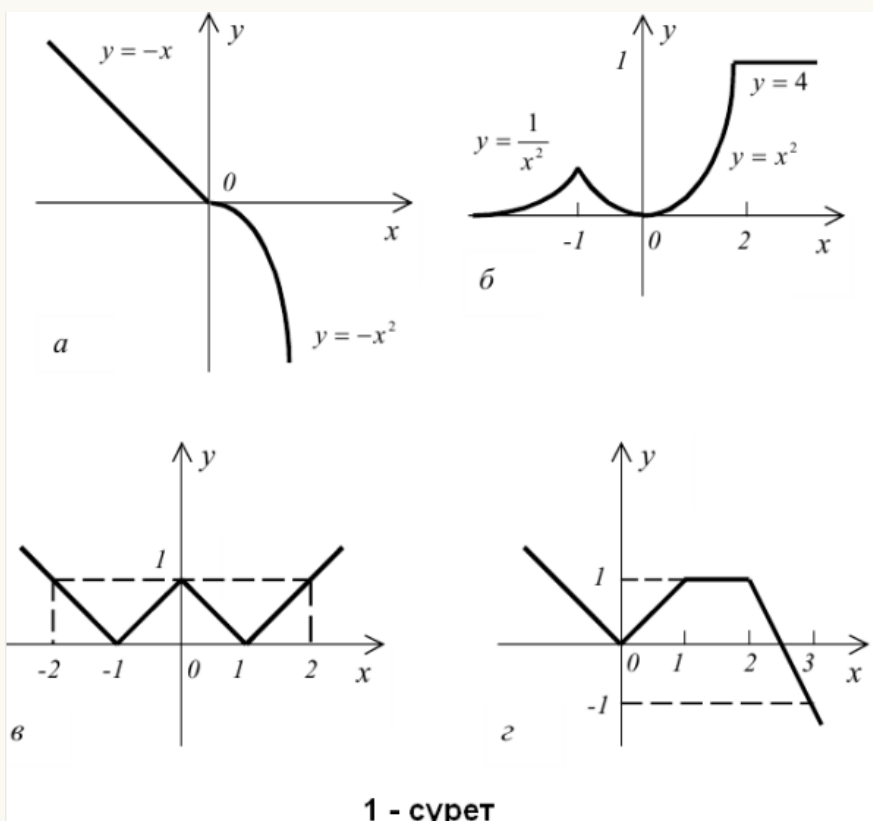
8. Нақты a, b, c сандары берілген ($a \neq 0$). $ax^2+bx+c=0$ теңдеуінің нақты түбірлері бар екенін тексеріңіз. Егер бар болса оларды есептеңіз, кері жағдайда экранда “нақты түбірі жоқ” деген мәлімдеме шығу керек.

9. Нақты x, y сандары берілген. Егер x және y оң сан болса, онда оларды түбірден шығарыңыз; егер тек біреуі оң болса, онда оларды квадраттаңыз. Егер x және y теріс сан болса, онда олардың абсолют шамасын табыңыз.

10. Нақты a саны берілген. $f(x)$ функциясына 1а-1г суреттерінде берілген графиктерді қолдана отырып, $f(a)$ функциясын есептеңіздер.

11. Нақты x, y сандары берілген. Егер x және y теріс сан болса, онда олардың әрқайсын квадраттаңыз; егер тек біреуі

$$f(x) = \begin{cases} x, & \text{егер } x \geq 1 \text{ немесе } x < -2 \\ x^2 + 2x - 2 & \text{баска жағдайда} \end{cases}$$



1 - сурет

теріс болса, онда оларды 10-ға кемітіңіз. Егер x және y оң сан болса және біреуі $[1;3]$ аралығында жатса, онда оларды түбірден шығарыңыз. Қалған жағдайда “Шарттың ешқайсысы орындалмады” деген мәлімдеме шығарыңыз.

12. Нақты a, b, c, d сандары берілген. Осы сандардың терістерін квадраттап, ал оң сандарын түбірден шығарыңыз.

13. Нақты a, b, c, d сандары берілген. Егер кем дегенде бір сан нөлге тең болса, ол жайлы экранға мәлімет шығарыңыз, кері жағдайда a -ның b -ға және c -ның d -ға қалдықсыз бөлінетіндігін тексеріңіз.

14. Бүтін a, b, c сандары берілген. Егер $a \leq b \leq c$ болса, онда барлық сандарды олардың квадратымен алмастырыңыз; егер $a > b > c$ болса, онда әр санды ең үлкен санмен алмастырыңыз, кері жағдайда барлық сандардың таңбасын кері таңбаға ауыстырыңыз.

15. Нақты x, y, z сандары берілген. $\max(x + y + z, x * y * z) + 10$ өрнегін есептейтін программа жазыңыз.

16. Нақты x, y, z сандары берілген. $\max(x^2 + y^2, y^2 + z^2)$

– 1 өрнегін есептейтін программа жазыңыз.

17. Бүтін k, l, m сандары берілген. Оң сандардың квадратының қосындысын есептеңіз. Егер бір де бір оң сан жоқ болса, ол жайлы экранға мәлімет шығарыңыз.

18. Бүтін x және y сандары берілген. Егер екі санда жұп болса оларға 1-ді қосыңыз; егер тек біреуі жұп болса, онда олардың көбейтіндісін табыңыз; қалған жағдайда сандарды өзгеріссіз қалдырыңыз.

19. Нақты x, y, z оң сандары берілген. Қабырғаларының ұзындығы x, y, z -ке тең үшбұрыш бар екенін тексеріңіз.

Сандарды енгізгенде олардың теріс емес және нөлге тең емес екендігін тексеріңіз.

20. Бүтін a, b, c, d сандары берілген. Нөлге тең емес сандардың көбейтіндісін табыңыз. Егер барлық сандар нөлге тең болса, экранға мәлімет шығарыңыз.

21. Бүтін a, b, c сандары берілген. Тақ сандардың қосындысын табыңыз. Егер барлық сандар жұп болса экранға мәлімет шығарыңыз.

22. Бүтін a, b, c, d сандары берілген. Егер $a \geq b \geq c \geq d$ болса, онда барлық c аңдарды нөлге теңестіріңіз; егер $a < b < c < d$ болса, онда әр санды 1-ге өсіріңіз; қалған жағдайда әр санды 1-ге кемітіңіз.
23. Бүтін x, y, z ($x \neq y, x \neq z, y \neq z$) сандары берілген. Осы сандардың ең кішісін тауып, оның жұп екендігін тексеріңіз.
24. Нақты a, b, c сандары берілген. $(0, +\infty)$ аралығында жататын сан үшін натуралды логарифмді есептеңіз (сан оң және нөлден үлкен болуы керек).
25. Адамның жасына қарап, оларды: әлі мектепке бармаған, мектеп оқушысы, жұмысшы, зейнеткер деген төрт топтың біріне жатқызатын программа құрыңдар. Адамның жасы пернетақтадан енгізіледі.

ПРАКТИКА / №5-6

Зертханалық жұмыс №5-6

Тақырыбы: Қайталау (цикл) алгоритмдері мен операторлары.

Мақсаты: Циклдік алгоритмді программалауды меңгеру

Жұмысты қорғау нәтижесі:

1. Берілген есептерді шығару програмаларын құру.
2. Орындалған жұмыс бойынша есеп беру алгоритм блок- схемасы, программа мәтіні және есептеу нәтижелерінен тұрады.

ЗЕРТХАНАЛЫҚ жұмысты қорғау сұрақтары

1. Арифметикалық цикл алгоритмі, for операторы, параметрлі цикл.
2. Қадамдық цикл алгоритмі, whileжәне do .. while операторлары.
3. While, do .. while және for операторларын салыстыру.

Тапсырмалар

1. Ең соңында 9999 саны тұрған бүтін сандар тізбегі берілген. Солардың:

- жалпы санын;
- арифметикалық ортасын;
- максималь мәнін;
- теріс және оң элементтері санын;
- минималь элементінің нөмірін;
- тізбек ішіндегі жұп сандар санын;

- жұп сандары ішіндегі ең кіші элементін анықтау керек.

2. n натурал саны берілген. Төмендегі мәндерді



- a) 2^n (pow() функциясын қолдануға болмайды);
- b) $n! = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$;
- c) $\left(1 + \frac{1}{1^2}\right) \left(1 + \frac{1}{2^2}\right) \dots \left(1 + \frac{1}{n^2}\right)$
- d) $\frac{1}{\sin 1} + \frac{1}{\sin 1 + \sin 2} + \dots + \frac{1}{\sin 1 + \dots + \sin n}$
- e) $(1 + \sin 0.1) (1 + \sin 0.2) \dots (1 + \sin n)$;
- f) $\frac{\cos 1}{\sin 1} * \frac{\cos 1 + \cos 2}{\sin 1 + \sin 2} * \dots * \frac{\cos 1 + \dots + \cos n}{\sin 1 + \dots + \sin n}$;
- g) $\sqrt{2 + \sqrt{6 + \dots + \sqrt{3(n-1) + \sqrt{3n}}}}$;
- h) $\underbrace{\sqrt{2 + \sqrt{2 + \dots + \sqrt{2}}}}_{(n \text{ түбір})}$

3. 2-кестеден вариант бойынша тапсырманы алып, шартты операторды қолданып бағдарлама жазу.

4. 3-кестеден вариант бойынша тапсырманы алып, таңдау операторын қолданып бағдарлама жазу. 4-бұтақты таңдау үшін default бөлігі бар нұсқаны қолдану керек.

2-кесте

Вариант №	Мазмұны	Мәндері	Σ
	$Z = \begin{cases} a + bx - cx^2 & k = 1, 2, 3 \\ d + ex + fx^2 & k = 4, 5 \\ ab + fx + cx^2 & k = 8 \end{cases}$	$\begin{aligned} a &= 2 \\ b &= 1, 5 \\ c &= 1 \\ d &= 3 \\ e &= 0, 5 \end{aligned}$	3
	$Z = \begin{cases} I + c\sqrt{d} & n = 0 \\ x - a & n = 1, 6 \\ 2/3x^2 - 1/2d & n = 2, 3, 4 \end{cases}$	$\begin{aligned} c &= -2 \\ a &= 1, 5 \\ d &= 2 \\ x &= 3 \end{aligned}$	
	$Z = \begin{cases} y^2 + 0,3a & x = 3 \\ a + e^{yb} & x = 5, 2 \\ y^2 + y - b & x = 6, 7, 8 \end{cases}$	$\begin{aligned} a &= 5 \\ b &= 1, 2 \\ y &= 0, 6 \end{aligned}$	4
	$Z = \begin{cases} 1 + d \sin d / a & j = 1 \\ (i - 1)i + a^2 & j = 2, 4, 8 \\ i + 2/3x & j = 5, 6, 7 \end{cases}$	$\begin{aligned} d &= 2 \\ a &= 3, 5 \\ x &= 3 \\ i &= 2, 3 \end{aligned}$	5
	$Z = \begin{cases} \omega/3 + a^2x & c = 3 \\ \omega - \ln b & c = 8, 9, 10 \\ b^2 + \omega x & c = 2, 5, 7 \end{cases}$	$\begin{aligned} \omega &= 1 \\ x &= 2, 5 \\ a &= 4 \\ b &= 0, 4 \end{aligned}$	

3-кесте

Вариант №	Мазмұны	Мәндері
1	$v = \begin{cases} a + 2/b + 4w & w = 4 \\ a - xw & w = 8 \\ (a + b)^2 & w = 2 \\ \text{шешімі жоқ} & w = 4, 8, 2 \end{cases}$	$a = 1.5$ $b = 2$ $x = 0.5$
2	$v = \begin{cases} 1.5x + 9x^2 + 1.25 & n = 2 \\ d + a \sin x & n = 5 \\ (a - x + y/d & n = 1 \\ \text{шешімі жоқ} & n = 1, 5, 2 \end{cases}$	$x = 0.5$ $a = 4.3$ $y = 2.6$ $d = 0.3$
3	$v = \begin{cases} 0.5a + \cos a / y & m = 1 \\ y^2 - i & m = 5 \\ x + x^2 / 2 - 1/3 & m = 7 \\ \text{шешімі жоқ} & m = 1, 5, 7 \end{cases}$	$a = 3$ $y = 2.7$ $i = 2$ $x = 1$
4	$v = \begin{cases} \sin x + e & k = 3 \\ (x + y)/(1 - xy) & k = 1 \\ x + t^2 & k = 0 \\ \text{шешімі жоқ} & k = 0, 1, 3 \end{cases}$	$x = 0.73$ $y = 0.4$ $t = 2.6$
5	$v = \begin{cases} a \sin x + c & d = 2 \\ R^2 & d = 6 \\ /a + bx/ & d = 3 \\ \text{шешімі жоқ} & d = 2, 3, 6 \end{cases}$	$a = 2$ $x = 0.54$ $c = 2.3$ $b = 1.8$ $r = 3$

5. Switch немесе if операторларын қолданып бағдарлама құрыңыз:

1. Айдың реттік нөмірі берілген. Экранға жыл соңына дейін қалған ай санын шығарыңыз.

2. Реттік нөмірмен айдағы күн берілген. Экранға ай соңына дейін қалған күн санын шығарыңыз.

3. Масти нөмірі берілген m (1 m 4), масти атың табыңыз. Масти нөмірленеді: «пики» - 1, «трефы» - 2, «бубны» - 3, «червы» - 4.

4. Карта нөмірі берілген k (6 k 14), картаның үлкенің табыңыз. Келесі шартпен анықталады : «туз» - 14, «король» - 13, «дама» - 12, «валет» - 11, «ондық» - 10, ..., «алтылық» - 6.

5. Масти нөмірі берілген m (1 m

4) және үлкен карталар нөмірі k ($6 \leq k \leq 14$). Толық атың келетін карта түрін «дама пик», «шестерка бубен» және т.б.

6. 1 қаңтар 1990 жыл бірнеше n күн өтті, қандай күн екенін анықтаңыз.

7. 1 қаңтар 1990 жыл бірнеше m ай өтті, қандай ай екенін анықтаңыз.

8. Бір уақыттан осы уақытқа дейін m ай өтті, белгісіз айды табыңыз.

9. Бір уақыттан осы уақытқа дейін m ай өтті, белгісіз айды табыңыз.

10. Бір уақыттан осы уақытқа дейін n күн өтті, белгісіз айды табыңыз.

11. 1 қаңтар 1990 жыл осы уақыттан бірнеше ай m бірнеше күн n өтті, белгісіз айды табыңыз.

12. Дәрігердің қабылдау уақытының кестесі берілген. Экранға дәрігердің қабылдау уақытың апта күнің шығар.

Жалпы тапсырма (Барлық студенттер төмендегі 5 тапсырманы орындауы керек)
Цикл операторларын қолданып есептеу.

1. For, while, do...while циклдық операторларын қолданып

$Y=0,6*Z+9X$ функциясының мәндерін есептеңіз, мұндағы X

айнымалысы 0 - ден 5 - ке дейін 0.2 қадаммен және Z айнымалысы 1- ден 3- ке дейін 0,1 қадаммен өзгереді.

2. Көбейту кестесін басып шығаратын бағдарлама құрыңыз.

3. Пернетақтадан енгізілген оң сандар тізбегінің қосындысы мен арифметикалық ортасын есептейтін программа жазыңыз. Төменде бағдарламаны орындау кезінде экранның ұсынылған көрінісі келтірілген (қолданушы енгізген мәліметтер қалың қаріппе n көрсетілген). Оң сандар тізбегінің арифметикалық ортасын есептеу.

Бағыттаушыдан (стрелкадан) кейін сандарды енгізіңіз.

Жазбаны аяқтау үшін нөлді енгізіңіз.

-> 45 -> 23 -> 15 -> 0 Енгізілді: 3 сан Сандар қосындысы: 83 Арифметикалық ортасы: 27.67

4. Көбейту кестесі туралы білімдерін тексеру үшін бағдарлама жазыңыз. Бағдарламада 10 мысал келтірілуі керек және 10 дұрыс жауап үшін - «өте жақсы», 9 және 8 жауаптар үшін - «жақсы», 7 және 6 жауаптар үшін -

«қанағаттанарлық», 6 және одан төмен - «нашар» деген баға беру керек. Төменде бағдарлама жұмыс істеп тұрған кезде ұсынылған экран көрінісі берілген.

Пайдаланушының жауаптары қалың қарамен көрсетілген.

*** Көбейту кестесі жайлы білімдерін тексеру ***

$5 \times 3 = 15$

$7 \times 7 = 49$

$1 \times 4 = 4$

$4 \times 3 = 12$

☐ 1☐ 4☐ k☐ 1☐ 1☐ 4☐ k☐ 1

$9 \times 4 = 36$

$8 \times 8 = 64$

$7 \times 8 = 52$

Сіз қателестіңіз!

$7 \times 8 = 56$

$4 \times 7 = 28$

$3 \times 5 = 15$

$2 \times 5 = 10$

Дұрыс жауаптар: 9 Баға: Жақсы.



ПРАКТИКА / №7

Зертханалық жұмыс №7

Тақырыбы: Тізімдер, тіркестер.

Мақсаты: Java тілінде тізімдер мен тіркестерді өңдеуге бағдарламалар құру.

Жұмысты қорғау нәтижесі:

1. Берілген есептерді шығару програмаларын құру.
2. Орындалған жұмыс бойынша есеп беру алгоритм блок- схемасы, программа мәтіні және есептеу нәтижелерінен тұрады.

Жұмысты қорғау сұрақтары

1. Сөз тіркестерінің сипатталу тәсілдері.
2. Сөз тіркестерін енгізу (шығару) функциялары.

Тіркестерді өңдеу.

3. Тіркестердің ең ұзын, қысқа сөздерін, ұзындығын, сөздері санын, т.б. анықтау.
4. Бір тіркестік айнымалыға немесе тұрақтыға қанша символ жазуға болады?
5. Тіркестік өрнектер дегеніміз не?
6. Тіркестік айнымалылар мен тұрақтыларға қандай амалдар қолданылады?

Сөз тіркестерін өңдеуден қысқаша мәліметтер.

Символдық жолдарды немесе тіркестерді бірнеше тәсілмен өңдеуге болады, олардың негізгілері:

1. Тіркестік константаларды қолдану;
2. Char типті жиымды қолдану;
3. Char типіне сілтейтін нұсқауыштарды пайдалану;
4. Символдық тіркестерден тұратын жиымдарды қолдану.

Сөз тіркестері немесе тіркестік (жолдық) константа қостырнақшаға алынып жазылады. Тырнақшаға алынатын символдар тізбегінің ең соңына автоматты түрде ‘\0’ символы жазылады. Компилятор жолдық символдарды компьютер

жадына жазғанда, олардың көлемін анықтау үшін сол символдар санын есептейді. Символдық константа осы сөз тіркесі жаз ылған жады аймағына сілтейтін нұсқауыш болып табылады. Символдық тіркестер жиымын (массивін) беру кезінде компилятор компьютер жадының қажетті көлемін анықтау үшін жиымды сипаттағанда, оны тіркестік константа арқылы инициалдауға болады.

```
Мысалы: char c[] = "Атырау, Алтай - жеріміз";
```

Әдеттегі жиым қолданылатын жағдайлар сияқты бұл жиым аты с осы жиымның 1-элементіне сілтейтін нұсқауыш болып табылады.

```
c == &c[0];  
*c == '0';  
*(c+1) == c[1] == 'n';
```

Сөз тіркестерін анықтау үшін нұсқауыштарды мынадай түрде сипаттауға болады:

```
char *c1 = "\n студенттер саны";
```

осы сипаттауға эквивалентті болып келесі сипаттау есептеледі:

```
static char c1[] = "\n студенттер саны";
```

Осы қарастырылған екі сипаттау да c1 тіркесінің нұсқауыш екенін білдіреді. Компьютер жадының қажетті көлемін айқын көрсетуге де болады. Мысалы, сыртқы сипаттауда келесі жолдың мынадай түрде жазылғаны көрсетілген.

```
char c[25] = "Білім - өмір шырағы";
```

Элементтердің саны жолдың ұзындығынан бір символ артық болуы керек, яғни оның ең соңында '\0' символы болуы тиіс.

Символдар тіркесін пернелерден енгізу үшін екі стандартты функция – scanf() немесе gets() қолданылады, ал олардың прототиптері stdio.h тақырыптық файлында болады.

Символдық таңбаларды енгізу/шығару Символдарды біртіндеп енгізу/шығару үшін printf() және

```
scanf()
```

функцияларының %s форматы қолданылады.

getch() – параметрсіз функция, басылған перненің кодын (int) береді, экранға ешқандай символ шығармайды.

`getchar()` – параметрсіз функция. Пернеден символдарды бір- бірлеп енгізеді.

Сөз тіркесі `<Enter>` пернесі басылғанша енгізіле береді, оған дейін оны өзгертуге де болады.

`putch(c)` – бір символды (`c` – символдық айнымалы немесе константа), яғни бір таңбаны ғана экранға шығарады.

`putchar(c)` – бұл да тек бір таңбаны экранға шығарады.

Бұлар `conio.h` тақырып файлы бойынша жұмыс істейді.

Мысалы, латын алфавиті әріптерін экранға шығару программасы төмендегідей болады:

```
#include <conio.h>
#include <stdio.h>
void main()
{char z; clrscr();
for(z='A';z<='Z';z++)
putchar(z);
getch();
}
```

Сөз тіркестерін енгізу функциялары `scanf()`, `gets(str)`.

`Scanf()` функциясы тіркестік айнымалылар мәнін `%s` форматымен енгізеді, бірақ ол тіркесті тек бірінші босорын таңбасына дейін ғана енгізе алады.

Ал `gets(str)` функциясы арасында босорыны бар тіркестерді енгізеді, енгізу ENTER пернесімен аяқталады.

Екі функция да тіркес соңына нөлдік байт қосып жазады.

Оның үстіне тіркес –символдық жиым болып, ал жиым аты

– оның компьютер жадындағы алғашқы адресіне сілтеме болғандықтан, тіркестік айнымалы атының алдына «&» символы жазылмайды.

Сөз тіркестерін шығару функциялары `cprintf()`, `puts()`, `cputs()` `printf()` – экранға формат арқылы сөз тіркесін

шығарады;

`cprintf()` – экранға `printf()` сияқты формат арқылы сөз тіркесін шығарады, тек олардың түстерін `textcolor()` және `textbackground()` функциялары арқылы өзгертуге

мүмкіндік береді;

`puts(str)` – экранға сөз тіркесін шығарып, курсорды бірден келесі жолдың басына алып барады, мұндағы `str` – тіркестік константа немесе тіркестік айнымалы. Бұлар `stdio.h` тақырып файлы бойынша жұмыс істейді.

`printf()` функциясы символ тіркесі шығарылған соң,

курсорды келесі жолға көшірмейді, ол үшін арнайы формат (`\n`) жазылуы тиіс. Ал `puts( )` функциясы символдар шығарылған соң, автоматты түрде курсорды келесі жол басына көшіреді.

//puts функциясын пайдалану мысалы:

```
#include <stdio.h>
#include <conio.h>
main()
{
    char str1[] = "abc";
    char str2[] = "def\nghi\n";
    char str3[] = "jkl";
    puts(str1);
    puts(str2);
    puts(str3);
}
```

`cputs(str)` – экранға сөз тіркестерін шығарып, олардың түстерін `textcolor()` және `textbackground()` функциялары арқылы өзгертуге мүмкіндік береді, `conio.h` тақырып файлы бойынша жұмыс істейді.

Сөз тіркестерімен орындалатын басқа операциялар да стандартты функциялар арқылы атқарылады. Ол функциялар жұмыс істеуі үшін `string.h`, `stdlib.h` немесе `string.h` тақырыптық файлдары қолданылады.

Орындауға арналған тапсырмалар

1 вариант

1. Құрамына сандар кіретін сөз тіркесінің ұзындығын – `L` анықтап, егер `L` жұп сан болса, онда тіркестегі барлық екілік сандарды өшіріп тастандар.
2. Берілген сөз тіркесінің дәл ортасында тұрған сөзді керісінше жазып шығындар.

2 вариант

1. Берілген сөз тіркесінің ұзындығын – L анықтап, егер L жұп сан болса, онда тіркестегі алғашқы сөзді өшіріңдер, ал тақ сан болса, соңғы сөзді өшіру керек.

2. Берілген сөз тіркесінде палиндром сөз бар екенін анықтап, ол жайлы мәлімет беру керек.

3 вариант 1. Берілген сөз тіркесінің ішінде D символдардың нешеуі екенін анықтаңдар. 2. Берілген сөз тіркесінің ұзындығын – L анықтап, соңғы сөзді өшіріп тастаңдар.

4 вариант

1. Берілген сөз тіркесінің ұзындығын – L анықтап, егер L тақ сан болса, онда тіркестің дәл ортасындағы символды анықтау керек, ал жұп болса – тіркес ортасындағы екі символды анықтау керек.

2. Берілген сөз тіркесінің ішіндегі барлық ! белгісін ?

белгісіне алмастырыңдар.

5 вариант

1. Берілген тіркес ішіндегі бос орын таңбаларын астын сызу(_) таңбасына алмастырыңдар.

2. Берілген сөз тіркесінің ұзындығын – L анықтап, егер $L > 10$ болса, соңғы сөзді өшіріп тастаңдар.

6 вариант

1. Берілген сөз тіркесінің ұзындығын – L анықтап, егер L 3- ке қалдықсыз бөлінетін болса, тіркестегі екінші сөзді өшіріп тастаңдар.

2. Берілген сөз тіркесінің ұзындығын – L анықтап, егер L 5- ке қалдықсыз бөлінетін болса, тіркестегі барлық жақша түрлерінің санын анықтаңдар.

7 вариант

1. Екі сөйлемнен тұратын сөз тіркесінің сөйлемдерінің орындарын алмастырыңдар.

2. Берілген сөз тіркесінің ұзындығын – L анықтап, егер L 3- ке қалдықсыз бөлінетін болса, тіркестегі екінші сөзді өшіріп тастаңдар.

8 вариант

1. Берілген сөз тіркесінің алғашқы нүктесіне дейінгі символдарды “с” әрпіне алмастырыңдар.

2. Берілген сөз тіркесіндегі бірінші және соңғы сөзді керісінше жазып шығыңдар.

9 вариант 1. Берілген сөз тіркесінің алғашқы сөзі мен екінші сөзін керісінше жазып шығыңдар. 2. Бірнеше сөйлемнен тұратын сөз тіркесіндегі екінші сөйлем ішіндегі “Е” әрпінің санын анықтаңдар.

10 вариант

1. Берілген сөз тіркесінде жақшалар бар. Алғашқы жақшалар ішіндегі сөзді анықтаңдар.

2. Берілген сөз тіркесіндегі ең ұзын сөздің енін тауып, оны керісінше жазып шығыңдар.

11 вариант

1. Сөз тіркесі берілген. Оның ішіндегі леп белгілерін нүктемен алмастырып, нүктелер санын анықтаңдар. Леп белгісі жоқ болса, ол туралы мәлімет беріңдер.

2. Құрамына сандар кіретін сөз тіркесінің ұзындығын – L анықтап, егер L жұп сан болса, онда тіркестегі барлық екілік сандарды өшіріп тастаңдар.

12 вариант

1. Сөз тіркесі берілген. Оның ішіндегі нүктелерді үш нүктемен алмастырып, нүктелер санын анықтаңдар, нүкте жоқ болса, ол жайлы мәлімет беріңдер.

2. Берілген сөз тіркесінің ұзындығын – L анықтап, егер L тақ сан болса, онда тіркестің соңғы сөйлемін анықтау керек, ал жұп болса – тіркес ортасындағы символды анықтау керек.

13 вариант

1. Сөз тіркесі берілген. Оның ішінде үтірлер бары белгілі.

Алғашқы үтір мен соңғы үтірдің қай позицияда тұрғанын және оларды арасында неше символ бар екенін анықтаңдар.

2. Бірнеше сөйлемнен тұратын сөз тіркесіндегі соңғы сөйлем ішіндегі “А” әрпінің санын анықтаңдар.

ПРАКТИКА / №8

Зертханалық жұмыс №8

Тақырыбы: Функцияны жариялау, `function()`-мен жұмыс.

Мақсаты: Функцияның көмегімен бағдарлама құруды меңгеру.

Жұмысты қорғау нәтижесі:

1. Берілген есептерді шығару програмаларын құру.
2. Орындалған жұмыс бойынша есеп беру алгоритм блок- схемасы, программа мәтіні және есептеу нәтижелерінен тұрады.

Жұмысты қорғау сұрақтары

1. Тұтынушы функциясын сипаттау дегеніміз не?
2. Егер функцияның параметрлері жоқ болса, ол қалай сипатталады?
3. Тұтынушы функциясы мен стандартты функциялардың айырмашылығы неде ?
4. Функция аргументтерін беру жолдарын атаңыз?
5. “Жергілікті” және “ауқымды” айнымалылардың айырмашылығы.
6. “Жергілікті” айнымалылардың әрекет ету аймағы ұғымы.

Қысқаша теориялық мәліметтер Java тілінде стандартты функциялармен қатар тұтынушы өзі құрастырған функцияларымен де жұмыс істеу мүмкіндігі жасалған. Ол функцияны алдын ала `main` функциясына



дейін толық анықтау керек немесе оны алдын ала қысқаша сипаттап алып, main функциясынан кейін толық анықтауға болады. Сондықтан, тұтынушы функциялары декларациясы екі түрде: қысқаша сипаттау түрінде (main функциясына дейін) және анықтау түрінде (main функциясынан кейін немесе дейін) толық берілуі мүмкін.

Функцияны алдын ала сипаттау барысында программалық файлдың басында оның прототипі көрсетіледі, өйткені main функциясында оның айнымалылары типтері белгілі болуы тиіс. Ол келесі түрі жазылады:

```
<нәтиже_типі><функция_аты>(<тип><айнымалы1>,  
<тип><айнымалы2>, ...<тип><айнымалыN>);
```

Прототиптегі жай жақшаларға алынған айнымалылар идентификаторларын көрсетпесе де болады, өйткені тілдің компиляторы оларды өңдемейді.

Параметрлер тізімімен берілген fun функциясын сипаттаудың мысалы:

```
float fun(int, float, int, int);
```

Функция прототипі компиляторға программа мәтінінде негізгі программдан (main()) кейін оның толық анықтамасы келтірілетінін білдіреді.

Функцияның толық анықталуының жазылу түрі төмендегідей:

```
<нәтиже_типі>< функция_аты >(параметрлертізімі) {функция коды}  
Нәтиже_типі return операторының көмегімен функцияның шақырылу нүктесіне  
қайтарылатын мәnnің типін көрсетеді.
```

Егер функция типі көрсетілмесе, қайтарылатын мән int типті деп саналады.

Параметрлер тізімі үтірмен ажыратылған типтер мен параметрлер аттарының тізбегінен тұрады. Егер функцияның параметрлері болмаса да, жай жақшалар бәрі бір міндетті түрде көрсетілуі тиіс.

Return операторы берілген функциядан бірден шығып, нәтиженің оны шақырушы функцияға қайтарылуын қамтамасыз етеді, яғни бұл оператор функция жұмысы нәтижесін қайтару қызметін атқарады. Функция тұлғасында бірнеше return операторы болуы немесе бірде бір рет кездеспеуі де мүмкін екендігін айта кеткен жөн. Мұндай жағдайда шақырушы функцияға қайту оның тұлғасындағы соңғы оператор орындалғаннан кейін барып жүзеге асырылады.

Екі бүтін санның ішіндегі кішісін анықтайтын функцияның мысалы:

```
int mini(int x, int y)
{int t;
if (x<y) t=x; else t=y;
return t;
}
```

Mini функциясын келесі түрде де жазуға болады: mini (int x, int y)

```
{
return (x<y)? x:y;
}
```

Екі бүтін санның ішіндегі үлкенін анықтайтын функцияның мысалы:

```
int maxi(int x, int y)
{
if (x>y) return (x);
else return (y);
}
```

Тапсырмалар Нұсқа бойынша есепті алып, функция ретінде келесі әдістермен шешу:

1. функция оны шақырғаннан кейін қойылған;
2. функция оны шақырғанға дейіннен кейін қойылған;
3. функция басқа файлда одан кейін қойылған;

вариант № Тапсырма 1 Берілген $A(5,5)$ квадратты матрицасы бас диагоналына симметриялды қатысты болпып табылатынын анықтау 2 $B(4,4)$ матрицасы берілген. Бірінші баған элементтерінің өсу бойынша сұрыпталғанын анықтау.

3 $C(5,5)$ матрицасы берілген. Әрбір қатардың бірінші элементі мен максимальды элементтерінің орындарын ауыстыру.

4 $D(3,3)$ матрицасының әрбір қатарының 10 –нан үлкен бірінші элементін B массивіне көшіріп жазу 5 $Q(5,5)$ матрицасы берілген. Әрбір қатардың соңғы нолін 5-ке ауыстыру.

6 $D(4,4)$ матрицасы берілген. Оң сандар арасында максималды санды, теріс сандар арасында минималды санды анықтау және олардың орындарын ауыстыру.

7 $A(4,4)$ матрицасы берілген. Әрбір бағанның бірінші нолін сол бағандағы нолдер саныны алмастыру.

8 $F(9,3)$ матрицасы берілген. Бірінші бағанның барлық элементтері бас диагональдағы сәйкес элементтерге тең болуын анықтау . Егер тең болмаса, олардың орнын ауыстыру.

9 $C(5,5)$ матрицасы берілген. Элементтері Матрицаның бағанындағы нолдер санына тең B векторын алу.

10 $B(4,4)$ матрицасы берілген. Егер қатарда ең болмағанда бір, бір деген саны болса, онда сол қатарды нөл санына ауыстыру.

11 $Q(3,3)$ матрицасы берілген. Егер басты диагональда нөл саны болса, онда сәйкесінше сол қатарды бір санына ауыстыру.

12 $D(4,4)$ матрицасы берілген. Егер матрицаның максимальды элементі бас диагональда тұрса, онда бас диагональдың барлық элементтерін максимальды элементпен алмастыру.

13 $C(5,5)$ матрицасы берілген. Егер минималды элемент бірінші қатарда болса, онда сол элементтен кейін тұрған барлық элементті нөл санына ауыстыру.

14 $A(4,4)$ матрицасы берілген. Егер матрицаның максимальды элементі бірінші қатар элементтер санына тең болса, онда бірінші қатарды максимальды элемент тұрған қатармен ауыстыру.

15 $A(4,4)$ матрицасы берілген. Егер матрицаның максимальды элементі бірінші қатар элементтер санына тең болса, онда бірінші қатарды максимальды элемент тұрған қатармен ауыстыру.

ПРАКТИКА / №9

Зертханалық жұмыс №9

Тақырыбы: Жиындар.

Мақсаты: Жиындармен жұмыс, жиындар бойынша тапсырмалар

Жұмысты қорғау нәтижесі:

1. Берілген есептерді шығару програмаларын құру.
2. Орындалған жұмыс бойынша есеп беру алгоритм блок-схемасы, программа мәтіні және есептеу нәтижелерінен тұрады.

Зертханалық жұмысты қорғау сұрақтары

1. Жиым және матрицаның сипаттамасы.
2. Жиым элементтерін енгізу (шығару). Жиым элементтерін нөмірлеу.
3. Жиымның ең үлкен, ең кіші элементтерін, қосындысын, көбейтіндісін, арифметикалық, геометриялық орталарын, т.б. анықтау.
4. Матрицаның шарт бойынша таңдалып алынған элементтерін өңдеу.

Жиымдарды өңдеу мысалдары 1 есеп. $A(10)$ жиымындағы оң элементтерді екі есе кемітіндер, ал теріс элементтері болса, онда оларды индекстерінің мәнімен ауыстырыңдар.

```
#include<conio.h>
#include<stdio.h>
#include<stdlib.h>
main()
{
```



```
const n=10;
float a[n];
clrscr();
randomize();
printf("\nЭл-тыисходногомассива:");
for(int i=0;i<n;i++)
{a[i]=random(20)-10;
Printf ("%2.0f ",a[i]);
}
Printf ("\nЭл-тыобработанногомассива:");
//random();
for( i=0;i<n;i++)
{if(a[i]>0) a[i]/=2;
else a[i]=i;
printf("%3.1f ",a[i]);
}
getch();
}
```

Тапсырмалар А. Әр есептің блок-схемасын және соған сәйкес программасын құрып, компьютерде сол программаның нәтижесін алу керек.

В. Екіблок- схема Word программасындағы Сурет салу- Автофигуралар -Блок-схемалар мүмкіндігімен сызылып көрсетілуі тиіс.

С. Берілген жиым элементтері мен нәтижелік элементтерді экранға шығарып, нәтижелерді жазып алу қажет немесе нәтижені бірден файлға жазу керек.

Жиымдарға қойылатын талаптар

1. Жиым өлшемі (размерность) тек константа немесе константалық өрнек бола алады. Жиым өлшемін идентификатор түріндегі ат қойылған константа арқылы беру дұрыс деп саналады.

2. Жиым элементтері нөлден бастап нөмірленеді, сол себепті элементтің ең үлкен нөмірі элементтер санынан бірге кемболады.
3. Индекс нөмірлерінің жиым шекарасынан асып кетпеуін программа қадағаламайды, оны программалаушы өзі бақылауы тиіс.
4. Нұсқауыш — жедел жады аймағының адресін сақтайтын айнымалы. Жиым аты оның 0- элементіне нұсқауыш болып табылады.
5. Егер программаға енгізілетін элементтер мәні алдын ала белгілі болса, онда жиымды сипаттау кезінде оның мәндерін көрсетіп (инициалдау) кету керек. Жиымды глобальді айнымалы емес, локальді айнымалы етіп сипаттаған дұрыс.
6. Жиым элементтерін пернеден енгізгеннен гөрі `rand()` және `random(n)` функцияларын пайдаланып, кездейсоқ сандар тізбегін беру арқылы енгізген ыңғайлы болады.
7. Жиымдарды сұрыптау алгоритмдерінің жұмыс істеу жылдамдығы, алатын көлемі және қолданылу аймағы әр түрлі болып келеді.

Тапсырмалар нұсқалары:

n нақты элементтерден тұратын бір өлшемді жиым берілген.

Жиым элементтерін төмендегі нұсқаларға сәйкес өңдейтін программалар құру керек.

1 нұсқа

1) жиымның теріс элементтерінің қосындысын;

2) жиымның ең үлкен (максимал) және ең кіші (минимал) элементтерінің арасында орналасқан элементтерінің

көбейтіндісін анықтаңдар;

3) Жиым элементтерін өсуі бойынша реттендер.

2 нұсқа

1) жиымның оң элементтерінің қосындысын;

2) жиымның модулі бойынша ең үлкен (максимал) және модулі бойынша ең кіші (минимал) элементтерінің арасында орналасқан элементтерінің көбейтіндісін

анықтаңдар;

3) Жиым элементтерін кемуі бойынша реттендер.

3 нұсқа

1) жиымның жұп нөмірлі элементтерінің көбейтіндісін;

2) жиымның алғашқы және екінші нөлге тең екі элементінің арасында орналасқан элементтерінің

қосындысын анықтаңдар;

3) Жиымның барлық оң элементтері оның алдыңғы жағында, ал теріс элементтері соңғы жағында орналасатындай етіп түрлендіріңдер.

4 нұсқа

1) жиымның тақ нөмірлі элементтерінің қосындысын;

2) жиымның алғашқы және ең соңғы теріс екі элементінің арасында орналасқан элементтерінің қосындысын

анықтаңдар;

3) Жиымның модульдері 1-ден кіші элементтерін жойып, қысқартыңдар да, жиымның соңында бос қалған элементтерін нөлдермен толтырыңдар.

5 нұсқа

1) жиымның ең үлкен элементін;

2) жиымның алғашқы теріс элементіне дейін орналасқан

элементтерінің қосындысын анықтаңдар;

3) Жиымның модульдері $[a, b]$ аралығында орналасқан элементтерін жойып, қысқартыңдар да, жиымның соңында бос қалған элементтерін нөлдермен толтырыңдар.

6 нұсқа

1) жиымның ең кіші элементі мен оның нөмірін;

2) жиымның ең алғашқы және ең соңғы оң элементтері арасында орналасқан элементтерінің қосындысын

анықтаңдар;

3) Жиымның алдыңғы жағында тек нөлге тең элементтері, сонан кейін барып басқалары орналаса- тындай етіп түрлендіріңдер.

7 нұсқа

1) жиымның ең үлкен элементі мен оның нөмірін;

2) жиымның ең алғашқы және ең соңғы нөлге тең элементтері арасында орналасқан элементтерінің

көбейтіндісін анықтаңдар;

3) Жиымның бірінші жартысында – оның тақ нөмірлі элементтері, ал екінші жартысында – жұп нөмірлі элементтері орналасатындай етіп түрлендіріңдер.

8 нұсқа

1) жиымның ең кіші элементінің нөмірін;

2) жиымның ең алғашқы және одан кейінгі теріс элементтері арасында орналасқан элементтерінің қосындысын

анықтаңдар;

3) Жиымның алғашқы жағында – оның модульдері 1 -ден кіші элементтері, ал соңында – қалған элементтері орналасатындай етіп түрлендіріңдер.

9 нұсқа

1) жиымның модулі бойынша ең үлкен элементін;

2) жиымның ең алғашқы және одан кейінгі оң элементтері арасында орналасқан элементтерінің қосындысын

анықтаңдар;

3) Жиымның нөлге тең элементтері оның соңғы жағында орналасатындай етіп түрлендіріңдер.

10 нұсқа

1) жиымның модулі бойынша ең кіші элементін;

2) жиымның нөлге тең алғашқы элементінен кейінгі

элементтерінің модульдерінің қосындысын анықтаңдар;

3) Жиымның бірінші жартысында – оның жұп нөмірлі элементтері, ал екінші жартысында – тақ нөмірлі элементтері орналасатындай етіп түрлендіріңдер.

11 нұсқа

1) жиымның модулі бойынша ең кіші элементі нөмірін;

2) жиымның алғашқы теріс элементінен кейін орналасқан

элементтерінің модульдері қосындысын анықтаңдар;

3) жиымның мәндері $[a, b]$ аралығында орналасқан элементтерін жойып, қысқартыңдар да, жиымның соңында бос қалған элементтерін нөлдермен толтырыңдар.

12 нұсқа

1) жиымның модулі бойынша ең үлкен элементі нөмірін;

2) жиымның алғашқы оң элементінен кейін орналасқан

элементтерінің қосындысын анықтаңдар;

3) жиымның мәндерінің бүтін бөлігі $[a, b]$ аралығында орналасқан элементтерін алдыңғы жағына, қалғандарын – соңғы жағына орналасатындай етіп түрлендіріңдер.

13 нұсқа

1) жиымның мәндері A мен B аралығында жатқан

элементтері санын;

2) жиымның ең үлкен элементінен кейін орналасқан

элементтерінің қосындысын анықтаңдар;

3) Жиым элементтерін оның модульдерінің кемуді бойынша реттеңдер.

Зертханалық жұмыс №10



Тақырыбы: Екі немесе көп өлшемді массивтер.

Мақсаты: Екі немесе көп өлшемді массивтерді (жиымдарды) өңдеу.

Жұмысты қорғау нәтижесі:

1. Берілген есептерді шығару програмаларын құру.
2. Орындалған жұмыс бойынша есеп беру алгоритм блок-схемасы, программа мәтіні және есептеу нәтижелерінен тұрады.

Зертханалық жұмысты қорғау сұрақтары

1. Жиым және матрицаның сипаттамасы.
2. Матрица элементтерін енгізу (шығару). Матрица элементтерін нөмірлеу.
3. Матрицаның ең үлкен, ең кіші элементтерін, қосындысын, көбейтіндісін, арифметикалық, геометриялық орталарын, т.б. анықтау.
4. Матрицаның шарт бойынша таңдалып алынған элементтерін өңдеу.

Жұмысқа арналған тапсырмалар А. Әр есептің блок-схемасын және соған сәйкес программасын құрып, компью-терде сол программаның нәтижесін алу керек.

В. Екі блок-схема Word программасындағы Сурет салу-Автофигуралар-Блок-схемалар мүмкіндігімен сызылып көрсетілуі тиіс.

С. Берілген жиым элементтері мен нәтижелік элементтерді экранға шығарып, нәтижелерді жазып алу қажет немесе нәтижені бірден файлға жазу керек.

1 вариант а. Екіөлшемді жиымның – $a_{2,5}$ ең үлкен элементін және оның индексін табындар.

б. $A[5][4]$ жиымы берілген. Оның бірінші және соңғы жолдарының орындарын алмастырындар.

2 вариант

1. Екіөлшемді жиымның – $c_{4,5}$ әрбір жолдарының қосындыларын табындар.

2. $C[5][5]$ өлшемді матрицаның жолдарының минимумдарының арифметикалық ортасын табатын программа құрындар.

3 вариант

1. Екіөлшемді $A(4,7)$ жиымындағы оң элементтердің арифметикалық ортасын есептеп шығарындар.

2. Бүтін санды $B[5][5]$ жиымы берілген. Жиым бас диагоналінің сол жағынан жоғары орналасқан оң таңбалы элементтерінің санын табындар.

4 вариант

1. Екіөлшемді $A(4,4)$ жиымының бас диагоналіндегі элементтердің ең үлкенін табындар.

2. Бүтін сандардың екі өлшемді $C(3,8)$ жиымындағы жолдардың максимал элементтерінің арасындағы минимал элементті табындар.

5 вариант

1. Екіөлшемді $A(4,4)$ жиымының қосымша диагоналіндегі ең үлкен элементті табындар.

2. Екі өлшемді $C(4,5)$ жиымының әрбір бағанасының арифметикалық ортасын табатын программа құрындар.

6 вариант

1. Екіөлшемді $A(4,4)$ жиымының қосымша диагоналіндегі ең кіші элементті табындар.

2. Екіөлшемді $A(6,4)$ жиымының берілген диапазонда ($a;b$) жататын элементтерінің арифметикалық ортасын есептеп шығарыңдар.

7 вариант

1. Екіөлшемді $A(5,10)$ жиымында оң сан көп пе, әлде теріс сан көп пе, соны анықтаңдар.

2. Екіөлшемді $A(7,7)$ жиымында бас диагональдағы элементтердің арифметикалық ортасын табыңдар.

8 вариант

1. Бүтін санды $B[5][5]$ жиымы берілген. Оның диагональдарында орналасқан элементтерінің ең кішісін табыңдар.

2. Екіөлшемді $A(4,4)$ жиымында бас диагональдан жоғарғы орналасқан элементтердің қосындысын есептеп шығарыңдар.

9 вариант

1. Екіөлшемді $A(4,4)$ жиымында бас диагональдан төмен орналасқан элементтердің көбейтіндісін анықтаңдар.

2. Екіөлшемді $A(4,4)$ жиымының берілген диапазонда ($a;b$) жататын элементтерінің максимумын табыңдар.

10 вариант

1. Екіөлшемді $A(4,4)$ жиымында 5-тен артық элементтердің қосындысын есептеп шығарыңдар.

2. Екіөлшемді $A(4,4)$ жиымының бас диагоналі элементтерінің ең үлкенін табыңдар.

11 вариант

1. Екіөлшемді $A(4,4)$ жиымының бас диагоналі бойындағы элементтердің ең кішісін табыңдар.

2. Екіөлшемді $A(4,4)$ жиымының қосымша диагоналіндегі ең үлкен элементті табыңдар.

12 вариант

1. Бес цехтың әрқайсысының 4 бөлімшесіндегі барлық шикізат туралы мәлімет кестесі берілген. Шикізаты ең аз цехтың нөмірін анықтаңдар.

2. Екіөлшемді $A(4,4)$ жиымындағы элементтердің арифметикалық ортасын анықтаңдар.

13 вариант

1. Екіөлшемді $A(7,7)$ жиымында бас диагональдағы элементтердің арифметикалық ортасын табыңдар.

2. Екіөлшемді $A(4,4)$ жиымында бас диагональдан жоғарғы орналасқан элементтердің максимумын табыңдар.

Зертханалық жұмыс №11

Тақырыбы: Сөздіктер.

Мақсаты: Сөздіктердің мысалдары, конкатенация, сөздіктердегі салыстыруды жүзеге асыру.

Жұмысты қорғау нәтижесі:

1. Берілген есептерді шығару програмаларын құру.
2. Орындалған жұмыс бойынша есеп беру алгоритм блок- схемасы, программа мәтіні және есептеу нәтижелерінен тұрады.

Жұмысты қорғау сұрақтары

7. Сөз тіркестерінің сипатталу тәсілдері.
 8. Сөз тіркестерін енгізу (шығару) функциялары.
- Тіркестерді өңдеу.
9. Тіркестердің ең ұзын, қысқа сөздерін, ұзындығын, сөздері санын, т.б. анықтау.
 10. Бір тіркестік айнымалыға немесе тұрақтыға қанша символ жазуға болады?
 11. Тіркестік өрнектер дегеніміз не?
 12. Тіркестік айнымалылар мен тұрақтыларға қандай амалдар қолданылады?

Сөз тіркестерін өңдеу тапсырмалары

1. Берілген сөздегі әр түрлі әріптердің санын есептейтін программа жазыңдар.
2. Сөздердің әрбір үшінші әрпін жоятын программа жазыңдар.
3. Берілген мәтіндегі “Айна” сөзін “Асыл” сөзіне өзгертетін программа жазыңдар.
4. Пернетақтадан енгізілген символды жазылған сөз тіркесінен өшіретін программа жазыңдар. Өшіру процесін жеке функция етіп қарастырыңдар.
5. Берілген мәтіндегі кездесетін “а” әрпін “о” әрпіне ауыстырыңдар.

6. Енгізілген сөз тіркесін керіге айналдыратын, яғни символдарды кері тәртіпте орналастыратын программа құрындар.
7. Енгізілген сөздің палиндром болатынын/болмайтынын анықтайтын программа құрындар.
8. Пернетақтадан енгізілген символдарды ASCII кестесінде нөмірлерінің өсу реті бойынша сұрыптайтын программа жазындар.
9. Бос орындармен бөлініп жазылған үш сөзден тұратын сөйлемдегі ең қысқа сөздің ұзындығын есептеп шығаратын программа жазындар.
10. Экранға жылжымалы жолды шығаратын программа жазындар.
11. Сөйлемдегі жүйелі жақшаға алынған мәтіндерді және жақшаның өзін өшіретін программа құрындар.
12. Ұзындығы 25 символдан артпайтын сөз тіркесін алып, одан мүмкіндігінше бірнеше жаңа сөз құрастырындар.
13. Сөз тіркесіндегі кездесетін “а” әрпін “ә” әрпімен ауыстыратын программа құру керек.
14. Пернелерден енгізілген сөздің ұзындығын анықтайтын программа жазу қажет. Программаны while do циклі арқылы ұйымдастырып, программа жұмысын аяқтауды ‘999’ тіркесін енгізу арқылы жүзеге асыру керек.
15. Берілген тіркес құрамындағы сөздер бос орын арқылы бөлініп жазылған деп есептеп, олардың ішіндегі ең ұзын сөзді табу керек.
16. Берілген мәтінде өзің қалап алған сөз қанша рет кездесетінін анықтау керек.
17. Берілген мәтін сөздерінде ең көп кездесетін әріпті табу қажет.
18. Берілген мәтін сөздерінің ең жиі ұшырасатын алғашқы әрпін анықтау керек.
19. Берілген сөз тіркесіндегі “а” әрпінің санын анықтайтын программа құрындар.
20. Берілген мәтін сөздері арасындағы бос орынды үтірмен алмастырындар.

Зертханалық жұмыс №12

Тақырыбы: Объектілі-бағытталған программалаудың негіздері, мүмкіндіктері

Мақсаты: Бағдарламалау тілінің класстары және объектілерімен танысу мақсатында жұмыс жасау.

Жұмысты қорғау нәтижесі:

1. Берілген бағдарламаны ортада орындау.
2. Орындалған жұмыс бойынша есеп бағдарламаны талдау жасап, нәтижені баспадан шығару

Жұмысты қорғау сұрақтары

1. Global класы
2. Math классы
3. Window классы
4. Фреймдер колДәріссы (window.frames)
5. Document класы (window.document)
6. Location класы (window.location)
7. Link (document.link) класы
8. History класы
9. MimeType класы
10. Navigator класы Global класы Global класы ECMAScript спецификациясының бөлігі болып табылады, оның негізгі мақсаты – глобальды әдістер мен қасиеттер қатарын бір объектке біріктіру. Әдістерге байланысты объектің өзі көрсетілмейді және ол конструктор болмайды. Осы объектінің қасиеттер мен әдістеріне келесілер жатады:

Қасиеттердің мағынасы Nan мағынасын асырайды NaN (Not A Number).

Infinity мағынасын асырайды Number.POSITIVE_INFINITY.

Әдістердің мағынасы escape Қатарды барлық платформада оқылатындай етіп түрлендіреді. eval Қарапайым өрнектерді де қатарға берілгендей орындайды.

JavaScript. isFinite Аргументтің соңғы сан болатынын анықтауға мүмкіндік береді.

isNan Аргументтің сан ба немесе жоқ болуын анықтауға мүмкіндік береді.

parseFloat Қатарды санға түрлендіреді. parseInt Қатарды бүтін санға түрлендіреді.

unescape escape функциясының нәтижесін түрлендіреді.

taint() және untaint() функциялары – қосылған және қорғалған берілгендердің бұрмалануында объект немесе сілтеме мағыналарының бұрмаланған көшірмелерін қайтарып береді, немесе, керісінше бұрмаланбаған.

escape және unescape функциялары –s қатарының көшірмелерін кодталған және сәйкесінше коды шешілген түрде қайтарып береді.

Кодтау – барлық бос орындар, тыныс белгілер және т.б.

%xx формасына ауыстыру, мұндағы xx – ISO-8859-1(Latin-

1) символ кодына тиісті екі 16-лық сан.

eval(s) функциясы –s қатарын JavaScript операторлар тізбектері сияқты түсіндіріп береді. (См. Также объект Object) getClass (Obj) функциясы – JavaScript типіндегі аргумент үшін JavaScript классының көшірмесін қайтарып береді.

Мысалы: var myJavaRClass=new java.awt.Rectangle() Var

myJavaRClass=getClass(myJavaRect) Java мен getClass() әдісін шатастырмай Var

```
myJavaRObject=myJavaRect.getClass()
```

Бұл – Java java.awt.Rectangle класының ұсынылуы isNaN(x) функциясы –x “Not a Number” сан еместігін тексеру. parseFloat(s) функциясы –Float(санға кірмейтін қатар басынан бірінші символға дейін). типіндегі s санын табады. Егер сан табылмаса, онда NaN (“Not a Number”) мағынаға қайтады. parseInt(s) – бұл да Integer үшін parseInt(s,n) –n негізіндегі бүтін сан үшін (2 ден 36). n=0 болғанда – parseInt(s). Бұл жағдайда автоматты түрде негіз табылады.

eval(s) функциясы eval(s) функциясы — JavaScript-ң қоса салынған функциясы.

JavaScript-ң бір немесе бірнеше операторларынан тұратын `s` қатарының аргументімен берілген кодты орындайды. Осы функцияны тек қана операторды орындау үшін ғана емес, өрнектерді есептеу үшін де қолдануға болады. Ол берілген кодтағы соңғы есептелінген өрнектің мағынасын қайтарады. `eval(s)` функциясы JavaScript-бағдарламада орындалатын кодтың динамикалық модификациялау мүмкіншіліктерін қамтамасыздандырады.

`isNaN(x)` функциясы Бұл функция `x` аргументі «сан емес» -тігін тексереді. Нөлді нөлге бөлу нәтижесін анықтайды. Бұл міндетті функция, өйткені JavaScript-те енгізуге болмайтын санды ұсынатын NaN резервтелген мағынаны береді. Ол `parseFloat(s)` және `parseInt(s)` орындалуының нәтижесін тексеру үшін пайдаланылады және арифметикалық қателерді барлығын тексереді, мысалы, нөлге бөлу.

`parseFloat(s)` функциясы `s` қатарында табылған синтактикалық анализді орындайды және бірінші санды қайтарады. `parseFloat(s)` `s` символда санды енгізуге болмайтын элементі кездессе, анализ тоқтатылады және мағына қайтарылады. `parseFloat(s)` саннан басталмаса бұл функция NaN- ға қайтады.

`parseFloat(s)` (`s` — синтаксистік талдауға жататын қатар және санға түрлендіру) `parseInt(s)` функциясы `parseInt(s)` `parseInt(s, негіз)` (`s` — негіз синтаксистік талдауға жататын қатар — бүтін негіз сан талдауына жатаы) Осы функция қатарды бүтін санға түрлендіреді. Көрсетілген санау жүйесі негізінде рұқсат етілмейтін сан `parseInt(s)` `s` символды қатар түрінде қабылдаса, онда синтактикалық талдау тоқтатылады және мағына қайтарылады. `S` саннан басталмаса `parseFloat` ұқсас `parseInt` NaN мағынасын қайтарады.

Егер негіз 10 деп берілген болса, онда `parseInt(s)` қатарды барынша ондық санда талдайды.. 8мағына барынша сегіздік сан талдауының талабына сай болады(бұл жүйеде 0 ден 7 дейінгі сандар қолданылады). 16 мағына барынша он алтылық сан талдауының талабына сай болады (0 ден 9 ға дейінгі сандар және A дан F ға дейінгі әріптер қолданылады). Негіз 2 ден 36 ға дейінгі кез келген сан болуы мүмкін. Егер негіз 0 -ге тең болса, онда `parseInt(s)` санның негізін қатардың құрамынан анықтауға тырысады.Егер қатар 0х-тен басталса, онда функция қатардың қалған бөлігін он алтылық сан түрінде талдайды.Егер қатар 0-ден басталса, онда функция қатарды сегіздік сан түрінде талдайды.

`Math` класы `Math` — тұрақты және әдістерден тұратын класс. Олар әдетте объектілерге арналған бейнемен шақырылады:

Math.тұрақты Math.функция(i..) Math класының тұрақтылары:

E — e саны (натуральды логарифмдердің негізі) LN10 — 10 натуральды логарифм (ln10 саны) LN2 — 2 натуральный логарифм (ln2саны) LOG10E —10 негізіндегі e логарифмі(log10e саны) LOG2E —2 негізіндегі e логарифмі (log2e саны) ("пи" саны) PI — тұрақты 2 саны) SQRT1_2 —2ден кері түбірден квадратты сан (1/ 2) SQRT2 — 2 квадрат түбір (Math класының әдістері (функция) abs(x)(x — кез келген сан немесе өрнек) — абсолютті мөлшерін есептейді acos(x) (x — -1.0 ден 1.0 аралығындағы сан немесе өрнек) — аркко синусты есептейді. Қайтаратын мағына 0 ден 10 радиан аралығында болады.

/2 радиан аралығында жататын мағынаны қайтарады.

/2 ден asin(x) (x — -1.0 ден 1.0 аралығындағы сан немесе өрнек) — арксинусті есептейді, - /2 радиан аралығында жататын мағынаны қайтарады.

/2 ден atan(x) (x — сан немесе өрнек) — радиандағы арктангенсті есептейді, - atan2(x,y)(x,y — тіктөртбұрыш координаттар жүйесінің нүктелер координатасы) — полярлық координаталарда (x,y) бұрыш нүктелерін есептейді.

Қайтарылатын мағына 0 ден 2 дейінгі радиан аралықта жатады.

□PI

□SQ

□SQ

□/2

□as

□/2

□

ceil(x) (x — кез келген сан немесе сандық өрнек) — бүтінге жақын үлкен жақты санды айналдырады cos(x) (x — радиандағы бұрыш) — косинусты есептейді; қайтарылатын мағына -1.0 ден 1.0 ге дейінгі радиан аралықта жатады.

$\text{Exp}(x)$ (x — сан немесе сандық өрнек) — e экспонентін есептейді.

$\text{Floor}(x)$ (x — сан немесе сандық өрнек) — бүтін санға жақын төмен жаққа қарай санды айналдырады.

$\text{Log}(x)$ (x — дұрыс сан немесе өрнек) — натуралды логарифмді есептейді.

$\text{max}(a,b)$ (a,b — сан немесе өрнек) — екі мағынаның үлкенін көрсетеді.

$\text{min}(a,b)$ (a,b — сан немесе өрнек) — екі санның кішісін көрсетеді.

$\text{pow}(x,y)$ — x -ті есептейді (бірінші аргументті дәрежеге көтереді) random — 0 ден 1 ге дейінгі аралықтағы кездейсоқ санды есептейді $\text{Math.round}(x)$ (x — кезкелген сан немесе өрнек) $\sin$ — синусты есептейді $\text{Math.sin}(x)$ (x — радиандағы бұрыш) sqrt — квадраттық түбірді есептейді $\text{Math.sqrt}(x)$ (x — 0-ге тең немесе үлкен кезкелген сан немесе өрнек) $\tan$ — тангенсті есептейді $\text{Math.tan}(x)$ (x — радиандағы бұрыш) Date класы $\text{Date}()$ әдісінің аргументін қолдана отырып, ағымдық күн мен уақытты көрсететін Date объектісін құрады. $\text{Date}()$ объектісінің объектілерімен жаңа объект үшін күн мен уақыт беріледі. Date объектісі JavaScript-те құралады және HTML-ға ұқсас болмайды. Әдетте Date объектісінің әдістері осы объектінің экземпляры арқылы шақырылады.

Мысалы:

```
d=new Date() //бүгінгі күн мен уақытты алу
system.write("Today is: "+d.toLocaleString());
//және осы ақпаратты сүреттеу
```

Date объектісін құру үшін синтаксистің келесі бес

вариантының біреуін қолдауға болады. `new Date();`

`new Date(миллисекунды)` (1.01.1970 дата мен жарты күн арасындағы миллисекунд) `new Date(дата қатары)` (дата қатары = аты_ай дд, гг [чч:мм[:сс]]) `new Date(жыл, ай, күн)` (жыл минус 1900; ай 0-11; күн 1-31) `new Date(жыл, ай, күн, сағат, минут, секунд)` (24-сағаттық жүйеде) Date класының әдістері `getDate()` (1 ден 30 диапазонында Date объектісінің санын көрсетеді) `getDay()` (0 [жексенбі] ден 6 [сенбі] диапазонында Date объектісінде апта күнін көрсетеді) `getHours()` (0 [түн жарым] ден 23 диапазонында Date объектісінің сағатын көрсетеді) `getMinutes()` (0 ден 59 диапазонда Date объектісінің минутыын көрсетеді) `getMonth()` (0 [қаңтар] до 11 [желтоқсан] диапазонда Date объектісінің айын көрсетеді) `getSeconds()` (0 ден 59 аралық диапазонда Date объектісінің секундын

көрсетеді) `getTime()` (`Date` объектісінің ішкі ұсынысын көрсетеді) `getYear()` (`Date` объектісінің жылын көрсетеді) `parse()` (датаның қатардық көрінісін синтаксистік талдайды және оның мағынасын миллисекунд форматында көрсетеді) `setDate()` (`Date` объектісінің күнін орнатады) `дата.setDate` (ай саны) //ай саны тең 1-31 және т.б. `toLocaleString()` (жергілікті сағаттық поясты қолданып, `Date`-ті қатарға түрлендіреді) `UTC()` (датаның сандық спецификациясын және уақыттың миллисекундтық форматын түрлендіреді)

`Window` класы Бұл объект терезені немесе `Web-браузер` кадрын ұсынады.

`Window` объектісінің контекстінде `JavaScript` коды сияқты интерпретацияланады, бұл объект `JavaScript`-тің барлық қажетті объектілеріне сілтеме жасауы қажет. `Window` объектісінің көптеген қасиеттері басқа қажетті объектілерге сілтеме болып табылады. Әдетте өзінің осы қасиеттері берілген терезеге сай объектілерге қатысты болады.

Мысалы, `location` қасиеті берілген терезенің `location` объектісіне қатысты. `Window` басқа қасиеттері (мысалы, `navigator` — осы және басқа терезелерге қатысты `Netscape` объектісіне сілтеме) глобальды объектілерге, ал бір-екі — тек осы терезенің өзіне қатысты болады.

`JavaScript` пайдаланушылық вариантында `Window` объектілеріне айналдыру үшін ешқандай арнайы синтаксис қажет емес және бұл объектіне қасиетіне айнымалыдай айналдыру мүмкін. Ағымдық терезеге сай `Window` объектісінің әдістерін функция ретінде пайдалануға болады.

(мысалы, `alert()` терезесінің орнына `alert()`).

`self` ағымдық терезе `window` ағымдық терезе Терезедегі кадрға өту үшін келесі әдістер қолданылады:

`frame[i]` (немес `self.frame[i]`) терезе.`frame[i]` Берілген кадрдың ата-аналық терезесіне өту үшін келесі варианттар қолданылады:

`parent` (немесе `self.parent`, `window.parent`) терезе.`parent` (берілген кадрдың ата-аналық терезесі) Кез келген құрамды жоғары дейгейлі браузер терезесіне назар аудармақшы болсақ, онда келесілер қолданылады: `top` (немесе `self.top`, `window.top`) Жоғары деңгейдегі жаңа терезе `Window.open()` әдісімен құрылады. Пайдаланушы бұл әдісті шақырған кезде айнымалыны `open()` шақыру нәтижесін сақтау және бұл айнымалыны жаңа терезеге айналдыру үшін қолдану қажет.

Терезе ашу және мәтінді динамикалық құру open әдісі бар терезені бейнелейді немесе жаңа терезе ашады. Терезе `open([URL, [аты, [мінездемелер]]])` Егер аргумент аты сол терезенің атымен берілсе, онда сілтеме осы терезеге өтеді. URL тапсырмасы бойынша сілтеме өткізілген терезе документ болып көрінеді. Егер URL — бос қатар болса, онда бос терезе ашылады.

Егер атақ берілген терезеге қатысты болмаса, онда бұл аргумент жаңа терезенің атын береді. Атақты `<a>` және `<form>` тегтерінің `target` атрибутінің мағынасы ретінде қолданса, онда құжаттарды осы терезеге ауыстыруға болады.

Мінездемелер — терезеде бейнеленуі қажет болған мінездемелердің тізімі. Егер бұл аргумент берілмеген болса, онда терезеде барлық мінездемелер бейнеленеді. Бұл қатарда бос орын болмауы керек.

```
myWindow=window.open("myFile.html")
```

 Мінездемелердің

тізімі:

```
open("URL","windowName","toolbar=no,scrollbars=yes,resizab
```

```
le=yes")
```

Эквиваленттік құруға `yes` немесе `1`, немесе `no`-да құрылған деп логикалық параметрлер қарапайым айтылған болуы мүмкін: `toolbar=yes` немесе `no` (немесе `1`, немесе `0`) – браузердің аспаптар панелі, клавиштер: "Back", "Forward" және т.б.

```
location –URL енгізу аймағы;
```

`directories` — клавиш сызығы "What's New", "What's Cool" және т.б.

`status` — төменгі жақтағы қатар жағдайы `menubar` — үстінгі меню сызғышы

`scrollbars` —құжат бұрғылағыш жолағы `resizable` — терезе өлшемінің өзгеруі.

Басқа параметрлер:

```
width= пиксельде терезе ені height= пиксельде терезе
```

```
биіктігі
```

Мысалы:

```
function windowOpener()
{
MyWindow=window.open("", "MyWindow", "menubar=yes, loca
tion, status");
MyWindow.document.bgColor="aqua";
MyWindow.document.write("<head><title>MyWindow</title>
</head>");
MyWindow.document.write("<center><big><b>Hello, world!</b
></big>
</center>");
}
```

Терезе жабылуы.

window.close() немесе self.close() — ағымдық терезені жабу. Егер бұл фрейм болса, онда барлық терезе жабылады.

Терезені былай жабуға да болады:

MyWindow.close()

Фреймдер кол Дәріссы (window.frames) Кейде frames объектіне сілтеме жасалғанымен, негізінде мұндай объекті жоқ. Терезедегі барлық кадрлар window объектінің экземпляры болып табылады, олар window объектісінің қасиеттеріне, әдістеріне және оқиға өңдеушілеріне ие болады және қолданады. Бірақ тәжірибеде жоғары деңгей терезесінде кадрлар ұсынатын window объектілері мен браузер терезесінде кадрлар ұсынатын window объектісінің арасында бір неше ерекшеліктер бар:

Егер кадрлар үшін defaultstatus орнатылса, онда қалып- күй туралы хабарлама тек тышқан курсоры сол кадрда тұрса ғана пайда болады. Жоғары деңгей браузер терезенің тор және parent қасиеттері жоғары деңгей терезесінің өзіне қатысты болады. Бұл қасиеттер тек кадрлар үшін шын мәнінде пайдалы. Window объектілері үшін кадр болып табылатын Close() әдісінің пайдасы шамалы:

Терезе.frames[i] Терезе.frames.lenth Frames[i]

Мысал:

<frameset

Rows=" биіктік1, биіктік2, ..." Cols=" ені1, ені2, ..." >

<frame src="фрейм_үшін_файл_адресі"

name="фрейм_аты">

```
<frame src="фрейм_үшін_файл_адресі"
```

```
name="фрейм_аты2">
```

```
...
```

```
</frameset>
```

Src –экранға шығарылатын URL кескін берілетін, оқу және жазу мүмкіндігі бар қатар. Үнсіздік бойынша фреймдер өлшемдері пиксельдерде. Егер соңында %тұрса, онда негізгі терезенің жалпы биіктігі немесе енінің %-те.

URL фреймдерде (#) белгісі қолданылмайды. Фрейм терезесіне жүктелетін құжаттың өзі де фреймдерден тұруы мүмкін.

Document класы (window.document) Document класының экзemplяры Window объектісінің Document өрісінде.

Шақыру: аты терезе. Document немесе жай Document (ағымдық терезе үшін)
Өрістер:

alinkcolor – HTML сәйкес параметріне қара vlinkcolor -- тағы anchors[i] – Anchor класының объектілер массиві. (бір-бірден контейнерге <a>...) anchors.lenth - бұл массив ұзындығы (RO) applets[i] – апплеттер массиві (Java объектілерінің) (біртіндеп контейнерге <applet></>) applets.lenth бұл массив ұзындығы (RO) bgcolor – фон түсін беретін қатар fgcolor – құжат мәтінінің түсін беретін қатар domain – құжат түскен жерден домен аты қатары embeds[i] - Plug-In қосымшалар массиві (сәйкес контейнерлерге) <embed> ...</embed бір бірден контейнерге>) Plugins[i] - ол да (синоним) Embeds.lenth - бұл массив ұзындығы(RO) Plugins.lenth – ол да Forms[i] - Form объектілерінің массиві (бір бірден контейнерге <form> ...</form>) Referrer – пайдаланушы ағымдық құжатқа жеткен жерден URL құжаты бар қатар (RO) Title - контейнердің мазмұнын көрсетеді <title> ...</title> (RO) URL – URL құжаты бар қатар (RO)

Әдістер:

Document.clear() - аты w болған терезе немес фреймді тазалау Document.close() – экранға буферден ақпарат қалдықтарын шығарады және құжатқа шығыс ағымын жабады Document.open()- құжатқа кіріс ағымын ашады және write(...) операторымен экранға (құжатқа) мәтін жазуға мүмкіндік туады Document.write(мән1,мән2,...) - құжатқа мәндер жазу Document.writeln -write(...) ұқсас, бірақ келес қатарға өту символын қоюмен. Тек жазық мәтінді форматтау үшін мәні бар.

Мысалы,

`<body.... onLoad="...." onUnload="...." >` денесіндегі оқиғалар өңдеушілерінің.

Document клас ының ағымдық экземпляры document деп аталады. Бұл берілген моментте экранға шығарылған сол жай ғана HTML құжат. Document объектісі window объектісінің document өрісінде сақталады. Ағымдық терезенің document объектісіне қатынас жасағанда (яғни, JavaScript бағдарлама орындалатын терезе) терезе сілтемесін тастап кетуге және жай document қолдануға болады.

Жоғарыда айтып өтілгендей, document объектісінің көмегімен құжатты «нөлден» бастауға болады.

Фреймдер бар болғанда window фрейм ішінде ағымдық терезеге қатысты болатынын ескеру керек. Фреймдерді шақыру фрейм аттары арқылы жүруі мүмкін.

Оқиғи немесе оқиға өңдеушілері:

onBlur – пернетақтадан енгізу фокусын жоғалту onClick – компонент немесе гиперсілтеме бойынша шерту onChange – енгізуден кейін мәннің өзгеруі onFocus – пернетақтадан енгізу фокусын алу onLoad – құжаттық жүктелуі onMouseOver – тышқан гипербайланыс немесе белгі үстінде onMouseOut – тышқан гипербайланыс немесе белгі облысы шекарасынан шығады onSelect – енгізу өрісінде мәтінді ерекшелеу onSubmit – форманы алу onUnload – пайдаланушы құжаттан шығады.

..... Мысал:

```
s=" URL"
...
<a href="..." onClick="this.href=s"
onMouseOver="window.status=ең жақсы тораптық адрес;
return true"
```

ең жақсы тораптық адрес>

HTML мен салыстыру

`<body Background="Bgimg.gif" Bgcolor="aqua" Text="black"`

```
Link="blue"
Alink="red" Vlink="green"
onLoad="оқиға өңдеушісі" onUnload=" оқиға өңдеушісі">
</body>
```

Background параметрі – фон қызметін атқаратын сурет.

Ол кез- келген тораптық адрес бойынша болуы мүмкін.

Bgcolor – терезе фонының түсі Text – берілген құжатта үнсіздік бойынша мәтіннің түсі Link – таңдаулар болмаған гиперсілтеме түсі Alink – таңдалғаннан соң жүктеу жүріп жатқан белсенді гиперсілтеме түсі Vlink – таңдаулар болып қойған гиперсілтемелер түсі onLoad – құжатты жүктейтін JavaScript коды onUnload - құжаттан шығуға орындалатын код

Мысалы:

```
<body onLoad="document1.location=http://phys.runnet.ru"> ....
</body>
```

Түс беретін барлық атрибуттар үшін мән JavaScript тілі түсінетін стандартты түс аттарының бірі, мысалы, қызы түс үшін red, жасыл түс үшін green және т.с.с., немесе алты оналтылық (RRGGBB) саннан тұратын RGB форматта болуы мүмкін. Бұл жағдайда қызыл түс FF0000, жасыл - 00FF00 болып ұсынылады және т.с.с. RGB –ді пайдалану – түстің аты белгісіз болғанда пайдалы, мысалы, blanchalmond – бұл FFEBCD комбинациясы.

RGB форматтағы түстер:

Қара 000000 Зайтүн 808000 Ашық қызыл FF0000 Сирень FF00FF Ашық жасыл 00FF00 Сия көк 800080 Ашық көк 0000FF Ауа көк 00FFFF Қоңыр 800000 Жасыл күлгін 008080 Жасыл 008000 Ақ FFFFFFFF Төк көк 000080 Күлгін 808080 Сары FFFF00

Location класы (window.location) Бұл URL. Location объектісі қасиеттерінің әр бірі өзімен бірге берілген объектімен сипатталатын бір немесе бір неше URL фрагменттен тұратын, оқу және жазу мүмкіндігі бар қатарды ұсынады. Жалпы форматы:

Protocol://hostname:port/pathname?search#hash өрістер: Link қара Өдістер:

Reload() – егер құжат өзгерген болса оны қайта жүктеу Reload(B) – егер B бұльдік шамасы true мәніне ие болса, онда құжат өзгермесе де құжаттық қайта жүктелуі жүзеге асады.

Позиция. `Replace(URL)`–ағымдық құжатты жаңа құжатпен алмастырады. (браузер сеансы протоколында жаңа позицияны пішімдемей `URL` мен `newURL` ді).

`Window` объектісінің `location` қасиетінде құжаттың `URL`-ін беретін `location` класының объектісі бар. `Location` қасиетінің өзгеруі браузерді `URL` өзгерістерді оқуға мәжбүр етеді.

Жаңа `URL`- ді жүктеу үшін `location` қасиетіне қатарлық мән меншіктеледі немесе `location` объектісі қасиеттерінің кез келгенінің бірі орнатылады. `Href` қасиеті кең қолданылады. Егер `window. Location` объектісінің `hash` қасиетін жай орнатса, онда браузер жаңа якорға секіреді.

Егер пайдаланушы әлдеқашан қатынас жасаған `location` немесе `location. Href` қасиетін орнатқан болса, онда браузер кэштең бұл `URL`- ді жүктейді немесе сервермен байланысып, бұл құжаттың өзгергенін тексереді, егер өзгерсе, оны қайта жүктейді. `Location` объектісін қасиеттері әртүрлі бөліктерге жатады және келесі форматқа ие:

`Protocol://hostname:port/pathname?search#hash` `Hash` хеш – `URL` бөлігі (`#` бастапқы белгісін қосатын) Бұл бір `HTML` файлдағы якор атының бөлігі `Host` – `URL` де хост немесе порт атының комбинациясы `Hostname` - `URL` де хост аты `Href` – толық `URL` `Pathname` – `URL`-де толық атау `Port` - `URL`-де порт `Protocol` - `URL`-де протокол (соңғы қос нүктені қосатын) `Search` - `URL`-де іздеу немесе сұраныс элементі (бастапқы сұрақ белгісін қосатын)

`Link (document.link)` класы `Location` объектісінің ішкі класы, экранда гиперсілтемені немесе `HTML`-құжатта клиент жағынан бейненің сезімтал облысының фрагментін ұсынады. Бұл `Location` объектісінің ішкі класы, бірақ ол жаңа `URL`-ді автоматты жүктеуді қамтамасыз етпейтінімен ерекшеленеді. `JavaScript`- те гипермәтінді сілтеме өзімен `Link` объектін, ал сілтеменің атанған адресі - `Anchor` объектін ұсынады.

Шақыру: `document.links[i]` Аймақ: `hash` – `URL` хэш-бөлік, `#` белгі қосумен `Host` – хост атының комбинациясы және `URL` порт нөмері `Hostname` –`URL`-дағы хост аты `Href` – толық `URL` `Pathname` –`URL` ға толық жол `Part` –`URL` порт `Protocol` –`URL` протоколы (“:” соңғы белгіні қосумен) `Search` –`URL`-дағы сұраныс элементі (“?” бастапқы белгіні қосумен) `Target` – гиперсілтеме үшін `target` атрибуты Оқиға өңдеушілері: `OnClick()` `Onmouseout()`– тышқавнның кетуінде гиперсілтемемен `Onmouseover()` – гиперсілтемеге орнату

Мысалы:

```
<a onMouseOut="...">...</a> <map...> <area onMouseOver="..."
onMouseOut="..." .....
> </map>
```

Link объекті onClicк қосумен <a> және стандартты тектерімен құрылады. onMouseOut және onMouseOver. href атрибуты Link-тің барлық объектілері үшін міндетті.

```
<a href="URL"
```

```
name="тег_якоря" Anchor объектінің құрады; target="терезе
```

```
аты" көрсетілген сілтеме құжатты бейнелейтін терезе.onClicк="өңдеуші"
сілтемені шерткенде шақырады.
```

```
onMouseOver="өңдеуші" тышқан курсоры сілтемеде
```

болғанда шақырады. onMouseOut="өңдеуші" тышқанның курсоры сілтеме сыртында болғанда шақырады.

> Мәтін немесе бейне Тышқанды шерткенде пайдаланушы жақтағы бейнеге қатысты Link класының объектінің <area> тегімен де құруға болады.

```
<map name="облыс аты" <area share="облыс формасы" coords=coordinates
```

```
href="URL"
target="терезе аты" көрсетілген сілтеме құжатты
бейнелейтін терезе;
onClicк="өңдеуші" onMouseOut="өңдеуші"
>
...
</map>
```

History класы URL-ға тапсырма беретін осы кезге дейін кірістірілген (RO) қатар массивінің жазылуынан қорғау. Осы тізімнің құрамы Go менюінде саналған Netscape барузерімен URL эквивалентті. History объектінің көмегімен пайдаланушы терезеде өзінің Forward және Back батырмаларын және басқа да басқару органдарының навигацияларымен іске асыруға болады.

Шақыру : терезе.history Фрейм.history History URL орынымен (RO) қатар массиві. Берілгендер аймағы:

current - URL ағымдағы құжат (RO, P – тек қана қосылған қорғалған берілгендердің бұрмалануында ыңғайлы) length

– тізімдегі URL саны next - URL құжат тізімінде келесіні (RO,P) previons – алдындағы үшін Әдістер:

back() - Басуға эквиваленттік Back пернелері немесе history.

go(- 1) командасы forward() – Forward үшін, эквивалентті history.go(1) go(n) - URL –ға тізімде n позициясымен салыстырмалы өту go (подстрока) – URL-ға өту toString() - (P) форматтағы тізіммен қатарды HTML-ға қайтарады.

MimeType класы браузер қолдаулары және оның Plug-In қосымшаларымен берілгендердің типін көрсетеді.

Шақыру: navigator.mimeTypes[i] navigator.mimeTypes[“аты”]
navigator.mimeTypes.length массивіндегі mimeTypeе объекттер саны Аймақ:
description – (RO) типтің қысқа жазылуы EnabledPlugin – осы типке қатысты Plugin объектіге сілтеме, немесе Null – егер мұндай объект жоқ болса.

Name – (RO) типтің аты mime (мысалы, “text/html” немесе “text/plain”) Suffixes – RO кеңейтілулер тізімі (мысалы, “html,htm”).

Мысалы:

```
Varshow_movie=(navigator.mimeTypes[“video/mpeg”]!=null);-
```

mPEGфайлға қатыстыларды тексеру

Navigator класы

– бір экземпльдерде болады. Шақыру: терезе.navigator Берілгендер аймағы:

AppCodeName – RO браузердің қадамдық атымен қатар AppName – RO браузер аты AppVersion – RO браузер версияларының қатары mimeTypes[i] – MimeType типтері мен объекттер тізімі.

mimeTypes.length – массив ұзындығы.

plugins[i] – дәл mimeTypes[i]-дай, бірақ тек қана Plug-In типтегі қосымшалар үшін.

plugins.length – массив ұзындығы.

userAgent – RO HTTP –сұраныс агентінің тақырыбы үшін қатар.

Әдістер:

JavaEnabled()—true, егер браузерде Java қолдауы қосылған болса.

TaintEnabled() – true, егер берілгендердің қорғалған бұрмалануы қосылған болса.

```
import java.io.*;

public class Employee {
    String name;
    int age;
    String designation;
    double salary;

    // This is the constructor of the class Employee
    public Employee(String name) {
        this.name = name;
    }

    // Assign the age of the Employee to the variable age.
    public void empAge(int empAge) {
        age = empAge;
    }

    /* Assign the designation to the variable designation. */
    public void empDesignation(String empDesig) {
        designation = empDesig;
    }

    /* Assign the salary to the variable salary. */
    public void empSalary(double empSalary) {
        salary = empSalary;
    }

    /* Print the Employee details */
    public void printEmployee() {
        System.out.println("Name:" + name);
        System.out.println("Age:" + age);
        System.out.println("Designation:" + designation);
        System.out.println("Salary:" + salary);
    }
}
```

```
import java.io.*;
public class EmployeeTest{

    public static void main(String a
    /* Create two objects using c
    Employee empOne = new Employee
    Employee empTwo = new Employee

    // Invoking methods for each
    empOne.empAge(26);
    empOne.empDesignation("Senior
    empOne.empSalary(1000);
    empOne.printEmployee();

    empTwo.empAge(21);
    empTwo.empDesignation("Softwa
    empTwo.empSalary(500);
    empTwo.printEmployee();

}
```

<<Java Class>>
Employee (default package)
△ name: String △ age: int △ designation: String △ salary: double
● Employee(String) ● empAge(int):void ● empDesignation(String):void ● empSalary(double):void ● printEmployee():void

ПРАКТИКА / №13

Зертханалық жұмыс №13

Тақырыбы: Модульдер, модульдер негіздері.

Мақсаты: Модульдермен жұмыс: КҰРУ> ип нұсқауларымен қосу.

Жұмысты қорғау нәтижесі:

1. Теорияны меңгеру.
2. Орындалған жұмыс бойынша есепке бағдарламау жасап, нәтижені баспадан шығару

Жұмысты қорғау сұрақтары

1. Java тілінде құрылымдар мен жолдар қалай сипатталады?
2. Айырмашылықтары неде?
3. “Құрылымға нұсқауыш” термині дегеніміз не?
4. Нұсқауыштарды құрылымдарға қатысты қолдану ерекшеліктерін атаңыз.
5. Құрылымдармен жұмыс істеу ерекшеліктері.
6. Құрылым элементтерін экранға шығару мүмкіндіктері.

Java жиынтығы пакеттер (packages) жиынтығынан тұрады.

Әрбір пакет салынған пакеттері қамтуы мүмкін, сондай -ақ кластар мен интерфейстерден тұрады. Әрбір топтамасының өз кеңістігі атаулар бар, бұл аттас сыныптарды әр түрлі пакеттерде құруға мүмкіндік береді. Аттар кейде жай (simple) бір сәйкестендіргіш аттан тұратын және құрамдас (qualified), тұрған келген ретпен берілген нүктемен ажыратылған идентификаторлардан тұрады,. Кез-келген пакетін элементінің құрама аты сы пакетін құрамдас атығы және қарапайым элементі атынан құралады. Қарапайым тәсілімен ұйымдастыру пакеттер мен типтері болып табылады әдеттегі файлдық құрылымы.

Мысалы, бастапқы код класы `space.sunsystem.Moon` файлда сақталады `space\sunsystem\Moon.java` бағдарламаны іске қосу JAVA, пакеттерде орналасқан директориядан жүргізіледі.

Жаваны тікелей папкалар `space\sunsystem` арқылы іске қосу қате болар еді. Оның `Moon` класында орналасқанына қарамастан. Қажетті файлды пакеттен табу үшін Java ға директориядан екі деңгей жоғары көтерілу керек.

Компиляция модулі Модуль компиляциясы `java-` файлда мәтін күйінде сақталады, компьютерге кіретін бірден-бір дерек болып есептеледі үш бөліктен тұрады:

- Пакетті хабарлау;
- `Import` өрнек;
- Жоғарғы деңгейді хабарлау;

Пакетті хабарлау төменде хабарланатын типтердің қай пакетке тиесілі екенін білдіреді. Пайдаланылады кілт сөз

`package`, кейін оның толық аты-жөні топтамасы.

Мысалы, файлда `java/lang/Object.java` жүріп жатыр: `package`

`java.lang;`

Бұл бір уақытта `lang` пакетін `java` пакетінде және `java.lang.Object` пакеті оның ішіне кірістірілгенін жариялау.

Бұл `java.lang.Object.` класының толық аты.

Типтің көріну аймағы - орналасқан пакеті, онда ол орналасқан. Осы ішкі пакетте оны қарапайым атымен іздеуге болады. Барлық басқа пакеттердің арасында оны құрама атымен табуға болады.

Осы проблеманы шешу үшін `import-өрнек` енгізіледі. Ол типтерді модуль компиляциясына импорттайды. Кейін қарапайым атпен табуға мүмкіндік береді. Ондай өрнектің екі түрі бар:

- `import java.net.URL` мекенжайын;
- `import java.awt.*`

Орындалатын тапсырмалар

Нұсқа	Есептің шарттары
1	Қазақстан, Қырғызстан Түркменстан.Өлкедегі тұрғын орналасуының тығыздығы, мектеп, емхана, ЖОО саны максималды болатын өлкенің атауын баспаға шығару.
2	Белоруссия, Украина,Ресей. Өлкедегі тұрғын орналасуының тығыздығы, мектеп, емхана, ЖОО саны минималды болатын өлкенің атауын баспаға шығару.
3	Грузия, Армения, Әзербайжан. Өлкедегі тұрғын орналасуының кему реті бойынша баспаға шығару.
4	Қазақстан, Өзбекстан, Түркменстан. Өлкедегі тұрғын орналасуының өсу реті бойынша баспаға шығару.
5	Тәжікстан, Молдавия, Украина. 1000 адамға шаққандағы өледегі тұрғындардың орналасу тығыздығы, мектеп, емхана санын көрсетіп, өлкелердің тізімін баспаға шығару.
6	Армения, Молдова, Белоруссия. Емхана саны минималдық өлкелердің атауларын баспаға шығарыңыз.
7	Ресей, Тәжікстан, Грузия.1000 адамға шаққандағы емханалардың санының өсу реті бойынша өлкелерді орналастырыңыз.
8	Түркменстан, Армения, Қазақстан.1000 адамға шаққанда еңкөп ЖОО келетін өлке атауын, республиканың ауданы және тұрғындарының тығыздығын баспаға шығару.
9	Қырғызстан,Украина,Әзірбайжан. 1000 адамға шаққанда келетін ЖОО, емхана, мектептер санын көрсете отырып, тұрғындардың орналасу тығыздығының өсу реті бойынша өлкелердің тізімін баспаға шығарыңыз.
10	Грузия, Молдавия, Украина. 1000 адамға шаққандағы мектептер санының кему реті бойынша осы өлкелер туралымәліметтерді баспаға шығарыңыз.
11	Қазақстан, Белоруссия, Грузия. 1000 адамға шаққандағы емхана санын көрсете отырып, тұрғындардың орналасу тығыздығының өсу реті бойынша өлкелердің тізімін баспаға шығарыңыз.

		23	1000 адамға шаққандағы мектептердің шамасының бойынша ТМД елдерінің барлық тізімін баспаға шығару.
12	Өзбекстан, Ресей, Тәжікстан.1000 адамға шаққандағы мектептер санын көрсете отырып, тұрғындардың орналасу тығыздығының бойынша өлкелердің тізімін баспаға шығарыңыз.	Әкімдер саны	1000 адамға шаққандағы мектептер саны
13	Қазақстан, Молдавия, Қырғызстан. 1000 адамға шаққандағы емханалар саны ең көп болатын өлкелердің мінездемесі мен атауын баспаға шығарыңыз.		
14	Армения, Украина,Әзірбайжан. Тұрғындардың орналасу тығыздығы минималды болатын өлкелердегі мектептер мен емханалар саны туралы мәліметтерді баспаға шығарыңыз.		
15	Туркмения, Өзбекстан, Белоруссия.Тұрғындардың орналасу тығыздығы максималды болатын өлкедегі ЖОО туралы мәліметті баспаға шығару.		
16	Молдавия, Туркмения, Армения.1000 адамға шаққандағы емханалар саны ең көп болатын өлкенің атауын баспаға шығарыңыз .		
17	Ресей, Грузия, Қырғызстан.100 адамға шаққандағы мектептер санының өсу реті бойынша өлкелердің атауларын баспаға шығару.		
18	Тәжікстан, Әзірбайжан, Туркмения.1000 адамға шаққандағы ЖОО санының кему реті бойынша өлкелердің атауларының тізімін баспаға шығару .		
19	Украина, Грузия, Қазақстан. 1000 адамға шаққандағы емханалар санының өсу реті бойынша өлкелердің атауларын барлық мінездемелерімен баспаға шығарыңыз.		
20	1000 адамға шаққандағы ЖОО, емхана, мектептер саны, тұрғындардың орналасу тығыздығы бойынша ТМД елдерінің барлық тізімдерін алфавит бойынша баспағашығару.		
21	ТМД елдерінің ішіндегі территориясының ауданы ең көп алғашқы бес мемлекеттердің тізімін баспаға шығарыңыз .		
22	1000 адамға шаққандағы ЖОО, емхана, мектептер саны, тұрғындардың орналасу тығыздығын көрсете отырып, ТМД елдерінің тізімін алфавиттің кері реті бойынша баспаға шығару .		

өсу реті

кему реті

	бойынша ТМД елдерінің алғашқы сегіз елінің тізімін баспаға шығару.
25	1000 адамға шаққандағы ЖОО санының өсу реті бойынша ТМД елдерінің алғашқы сегіз елінің тізімін баспаға шығару
26	1000 адамға шаққандағы ЖОО шамасы ең көп болатын ТМД елдерінің алғашқы сегіз елінің тізімін алфавит бойынша баспаға шығару.
27	1000 адамға шаққандағы мектептер шамасы ең көп болатын ТМД елдерінің алғашқы сегіз елінің тізімін алфавиттің кері реті бойынша баспаға шығару.
28	Қырғызстан, Туркмения, Әзірбайжан.Тұрғындардың орналасу тығыздығының кему реті бойынша өлкелердің мінездемелерін баспаға шығару .
29	Қазақстан, Ресей, Белоруссия, Қырғызстан. Тұрғындардың орналасу тығыздығының кему реті бойынша өлкелердің мінездемелерін баспаға шығару.
30	Ресей, Украина, Белоруссия. 1000 адамға шаққанда келетін мектептер шамасы бойынша республикалардың мінездеме атауларын баспаға шығару.

Тақырыбы

Максаты:

құруға сәйес

Жұмысты

1. Теория

2. Орында

нәтижені б

Жұмысты

Зертханалық жұмыс №14

Mathlab тәрізді өзге кітапханалары қолдану.

Математикалық мен графиктерді, модульдерді

қорғау нәтижесі:

меңгеру.

жұмыс бойынша есепке бағдарламау жасап, шығару қорғау сұрақтары

1. IBM PC компьютерлері бейнемониторының қандай түрлері бар?
2. Олардың айырмашылығы неде?
3. Графикалық режимнің атқаратын қызметі қандай?
4. Оны программада қалай іске қосады?
5. Графикалық режим қандай функция арқылы орнатылады?
6. Монитордың мәтіндік және графикалық режимдерінің мүмкіндіктері.
7. Экранның түсін және оған шығарылатын символдардың түсін қалай өзгертуге болады?
8. Курсорды экранның кез келген нүктесіне қалай көшіреміз?
9. Графикалық режимде экранда қалай терезе құруға болады?
10. Графикалық режимде экранды қалай тазартуға болады?
11. Clrscr, Clreol, Deline және Incline функцияларының ортақ қасиеттері мен айырмалары.

Қысқаша теориялық мәліметтер Java тілінде растрлық графика жұмыс істейді, оның тақырыптық файлы graphics.h. График нүктелерден – пиксельдерден тұрады. Пиксель – экранның адрестелетін ең кіші элементі. Алдымен графика шығара алатын бейнережимді іске қосу керек. Мұнда экранның пиксельмен берілген мөлшері және түстер саны беріледі.

Графикада үш координаталық жүйе: абсолюттік, салыстырмалы және масштабталған жүйе қолданылады.

Тіке және көлденең өстер бойынша пиксельдер саны экран типіне байланысты болады.

Абсолюттік координатада координаталар басы – $(0;0)$ нүктесі сол жақ жоғарғы бұрышта болып саналады, x координатасы солдан оңға қарай, y координатасы жоғарыдан төмен қарай өседі.

Салыстырмалы режимде координаталар басы экранның кез келген нүктесіне ауыстырыла алады.

Масштабталатын режимде экран бетінде масштабталған координаталар беруге болады, онда x пен y өстері бойынша минимум және максимум мәндер енгізіп, жұмыс істеуге мүмкіндік бар.

Жалпы абсолюттік графикалық режимде

– әрбір пиксель берілген 16 түстің біріне боялады;

– $(0,0)$ –экранның сол жақ жоғарғы бұрышы

координатасы;

– $(639,479)$ – оң жақ төменгі бұрышы координатасы

болады;

– әр түрлі фигураларды экранға шығару үшін алдын ала графикалық режимді іске қосып алу керек.

Жалпы дисплей адаптерлері графикалық режимде 200, 350, 600 нүктелерден тұратын экран жолдарының әрқайсысында 640, 720, 800 нүктелер тізбегін бейнелей алады. Мұндағы нүкте деп отырғанымыз – көлемі $0,8 \times 1 \text{ мм}^2$ шамасында болып келген (CGA) кішкентай тіктөрбұрыш, яғни пиксель.

Әрбір нүктенің координаталары екі бүтін санмен (X , Y) анықталады. Дисплей экранына график салу үшін оның нүктелерінің координаталарын көрсету қажет.

Координаталар басы $(0,0)$ болып экранның сол жақ жоғарғы бұрышы есептеледі. X координаталары (бағаналар немесе позициялар нөмірлері) солдан оңға қарай, ал Y мәндері (жолдар немесе қатарлар) жоғарыдан төмен қарай өсіп отырады. Мысалы, VGA адапте- рінің экран бұрыштарының координаталарын $X=0..799$, $Y=0..599$ аралығында көрсету қажет. Экранда X өсі солдан оңға қарай, Y өсі

жоғарыдан төмен қарай бағытталған, ал оның шеткі нүктелерінің координаталары суретте көрсетілген.

Олардың ең жоғарғы мәндері пайдаланылған экран адаптеріне тәуелді болады, яғни (0,0)..(320x200), (0,0)..(640x480), (0,0)..(800x600) аралықтарында және т.б.

болуы мүмкін. Сонымен, графикалық режимде экрандағы кез келген объект көрініп тұрған нүктелер тобынан тұрады.

Мәтіндік режимнен графикалық режимге көшкенде экран тазартылады.

Графикалық режимде экраннан курсор көрінбейді. Дисплей экранының бетіне (кейін қағазға) нүкте, түзу немесе қисық сызық, шеңбер, эллипс және кез келген тұйық сызық сызып шығаруға болады. Сонымен қатар тұйық сызықтардың ішін әр түрлі түске бояп қою мүмкіндіктері де бар. Сызықтарды жылжыту, айналдыру және басқа орынға көшіру арқылы көрнекі бейнелер мен мультфильмдер жасауға болады.

Графикалық режимдегі жұмыстарды атқаратын ТурбоС нұсқасында бірсыпыра графикалық функциялар бар, енді біз солардың негізгілеріне тоқталып өтеміз.

Графикалық режимде жұмыс істеу былай басталады:

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>
void main ( )
{
    int gdriver = DETECT; //драйвер int gmode; //режим аты int
    errorcode; // қате коды initgraph (&gdriver,&mode,"C:\\TC\\bgi\\"); errorcode =
    graphresult();

    if errorcode != grOk) //іске қосу қатесі

    {
        printf("Қате: %d\n", errorcode);
        puts("Аяқтау үшін Enter басыңыз");
        getch ();
        return;
    }
    ... Ары қарай программа мәтіні... getch(); closegraph ( );
```

Тапсырмалар

1. Уақытты есептеп отыратын сикыршымен кездестіңдер делік және ол сендерге жұлдыздар сырын ашып береді деп ойлайық. Неге аспанда жұлдыздар жымыңдайды?

Аспандағы әрбір жұлдыз—бір адамның өмірі. Сендерге көне ұлдыздарды дәл уақытында өшіріп, жаңасын дер кезінде жағып отыру керек. Уақыт есептеушінің жұмысын көрсететін программа жазу керек. Есеп модель құрудан басталады: экрандағы әрбір жұлдыз нүктемен көрсетіледі және барлығы 20 жұлдыз жанып тұрсын делік (кездейсоқ түрде таңдап алынған): жанып тұрған бірінші нүктені өшіріп, оны кез келген басқа орыннан жандыру керек, содан кейін екіншісін, т.с.с. Жиырмасыншы жұлдыз өшкеннен кейін қайтадан біріншісіне көшу керек.

2. “Жаңбыр” программасын жазыңдар — экранда (кездейсоқ күйде) 50 нүкте жоғарыдан төменге қарай қозғалып келе жатқандай болуы тиіс. Нүктелердің бірі экранның төменгі шекарасына жеткенде, ол қайтадан экранның жоғарғы бөлігінде пайда болуы керек.

3. “Жаңбыр” программасын кейбір нүктелер шапшаң, кейбіреулері баяу қозғалатындай етіп өзгертіндер.

4. “Жаңбыр” программасын нүктелер өз қозғалысын төменгі сол жақ бұрыштан бастап, жоғарғы оң жақ бұрышта аяқталатындай етіп өзгертіндер.

5. Қағазға оралған “кәмпиттің” бейнесін: диагональдары жүргізілген горизонталь тіктөртбұрыш пен оның екі жағына жалғасып жатқан тең қабырғалы үшбұрыштар салыңдар.

Тіктөртбұрыштың сол жақ төменгі төбесінің координатасы (x, y) , биіктігі a -ға, ұзындығы $2a$ -ға тең болсын.

Үшбұрыштың биіктіктері $a/2$. Тікбұрыштың диагональдары мен қабырғаларынан пайда болған қарама-қарсы екі үшбұрышты бояңдар.

6. Үш тісті және жарты дөңгелек пішінді аркасы бар мұнара салыңдар. Мұнараның сол жақ төменгі төбесінің координатасы (x, y) , табанының ұзындығы a . Қалған өлшемдерін a арқылы өрнектеп табыңдар. Мұнараны бояңдар.

7. Куб салыңдар. Кубтың алдыңғы сол жақ төменгі төбесінің координатасы (x, y) , бүйір қыры. Жоғарғы бүйір бетін бояңдар, оң жақ бүйір бетінде диагональ жүргізіндер.

8. Алтыбұрышты тік пирамида салыңдар. Табанының алдыңғы сол жақ төбесінің координатасы (x, y) , табанындағы қыры. Қалған өлшемдерін a арқылы өрнектеп табыңдар. Пирамиданың екі бүйір жақтарын бояңдар.

9. Тіктөртбұрышты қиық пирамида салыңдар. Пирамида табанының төменгі сол жақ төбесінің координатасы (x, y) , табанындағы қыры a , жоғарғы жақтың қыры — $a/2$. Қалған өлшемдерін a арқылы өрнектеп табыңдар. Пирамиданың оңжақ бүйір бетін бояңдар.

10. Қарайтылған “Е” әрпі тәрізді фигура салатын программа құрыңдар. Әріп құрайтын нүктелердің абсциссалары мен ординаталары жиым элементтері ретінде берілген. Сурет салу үшін циклге енгізілетін тек қана бір LINE операторын пайдаланыңдар.

11. Экранда боялған шырша салатын программа жазыңдар.

Сурет элементтерін жиымда сақтап, өшіріп және содан кейін қайтадан экранға шығарыңдар.

12. Боялған 7 шыршаның суретін салатын программа жазыңдар. Шыршалар бір-біріне ұқсас, горизонталь бойында бір қалыпта орналасады, шыршалардың биіктігі солдан оңға қарай сызықтық түрде өсуі керек. Шыршаларды бойлап Қызыл Телпек оларды уақытша “үсіп қалмасын” деп бір затпен жауып шығатын болсын.

13. Графикалық режимде ESC пернесін басқанша монитор экранына кездейсоқ күйде бірсыпыра нүктелер шығаратын программа құрыңдар.

14. Графикалық режимде ESC пернесін басқанша монитор экранында $(100, 100)$ — $(300, 200)$ тіктөртбұрышының нүктелерін кездейсоқ күйде жоятын программа құрыңдар.

15. Графикалық режимде ESC пернесін басқанша монитор экранының центрі $(200, 200)$ нүктесімен дәл келетін және радиусы 80 болатын дөңгелек нүктелерін кездейсоқ күйде жоятын программа құрыңдар.

ПРАКТИКА / №15

Зертханалық жұмыс №15

Тақырыбы: Қателерді талдау (throwExceptions).

Мақсаты: Қателерді талдауды меңгеру.

Жұмысты қорғау нәтижесі:

1. Теорияны меңгеру.
2. Тапсырмалық бағдарламаларды жуктеп ескетулерді сараптау және рәсімдеу.

Жұмысты қорғау сұрақтары

1. Exceptions
2. Error
3. Throwable
4. ArrayIndexOutOfBoundsException
5. InputMismatchException
6. Finally
7. try...catch Ескерту деп бағдарлама барысындағы орын алған қателерді айтамыз.

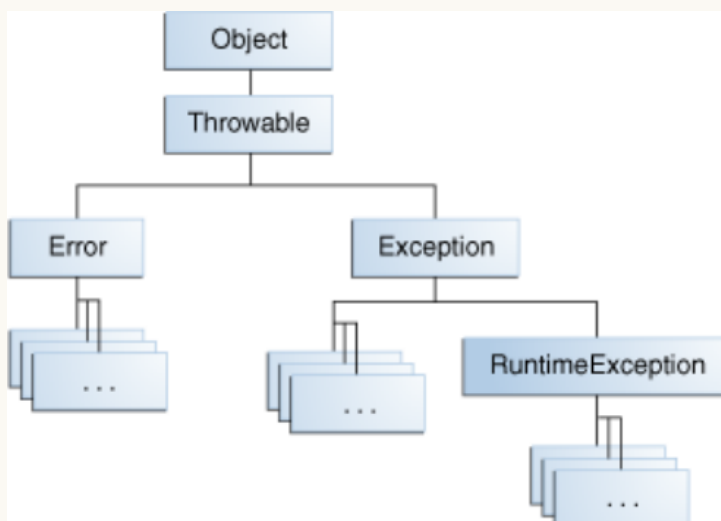
Ескертулер Java да объектілер. Сондықтан олар автоматты түрде ғана емес құрушымен жасалуы да мүмкін.

Ескертулер иерархия:

Ескертулер бірнеше кластарға бөлінеді: бірақ бәрі — класс Throwable. Олардың ізбасарлары ішкіклассы Exception және Error.

Ескерту (Exceptions) бағдарламадағы шешілетін алдын ала қарастырылған қиындықтар нәтижелері.

Мысалы 7/0



Ескерту (Errors) JVM деңгейіндегі күрделі қателерді сипаттайды. Мысалы, егер виртуалды машинада жады жетпей қалса. Бағдарлама JVM. ға жад бере алмайды.

Java да барлық ескертулер 3 типке бөлінеді: бақыланатын (checked), бақыланбайтын (unchecked) (Errors) қателеріне жататын және (RuntimeExceptions, Exception класының із басары). Бақыланатыни қателер Exception (бірақ RuntimeException емес).

Ескертулерді өңдеу try...catch операторларының көмегімен орындалады немесе бағдарламаның ішкі бөлігіне беріледі. Мысалы, қатені тәсіл жоғары иерархияға жібере салады, өзі өндеместен.

Бақыланбайтын қателер өңделмейді, дегенмен RuntimeException класының ескертулерін өндеуге болады.

Мына бағдарламаны енгізейік:

```

class Main {
public static void main(String[] args) { int a = 4;
System.out.println(a/0);
}
}

```

Жүктелу кезінде консольге мына хабарлама шығады:

Exception in thread "main" java.lang.ArithmeticException: / by zero at
Main.main(Main.java:4) Хабарламадан ArithmeticException ескертуін көреміз.
Мұны өндеуге болады:

```
class Main {
public static void main(String[] args) { int a = 4;
try {
System.out.println(a/0);
} catch (ArithmeticException e)
{
System.out.println ("Произошла недопустимая арифметическая
операция");
}
}
}
```

Енді стандартты хабарламаның орнына catch блогы қосылады, параметрі ескерту класына тура келетін. Ол бізге ескертуді ерік сіз басқа өңдеуші тексертуге қажет болады.

Осы кезде try блогына ескерту пайда болуы мүмкін бағдарлама фрагменті орналасады.

Бір try ға catch әртүрлі ескерту кластарынан тұратын бірнеше блок тұспа тұс келуі мүмкін.

```
import java.util.Scanner; class Main {
public static void main(String[] args) { int[] m = {-1,0,1};
Scanner sc = new Scanner(System.in);
try
{
int a = sc.nextInt();
m[a] = 4/a;
System.out.println (m[a]);
} catch (ArithmeticException e) {
System.out.println ("Произошла недопустимая арифметическая
операция");
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println ("Обращение по недопустимому      и ндексу массива");
}
}
}
```

Егер осы бағдарламаны жүктеп, қолданушы 1 немесе 2 ні клавиатурадан енгізсе бағдарлама ескертусіз орындап шығуы мүмкін.

Қолданушы 0 ді енгізсе, ArithmeticException ескертуі шығып, catch блогымен өңделеді. Егер қолданушы 3 ті енгізсе ArrayIndexOutOfBoundsException (массивтен шығып кету) класының ескертуі шығады және catch екінші блогымен өңделеді.

Егер қолданушы бүтін емес сан енгізсе `InputMismatchException` (енгізілген сан типінің сай келмеуі) класының ескертуі шығады және стандартты қателерге тасталады өйткені біз оны өңдеген жоқпыз.

Басқа барлық ескертулерді (оның ішінде `InputMismatchException`) аулап тұратын өңдеуші қосуға болады, өйткені `Exception` класы басқа бақыланатын ескертулерге ата-аналық болып есептеледі.

```
import java.util.Scanner; class Main {
public static void main(String[] args) { int[] m = {-1,0,1};
int a = 1;
Scanner sc = new Scanner(System.in);
try {
a = sc.nextInt(); m[a-1] = 4/a;
System.out.println(m[a]);
} catch (ArithmeticException e) {
System.out.println ("Произошла недопустимая арифметическая
операция");
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println ("Обращение по недопустимому индексу
массива");
} catch (Exception e) {
System.out.println ("Произошло ещё какое-то исключение");
}
}
```

Ескертулер иерархияда құрылған болғандықтан алдымен жекелеген содан соң жалпылама ескертулерді өңдеу керек.

Яғни `Exception` класының қатесін бірінші қойғанда біз ешқандай қателер жайлы хабарлама көрмес едік.

```
try...catch блогына міндетті емес finally блогын жатқызуға
```

болады. Ондағы командалар кез келген жағдай да орындалады, ескерту болса да болмаса да. Осындай кезде бағдарламаның бір бөлігі орындалмай тұруы мүмкін.

Мысалы, егер ескерту ұзақ есептеулердің салдарынан болса, онда `finally` блогында аралық нәтижелерді сақтауға болады.

08 / ӨЗІНДІК ТЕКСЕРУ

Білімді бақылау

15 тақырып · 150 теориялық сұрақ · 75 практикалық тапсырма

Әр тақырыптың теориялық бөлігі үш деңгейге бөлінеді: оңай — ұғымды түсіну; орташа — қолдану мен салыстыру; қиын — талдау және шешімді негіздеу. Практикалық тапсырмалар өзіндік жұмыс ретінде орындалады.

Ұсынылатын бағалау өлшемдері

Теория: анықтаманың дұрыстығы, ойдың дәлелділігі және тақырыпқа сай мысал.
Практика: алгоритмнің дұрыстығы, кодтың орындалуы, шекаралық деректермен тексеру және нәтиженің түсіндірмесі. Жауаптарды тек жатқа айтып емес, мысалмен негіздеңіз.

01 / Кіріспе

Дәріс 1. Білімді бақылау

Кіріспе

Оңай · білу және түсіну

01. Java тілінің негізгі мақсаты қандай және ол қандай есептерді шешуге қолданылады?

Дәріс 1. Білімді бақылау

02. JVM атауын толық ашып жазыңыз және оның қызметін түсіндіріңіз.

Дәріс 1. Білімді бақылау

03. Байт-код дегеніміз не? Ол бастапқы Java кодынан несімен ерекшеленеді?

Дәріс 1. Білімді бақылау

Орташа · қолдану және салыстыру

04. Дәрістегі «теориялық ядро» мен практикалық білімнің байланысын түсіндіріңіз.

Дәріс 1. Білімді бақылау

05. Бастапқы кодтан орындалатын бағдарламаға дейінгі кезеңдерді ретімен сипаттаңыз.

Дәріс 1. Білімді бақылау

06. Java бағдарламасының әртүрлі компьютерлік архитектурада орындалуын JVM қалай қамтамасыз етеді?

Дәріс 1. Білімді бақылау

07. Пәнді меңгеру алгоритмдік ойлауды қалай дамытады? Бір қолданбалы мысал келтіріңіз.

Дәріс 1. Білімді бақылау

Қиын · талдау және негіздеу

08. Платформаға тәуелсіздік үшін бірдей байт-код жеткілікті ме? Орындалу ортасына қойылатын шарттарды талдаңыз.

Дәріс 1. Білімді бақылау

09. Ақпарат өңдеу есебіне алгоритм құру мен сол алгоритмді Java тілінде іске асырудың айырмасын негіздеңіз.

Дәріс 1. Білімді бақылау

10. Экономикадағы есепті бағдарламалық шешімге айналдыру үшін бастапқы деректерді, өңдеуді және нәтижені қалай анықтайсыз?

Дәріс 1. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. HelloWorld.java файлын құрып, «Сәлем, әлем!» мәтінін шығарыңыз. javac және java командаларын көрсетіңіз.

Деріс 1. Білімді бақылау

02. Екі бүтін санның қосындысын табатын алгоритмді сөзбен жазыңыз және Java бағдарламасын құрыңыз. 7 және 5 үшін нәтиже 12 болсын.

Деріс 1. Білімді бақылау

03. Бағдарламаны әдейі синтаксистік қатемен компиляциялап, қате хабарламасын түсіндіріңіз; түзетілген нұсқасын көрсетіңіз.

Деріс 1. Білімді бақылау

04. Студенттің аты мен білім беру бағдарламасын екі бөлек жолға шығарыңыз. Бастапқы код пен консоль нәтижесін ұсыныңыз.

Деріс 1. Білімді бақылау

05. Қарапайым сатып алу есебін таңдаңыз: баға 1200, саны 3. Алгоритм, Java коды және 3600 нәтижесін көрсетіңіз.

Деріс 1. Білімді бақылау

02 / Тілдің негіздері, мәліметтер түрлері және өңдеу ортасы

Дәріс 2. Білімді бақылау

Тілдің негіздері, мәліметтер түрлері және өңдеу ортасы

Оңай · білу және түсіну

01. Айнымалы дегеніміз не және оны хабарлау кезінде қандай бөліктер жазылады?

Дәріс 2. Білімді бақылау

02. `int`, `double`, `boolean` және `char` типтері қандай мәндерді сақтайды?

Дәріс 2. Білімді бақылау

03. JDK қысқартуын толық ашып, оның құрамындағы әзірлеу құралдарын атаңыз.

Дәріс 2. Білімді бақылау

Орташа · қолдану және салыстыру

04. Дәрістері `main` әдісінің `public static void main(String[] args)` жазылуын бөліктерге талдаңыз.

Дәріс 2. Білімді бақылау

05. Java-дағы бас және кіші әріптердің айырмасы идентификаторларға қалай әсер етеді?

Дәріс 2. Білімді бақылау

06. `//`, `/* ... */` және `/** ... */` түсініктемелерінің мақсатын салыстырыңыз.

Дәріс 2. Білімді бақылау

07. Примитивті тип пен сілтемелік типтің айырмасын `String` мысалында түсіндіріңіз.

Дәріс 2. Білімді бақылау

Қиын · талдау және негіздеу

08. Бүтін санды `double` типіне және `double` мәнін `int` типіне түрлендіру кезіндегі дерек жоғалуын талдаңыз.

Дәріс 2. Білімді бақылау

09. Айнымалының көріну аймағы кодтың сенімділігіне қалай әсер етеді? Ішкі блок мысалымен негіздеңіз.

Дәріс 2. Білімді бақылау

10. Кластың аты, файлдың аты және компиляциялау командасы сәйкес келмесе қандай қиындық тууы мүмкін?

Дәріс 2. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Аты, жасы, орташа балы және студент мәртебесі үшін сәйкес типтерді таңдап, мәндерді экранға шығарыңыз.

Дәріс 2. Білімді бақылау

02. `int a = 7; double b = 2.5;` үшін `a + b` нәтижесін және оның типін көрсететін бағдарлама жазыңыз.

Дәріс 2. Білімді бақылау

03. 9.8 санын `int` типіне айқын түрлендіріңіз. Нәтижесін шығарып, бөлшек бөліктің не болғанын түсіндіріңіз.

Дәріс 2. Білімді бақылау

04. `HelloWorld` класын үш түрлі түсініктемемен толықтырыңыз. Бір әдіс үшін `javadoc` сипаттамасын жазыңыз.

Дәріс 2. Білімді бақылау

05. Екі ішкі блокта бір атаулы айнымалы қолданып көріңіз. Қай нұсқа компиляцияланатынын көрсетіп, көріну аймағын түсіндіріңіз.

Дәріс 2. Білімді бақылау

03 / Мәліметтерді енгізу және шығару

Дәріс 3. Білімді бақылау

Мәліметтерді енгізу және шығару

Оңай · білу және түсіну

01. `System.out.println()` әдісі қандай қызмет атқарады?

Дәріс 3. Білімді бақылау

02. `Scanner` класын қолдану үшін қандай пакет импортталады?

Дәріс 3. Білімді бақылау

03. `nextInt()`, `nextDouble()` және `nextLine()` әдістері қандай мәлімет оқиды?

Дәріс 3. Білімді бақылау

Орташа · қолдану және салыстыру

04. `print()` пен `println()` айырмасын консольдегі нәтиже арқылы түсіндіріңіз.

Дәріс 3. Білімді бақылау

05. Енгізілген деректердің типін алдын ала таңдаудың маңызы қандай?

Дәріс 3. Білімді бақылау

06. `nextInt()` шақыруынан кейін `nextLine()` бос жол қайтаруының себебін түсіндіріңіз.

Дәріс 3. Білімді бақылау

07. Пайдаланушыға енгізу нұсқауын көрсету бағдарламаға қандай пайда береді?

Дәріс 3. Білімді бақылау

Қиын · талдау және негіздеу

08. Консольге енгізілген жарамсыз мәтінді сан ретінде оқу кезінде қандай жағдай тууы мүмкін?

Дәріс 3. Білімді бақылау

09. Бос жолы бар толық есімді оқуға `next()` және `nextLine()` таңдауы қалай әсер етеді?

Дәріс 3. Білімді бақылау

10. Бірнеше дерек оқылатын бағдарламада енгізу ретін және нәтижені форматтауды қалай тексересіз?

Дәріс 3. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Scanner арқылы екі бүтін сан оқып, олардың қосындысын шығарыңыз. 12 және 8 енгізілсе, 20 шығуы тиіс.

Дәріс 3. Білімді бақылау

02. Студенттің толық аты мен жасын оқып, «Аты: ...; жасы: ...» түрінде шығарыңыз. Атының ішінде бос орын болсын.

Дәріс 3. Білімді бақылау

03. Тік төртбұрыштың ені мен ұзындығын double ретінде оқып, ауданын екі ондық таңбамен шығарыңыз. 2.5 және 4 үшін 10.00.

Дәріс 3. Білімді бақылау

04. nextInt() пен nextLine() бірге қолданылатын код құрыңыз; жол ауысуының қалдығын дұрыс өңдеңіз.

Дәріс 3. Білімді бақылау

05. Үш студенттің аты мен балын енгізіп, бағандары біркелкі шағын консоль кестесін шығарыңыз.

Дәріс 3. Білімді бақылау

04 / Базалық математикалық операциялар және есептеулер

Дәріс 4. Білімді бақылау

Базалық математикалық операциялар және есептеулер

Оңай · білу және түсіну

01. Java-дағы негізгі арифметикалық операторларды атаңыз.

Дәріс 4. Білімді бақылау

02. Бүтін сандарды бөлу мен нақты сандарды бөлудің айырмасы қандай?

Дәріс 4. Білімді бақылау

03. % операторы нені есептейді?

Дәріс 4. Білімді бақылау

Орташа · қолдану және салыстыру

04. Амалдардың басымдық ретін жақшасы бар және жақшасыз өрнек арқылы түсіндіріңіз.

Дәріс 4. Білімді бақылау

05. `Math.sqrt()` және `Math.pow()` әдістерінің аргументі мен нәтижесін сипаттаңыз.

Дәріс 4. Білімді бақылау

06. Инкремент пен декременттің префикстік және постфикстік түрлерін салыстырыңыз.

Дәріс 4. Білімді бақылау

07. Орташа арифметикалық мәнді есептегенде бүтін бөлуден қалай сақтануға болады?

Дәріс 4. Білімді бақылау

Қиын · талдау және негіздеу

08. Нақты сандарды есептеуде дөңгелектеу қатесі неліктен пайда болады?

Дәріс 4. Білімді бақылау

09. Квадрат теңдеудің дискриминанты теріс болса, есептеу алгоритмі қандай шешім қабылдауы тиіс?

Дәріс 4. Білімді бақылау

10. Формулары бағдарламалағанда тип, өлшем бірлігі және жарамды аргументтерді бірге тексерудің маңызын негіздеңіз.

Дәріс 4. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Шеңбер радиусы $r = 3$ үшін аудан мен ұзындықты Math.PI арқылы есептеңіз; нәтижені үш ондық таңбамен шығарыңыз.

Дәріс 4. Білімді бақылау

02. $a = 3$, $b = 4$ катеттерінен гипотенузаны есептеңіз. Күтілетін нәтиже 5.0.

Дәріс 4. Білімді бақылау

03. 3672 секундты сағат, минут және секундқа айналдырыңыз. Нәтиже: 1 сағат 1 минут 12 секунд.

Дәріс 4. Білімді бақылау

04. Үш санның орташа мәнін double ретінде есептеңіз. 1, 2, 2 сандары үшін бөлшек нәтиже сақталсын.

Дәріс 4. Білімді бақылау

05. $a = 1$, $b = -3$, $c = 2$ квадрат теңдеуінің түбірлерін есептеңіз. Нәтижелер 1 және 2 екенін тексеріңіз.

Дәріс 4. Білімді бақылау

05 / Шартты оператор: if

Дәріс 5. Білімді бақылау

Шартты оператор: if

Оңай · білу және түсіну

01. if операторы қандай жағдайда орындалады?

Дәріс 5. Білімді бақылау

02. if және else тармақтарының қызметін атаңыз.

Дәріс 5. Білімді бақылау

03. Салыстыру операторлары қандай типтегі нәтиже береді?

Дәріс 5. Білімді бақылау

Орташа · қолдану және салыстыру

04. && және || логикалық операторларының айырмасын түсіндіріңіз.

Дәріс 5. Білімді бақылау

05. Бірнеше диапазонды тексеруге else if тізбегін қалай қолданады?

Дәріс 5. Білімді бақылау

06. Меншіктеу мен салыстыру операторларын шатастырудың салдарын түсіндіріңіз.

Дәріс 5. Білімді бақылау

07. Күрделі шартта жақшалардың рөлі қандай?

Дәріс 5. Білімді бақылау

Қиын · талдау және негіздеу

08. Қысқа тұйықталу арқылы null мәнін тексергенде && операторы қатеден қалай қорғайды?

Дәріс 5. Білімді бақылау

09. Бірдей есеп үшін жеке if операторлары мен else if тізбегі әртүрлі нәтиже бере ала ма? Негіздеңіз.

Дәріс 5. Білімді бақылау

10. Шарт шекараларын тексеру кезінде қандай мәндерді таңдау керек және неліктен?

Дәріс 5. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Санның оң, теріс немесе нөл екенін анықтайтын бағдарлама жазыңыз. -2, 0, 3 мәндерін тексеріңіз.

Дәріс 5. Білімді бақылау

02. Үш санның ең үлкенін if арқылы табыңыз. 4, 9, 9 енгізуі үшін де дұрыс нәтиже берсін.

Дәріс 5. Білімді бақылау

03. 0–100 балды бағаларға бөліңіз: 90–100 A, 75–89 B, 60–74 C, 0–59 D. Аралықтан тыс мәнге хабарлама шығарыңыз.

Дәріс 5. Білімді бақылау

04. Жылды кібісе жылға тексеріңіз: 400-ге бөліну немесе 4-ке бөлініп, 100-ге бөлінбеу. 1900 және 2000 жылдарын салыстырыңыз.

Дәріс 5. Білімді бақылау

05. Үш оң ұзындықтың үшбұрыш құрауын тексеріңіз. 3,4,5 және 1,2,3 деректерімен сынаңыз.

Дәріс 5. Білімді бақылау

06 / Циклдік операторлар: for және while

Дәріс 6. Білімді бақылау

Циклдік операторлар: for және while

Оңай · білу және түсіну

01. Цикл қандай есептерде қолданылады?

Дәріс 6. Білімді бақылау

02. for операторының үш басқару бөлігін атаңыз.

Дәріс 6. Білімді бақылау

03. while циклінің шарты қашан тексеріледі?

Дәріс 6. Білімді бақылау

Орташа · қолдану және салыстыру

04. Қайталау саны белгілі есепке for таңдау себебін түсіндіріңіз.

Дәріс 6. Білімді бақылау

05. break пен continue циклдің орындалуына қалай әсер етеді?

Дәріс 6. Білімді бақылау

06. Цикл айнымалысының бастапқы мәні, шекарасы және қадамы нәтижеге қалай әсер етеді?

Дәріс 6. Білімді бақылау

07. Ішкі цикл мен сыртқы циклдің жұмысын кесте құру мысалында сипаттаңыз.

Дәріс 6. Білімді бақылау

Қиын · талдау және негіздеу

08. Шексіз циклдің пайда болу себептерін және оны анықтау тәсілін талдаңыз.

Дәріс 6. Білімді бақылау

09. n элементті бір цикл мен екі ішкі циклдің орындалу санын салыстырыңыз.

Дәріс 6. Білімді бақылау

10. Жинақтау айнымалысын дұрыс инициализацияламау қосынды мен көбейтінді есебіне қалай әсер етеді?

Дәріс 6. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. 1-ден n -ге дейінгі сандар қосындысын `for` арқылы есептеңіз. $n = 10$ үшін 55 шығуы тиіс.

Дәріс 6. Білімді бақылау

02. `while` арқылы санның цифрлар қосындысын табыңыз. 5027 үшін 14; 0 үшін 0.

Дәріс 6. Білімді бақылау

03. 1–10 көбейту кестесін екі ішкі циклмен шығарыңыз; бағандардың орналасуын біркелкі етіңіз.

Дәріс 6. Білімді бақылау

04. Пайдаланушы 0 енгізгенше сандарды оқып, енгізілген нөлден өзге сандардың санын және орташа мәнін табыңыз.

Дәріс 6. Білімді бақылау

05. 2-ден n -ге дейінгі жай сандарды шығарыңыз. $n = 20$ үшін 2,3,5,7,11,13,17,19 алынсын.

Дәріс 6. Білімді бақылау

07 / Тізімдер

Дәріс 7. Білімді бақылау

Тізімдер

Оңай · білу және түсіну

01. String класы қандай мәліметпен жұмыс істейді?

Дәріс 7. Білімді бақылау

02. String.length() әдісі нені анықтайды?

Дәріс 7. Білімді бақылау

03. StringBuffer класының негізгі қызметі қандай?

Дәріс 7. Білімді бақылау

Орташа · қолдану және салыстыру

04. String пен StringBuffer объектісін өзгерту мүмкіндігін салыстырыңыз.

Дәріс 7. Білімді бақылау

05. charAt() пен getChars() әдістерінің айырмасын түсіндіріңіз.

Дәріс 7. Білімді бақылау

06. substring() әдісіндегі бастапқы және соңғы индекстердің мағынасын түсіндіріңіз.

Дәріс 7. Білімді бақылау

07. equals() арқылы мазмұнды салыстыру мен == арқылы сілтемені салыстырудың айырмасы қандай?

Дәріс 7. Білімді бақылау

Қиын · талдау және негіздеу

08. Дәрістері StringBuffer insert(), delete() және replace() операциялары индекстерге қалай әсер етеді?

Дәріс 7. Білімді бақылау

09. Циклде көп мәтін біріктіргенде өзгертін буферді қолданудың пайдасын негіздеңіз.

Дәріс 7. Білімді бақылау

10. String класының final болуы мен оның мәнінің өзгермейтін болуы бір ұғым ба? Айырмасын түсіндіріңіз.

Дәріс 7. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. «Алматы» тіркесінің ұзындығын және алғашқы, соңғы символын шығарыңыз.
Нәтижелер: 6, А, ы.

Деріс 7. Білімді бақылау

02. «Бағдарламалау» жолынан алғашқы 6 символды substring() арқылы бөліп алыңыз;
бастапқы жолды да шығарыңыз.

Деріс 7. Білімді бақылау

03. StringBuffer арқылы «Java» тіркесіне « 2026» қосыңыз және reverse() нәтижесін
шығарыңыз.

Деріс 7. Білімді бақылау

04. Жол ішіндегі барлық ! белгілерін ? таңбасына ауыстырыңыз. «Сәлем! Java!» үшін
«Сәлем? Java?» алынсын.

Деріс 7. Білімді бақылау

05. Енгізілген жолдың палиндром болуын тексеріңіз; «қазақ» және «кітап» сөздерін
сынаңыз.

Деріс 7. Білімді бақылау

08 / Рекурсия және екі өлшемді массивтер

Дәріс 8. Білімді бақылау

Рекурсия және екі өлшемді массивтер

Оңай · білу және түсіну

01. Рекурсия дегеніміз не?

Дәріс 8. Білімді бақылау

02. Рекурсивті әдістегі базалық жағдай не үшін керек?

Дәріс 8. Білімді бақылау

03. Екі өлшемді массивтің жолы мен бағаны нені білдіреді?

Дәріс 8. Білімді бақылау

Орташа · қолдану және салыстыру

04. Факториалды рекурсивті есептеудің қадамдарын түсіндіріңіз.

Дәріс 8. Білімді бақылау

05. Матрица элементіне екі индекс арқылы қалай қатынайды?

Дәріс 8. Білімді бақылау

06. Матрицаның барлық элементін аралауға екі цикл неліктен қажет?

Дәріс 8. Білімді бақылау

07. Рекурсия мен цикл арқылы есептеудің ұқсастығы және айырмасы қандай?

Дәріс 8. Білімді бақылау

Қиын · талдау және негіздеу

08. Рекурсияның тоқтау шарты орындалмаса шақыру стегінде не болады?

Дәріс 8. Білімді бақылау

09. Рекурсивті Фибоначчи алгоритмінде қайталанатын есептеулер неліктен көп болады?

Дәріс 8. Білімді бақылау

10. Матрицаларды көбейтуге өлшемдер бойынша қандай шарт қойылады? Нәтиже өлшемін негіздеңіз.

Дәріс 8. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. $\text{factorial}(n)$ рекурсивті әдісін жазыңыз. $n = 0$ үшін 1, $n = 5$ үшін 120; теріс мәнді қабылдамаңыз.

Деріс 8. Білімді бақылау

02. Берілген санның цифрларының қосындысын рекурсиямен табыңыз. 2046 үшін 12.

Деріс 8. Білімді бақылау

03. 3×3 матрицаның негізгі диагональ элементтерінің қосындысын есептеңіз; 1–9 мәндері үшін 15.

Деріс 8. Білімді бақылау

04. 2×3 матрицаны транспозициялаңыз. $[[1,2,3],[4,5,6]]$ үшін $[[1,4],[2,5],[3,6]]$.

Деріс 8. Білімді бақылау

05. 2×2 матрицаларды көбейтіңіз: $[[1,2],[3,4]]$ және $[[5,6],[7,8]]$. Нәтиже $[[19,22],[43,50]]$.

Деріс 8. Білімді бақылау

09 / Функцияны хабарлау. Жиындар

Дәріс 9. Білімді бақылау

Функцияны хабарлау. Жиындар

Оңай · білу және түсіну

01. Функция декларациясы қандай бөліктерден тұрады?

Дәріс 9. Білімді бақылау

02. Формальді параметр мен нақты аргументтің айырмасы қандай?

Дәріс 9. Білімді бақылау

03. return операторы қандай қызмет атқарады?

Дәріс 9. Білімді бақылау

Орташа · қолдану және салыстыру

04. Дәрістері Greeting() пен Factorial() функцияларының мақсатын салыстырыңыз.

Дәріс 9. Білімді бақылау

05. JavaScript function жазылуы мен Java әдісінің хабарлануын ажыратыңыз.

Дәріс 9. Білімді бақылау

06. Math.sin() аргументі қандай өлшем бірлігінде беріледі? Градустан радианға қалай ауыстырады?

Дәріс 9. Білімді бақылау

07. Массивті хабарлау, құру және инициалдау кезеңдерін түсіндіріңіз.

Дәріс 9. Білімді бақылау

Қиын · талдау және негіздеу

08. Дәрістері Factorial() мысалында $n=0$ базалық жағдайы болмаған кезде не болатынын талдаңыз.

Дәріс 9. Білімді бақылау

09. Math.ceil(), Math.floor() және Math.round() нәтижелерін теріс бөлшек сан үшін салыстырыңыз.

Дәріс 9. Білімді бақылау

10. Элементтері әртүрлі ұзындықтағы ішкі массивтерден тұратын құрылымды қалай аралау керек?

Дәріс 9. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Дәрістергі факториал идеясын Java әдісіне айналдырыңыз. Базалық жағдай қосыңыз; 0 үшін 1, 5 үшін 120.

Дәріс 9. Білімді бақылау

02. Java greeting(String name) әдісін жазыңыз. «Азат» аргументіне «Сәлем, Азат!» шығарсын.

Дәріс 9. Білімді бақылау

03. 30, 45 және 90 градус үшін Math.toRadians() арқылы синусты есептеп, нәтижені үш ондық таңбамен шығарыңыз.

Дәріс 9. Білімді бақылау

04. double массивіне 0.01,-3.4,2.89,4.5,-6.7 мәндерін беріп, ең кіші және ең үлкен мәнін табыңыз.

Дәріс 9. Білімді бақылау

05. Жол ұзындықтары 2,4,3 болатын char[][] массивін құрып, әр жолдың элементтерін бөлек шығарыңыз.

Дәріс 9. Білімді бақылау

10 / Сөздіктер

Дәріс 10. Білімді бақылау

Сөздіктер

Оңай · білу және түсіну

01. Дәрістегі сөз тіркесі ұғымы нені білдіреді?

Дәріс 10. Білімді бақылау

02. Бос тіркес пен символдары бар тіркестің айырмасы қандай?

Дәріс 10. Білімді бақылау

03. Конкатенация дегеніміз не?

Дәріс 10. Білімді бақылау

Орташа · қолдану және салыстыру

04. Тіркесті толық көшіру мен оның бөлігін көшірудің айырмасын түсіндіріңіз.

Дәріс 10. Білімді бақылау

05. Тіркестің ұзындығы мен жадыдағы сыйымдылығы бірдей ұғым ба?

Дәріс 10. Білімді бақылау

06. Ішкі тіркесті іздеу нәтижесінде бастапқы позицияны қалай пайдаланады?

Дәріс 10. Білімді бақылау

07. Тіркестердің бөліктерін салыстыруға қандай мәліметтер қажет?

Дәріс 10. Білімді бақылау

Қиын · талдау және негіздеу

08. Дәрістері `string`, `assign()`, `c_str()` және `<string>` атаулары C++ мысалына жатады; оларды Java `String` әдістерімен салыстырыңыз.

Дәріс 10. Білімді бақылау

09. Тіркес шекарасынан тыс индекс қолданудың салдарын Java және дәрістері `at()` мысалы үшін түсіндіріңіз.

Дәріс 10. Білімді бақылау

10. Қайталанған сөзді алмастырғанда тек бірінші сәйкестікті немесе барлық сәйкестікті таңдаудың айырмасы қандай?

Дәріс 10. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. №11 практикалық жұмыстағы «Айна» сөзін «Асыл» сөзіне ауыстыру есебін Java тілінде шешіңіз.

Деріс 10. Білімді бақылау

02. «Java тілін үйренемін» тіркесіндегі ең қысқа сөздің ұзындығын табыңыз. Нәтиже: 4.

Деріс 10. Білімді бақылау

03. Мәтіндегі берілген символдың барлық кездесуін өшіретін Java әдісін жазыңыз; «алма», а үшін «лм».

Деріс 10. Білімді бақылау

04. «Java» және «java» тіркестерін equals() және equalsIgnoreCase() арқылы салыстырып, нәтижелерін түсіндіріңіз.

Деріс 10. Білімді бақылау

05. Мәтіннің әрбір үшінші символын өшіріңіз. «abcdefghi» үшін «abdegh» алынсын.

Деріс 10. Білімді бақылау

11 / Объектіге бағытталған бағдарламалауға кіріспе

Дәріс 11. Білімді бақылау

Объектіге бағытталған бағдарламалауға кіріспе

Оңай · білу және түсіну

01. Класс пен объектінің айырмасы қандай?

Дәріс 11. Білімді бақылау

02. Объектінің өрісі мен әдісі нені сипаттайды?

Дәріс 11. Білімді бақылау

03. Объектіге бағытталған бағдарламалаудың негізгі қасиеттерін атаңыз.

Дәріс 11. Білімді бақылау

Орташа · қолдану және салыстыру

04. Инкапсуляция деректерді қорғауға қалай көмектеседі?

Дәріс 11. Білімді бақылау

05. Мұрагерлік арқылы жалпы қасиеттерді қайта пайдалану тәсілін түсіндіріңіз.

Дәріс 11. Білімді бақылау

06. Полиморфизмді бір атаулы әдістің әртүрлі объекте орындалуы арқылы сипаттаңыз.

Дәріс 11. Білімді бақылау

07. Абстрактілі сипаттама мен нақты іске асырудың байланысы қандай?

Дәріс 11. Білімді бақылау

Қиын · талдау және негіздеу

08. «Студент» объектісіне барлық өрісті public ету қандай мәселе туғызады?

Дәріс 11. Білімді бақылау

09. Мұрагерлік пен объект құрамына басқа объект енгізуді қай жағдайда таңдаған дұрыс?

Дәріс 11. Білімді бақылау

10. Бір жауапкершілігі бар кластарға бөлу бағдарламаны кеңейтуге қалай әсер етеді?

Дәріс 11. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Student класын name және grade өрістерімен құрып, екі объектінің мәліметін шығарыңыз.

Дәріс 11. Білімді бақылау

02. BankAccount класында private balance, deposit() және withdraw() әдістерін жасаңыз. Теріс сома мен артық алуды қабылдамаңыз.

Дәріс 11. Білімді бақылау

03. Shape ата-аналық класынан Rectangle және Circle кластарын құрыңыз; area() әдісін қайта анықтаңыз.

Дәріс 11. Білімді бақылау

04. Бір Shape тізіміне шеңбер мен тіктөртбұрышты қосып, аудандарын бір циклде полиморфизм арқылы шығарыңыз.

Дәріс 11. Білімді бақылау

05. Library және Book кластарын құрып, кітапты беру және қайтару әрекетін модельдеңіз; бір кітапты екі рет беруге жол бермеңіз.

Дәріс 11. Білімді бақылау

12 / Модульдер

Дәріс 12. Білімді бақылау

Модульдер

Оңай · білу және түсіну

01. Конструктордың негізгі қызметі қандай?

Дәріс 12. Білімді бақылау

02. Конструктордың аты класс атымен қалай байланысады?

Дәріс 12. Білімді бақылау

03. this кілт сөзі нені білдіреді?

Дәріс 12. Білімді бақылау

Орташа · қолдану және салыстыру

04. Дәрістегі класс, объект және конструктор ұғымдарының байланысын түсіндіріңіз.

Дәріс 12. Білімді бақылау

05. Параметрсіз және параметрлі конструкторларды салыстырыңыз.

Дәріс 12. Білімді бақылау

06. Өріс пен параметр атаулары бірдей болса this қолдану не үшін қажет?

Дәріс 12. Білімді бақылау

07. Класстарды пакетке топтастыру кодты ұйымдастыруға қалай көмектеседі?

Дәріс 12. Білімді бақылау

Қиын · талдау және негіздеу

08. Объект құру сәтінде жарамсыз мәндерді қабылдау кейінгі есептеуге қалай әсер етеді?

Дәріс 12. Білімді бақылау

09. Конструктордан басқа конструкторды шақыру қайталанатын инициализацияны қалай азайтады?

Дәріс 12. Білімді бақылау

10. Мұрагерлік кезінде ата-аналық объект бөлігінің инициализациясы неліктен маңызды?

Дәріс 12. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Book класын title және author параметрлері бар конструктормен жасаңыз. Екі кітап объектісін құрыңыз.

Деріс 12. Білімді бақылау

02. Rectangle класына параметрсіз және параметрлі конструктор қосыңыз; әдепкі өлшемдері 1 және 1 болсын.

Деріс 12. Білімді бақылау

03. Student класында this арқылы ат пен жас өрістерін инициализациялаңыз; теріс жасты қабылдамаңыз.

Деріс 12. Білімді бақылау

04. model және app пакеттерін құрып, app.Main ішінен model.Student класын импорттап қолданыңыз.

Деріс 12. Білімді бақылау

05. Бірнеше конструкторы бар Product класын жасаңыз: атау ғана берілсе баға 0, атау мен баға берілсе сол мән қолданылсын.

Деріс 12. Білімді бақылау

13 / Matlab тәрізді және өзге кітапханаларды пайдалану

Дәріс 13. Білімді бақылау

Mathlab тәрізді және өзге кітапханаларды пайдалану

Оңай · білу және түсіну

01. MATLAB жүйесі қандай есептерге арналған?

Дәріс 13. Білімді бақылау

02. MATLAB жүйесінің негізгі деректер объектісі қандай?

Дәріс 13. Білімді бақылау

03. Command Window терезесіндегі >> белгісі нені білдіреді?

Дәріс 13. Білімді бақылау

Орташа · қолдану және салыстыру

04. Ғылыми калькулятор режимі мен программалық режимді салыстырыңыз.

Дәріс 13. Білімді бақылау

05. М-файл дегеніміз не және оның кеңейтілуі қандай?

Дәріс 13. Білімді бақылау

06. Workspace ұғымының мақсатын түсіндіріңіз.

Дәріс 13. Білімді бақылау

07. Simulink динамикалық жүйені модельдеуде қандай мүмкіндік береді?

Дәріс 13. Білімді бақылау

Қиын · талдау және негіздеу

08. Матрицаларға жасалатын операцияларда өлшемдер сәйкестігі неліктен қажет?

Дәріс 13. Білімді бақылау

09. Функция графигін құруда аргумент диапазоны мен қадам нәтижеге қалай әсер етеді?

Дәріс 13. Білімді бақылау

10. MATLAB ортасы, Simulink пакеті және Java Math класының міндеттерін ажыратып, есепке сай таңдау жасаңыз.

Дәріс 13. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. MATLAB-да $A=[1 \ 2; 3 \ 4]$ және $B=[5 \ 6; 7 \ 8]$ матрицаларын құрып, $A+B$ және $A*B$ нәтижелерін есептеңіз.

Дәріс 13. Білімді бақылау

02. Command Window ішінде $\text{sqrt}(81)$, $\text{abs}(-9)$ және 2^{10} есептеңіз. Нәтижелер: 9,9,1024.

Дәріс 13. Білімді бақылау

03. $x=-\pi:0.05:\pi$ және $y=\sin(x)$ үшін график салыңыз; ось атаулары мен тақырып қосыңыз.

Дәріс 13. Білімді бақылау

04. Екі матрицаны қосатын M-файл функциясын құрыңыз; өлшемі сәйкес келмейтін деректі тексеріңіз.

Дәріс 13. Білімді бақылау

05. Дәрістегі есептеудің Java баламасын жасаңыз: 2×2 матрицаларды үш циклмен көбейтіп, MATLAB нәтижесімен салыстырыңыз.

Дәріс 13. Білімді бақылау

14 / Қателерді талдау: throwExceptions

Дәріс 14. Білімді бақылау

Қателерді талдау: throwExceptions

Оңай · білу және түсіну

01. Ерекше жағдай (exception) дегеніміз не?

Дәріс 14. Білімді бақылау

02. Throwable, Exception және Error атауларының байланысы қандай?

Дәріс 14. Білімді бақылау

03. throw кілт сөзі қандай қызмет атқарады?

Дәріс 14. Білімді бақылау

Орташа · қолдану және салыстыру

04. Checked және unchecked ерекше жағдайларды салыстырыңыз.

Дәріс 14. Білімді бақылау

05. try және catch блоктарының жұмыс ретін түсіндіріңіз.

Дәріс 14. Білімді бақылау

06. throws хабарлауы мен throw операторының айырмасы қандай?

Дәріс 14. Білімді бақылау

07. Қате туралы хабарлама мен шақыру стегі қате орнын табуға қалай көмектеседі?

Дәріс 14. Білімді бақылау

Қиын · талдау және негіздеу

08. Бірнеше catch блогын жалпы типтен нақты типке қарай орналастыру неліктен компиляцияға кедергі келтіреді?

Дәріс 14. Білімді бақылау

09. Ерекше жағдайды бос catch блогында жұтып қою бағдарлама нәтижесіне қандай қауіп туғызады?

Дәріс 14. Білімді бақылау

10. Мәнді алдын ала тексеру мен ерекше жағдайды ұстаудың қолданылуын салыстырып, таңдау себебін негіздеңіз.

Дәріс 14. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. Бүтін санды нөлге бөлуді try/catch арқылы өңдеңіз; бағдарлама түсінікті хабарлама шығарып, жұмысын жалғастырсын.

Деріс 14. Білімді бақылау

02. Массив шекарасынан тыс индекс үшін `ArrayIndexOutOfBoundsException` ұстаңыз. 3 элементті массивтің 5 индексін тексеріңіз.

Деріс 14. Білімді бақылау

03. `age < 0` болғанда `IllegalArgumentException` тастайтын `setAge()` әдісін жазыңыз; -1 және 18 мәндерін сынаңыз.

Деріс 14. Білімді бақылау

04. `Integer.parseInt()` үшін жарамды «42» және жарамсыз «abc» жолдарын сынап, `NumberFormatException` өңдеңіз.

Деріс 14. Білімді бақылау

05. Өзіңіздің `InvalidGradeException` класын жасап, 0–100 аралығынан тыс бағаға қолданыңыз; 101 және 75 мәндерін тексеріңіз.

Деріс 14. Білімді бақылау

15 / Ерекше жағдайларды өңдеу және бағдарламаның тұрақтылығы

Дәріс 15. Білімді бақылау

Ерекше жағдайларды өңдеу және бағдарламаның тұрақтылығы

Оңай · білу және түсіну

01. №15 зертханалық жұмыста қандай ерекше жағдайлар қарастырылады?

Дәріс 15. Білімді бақылау

02. InputMismatchException қандай енгізуде пайда болады?

Дәріс 15. Білімді бақылау

03. finally блогының негізгі мақсаты қандай?

Дәріс 15. Білімді бақылау

Орташа · қолдану және салыстыру

04. $\text{int[] } m = \{-1, 0, 1\}$ массиві мен $m[a] = 4/a$ өрнегі үшін $a=0$ және $a=3$ жағдайларын салыстырыңыз.

Дәріс 15. Білімді бақылау

05. Нақты ерекше жағдайларды жалпы Exception түрінен бұрын ұстау себебін түсіндіріңіз.

Дәріс 15. Білімді бақылау

06. Қате енгізуден кейін Scanner ішінде қалған токенді алып тастау не үшін қажет?

Дәріс 15. Білімді бақылау

07. finally мен try-with-resources тәсілдерінің ресурс жабуға қатысты айырмасын түсіндіріңіз.

Дәріс 15. Білімді бақылау

Қиын · талдау және негіздеу

08. Бір өрнекте бірнеше қате мүмкін болса, бағалау реті қай ерекше жағдайдың алдымен туатынын қалай анықтайды?

Дәріс 15. Білімді бақылау

09. Лабораториядағы $m[a-1] = 4/a$ өрнегін $a=0, 1, 2, 3$ мәндері үшін талдап, қауіпсіз нұсқасын ұсыныңыз.

Дәріс 15. Білімді бақылау

10. Қалыпты орындалу, нөлге бөлу, қате индекс және қате енгізу жағдайларын қамтитын тексеру жоспарын негіздеңіз.

Дәріс 15. Білімді бақылау

Практикалық өзіндік тапсырмалар

Алгоритмді, бастапқы кодты және тексеру нәтижелерін ұсыныңыз.

01. №15 жұмысындағы $m[a] = 4/a$ мысалын үш бөлек catch блогымен іске асырыңыз: арифметика, индекс және енгізу типі.

Дәріс 15. Білімді бақылау

02. Бағдарламаға 1, 0, 3 және «abc» енгізіңіз. Әр жағдай үшін күтілетін және нақты нәтижені кестеде тіркеңіз.

Дәріс 15. Білімді бақылау

03. Жарамды индекс және нөлден өзге сан енгізілгенше сұрауды қайталайтын нұсқа жасаңыз; қате токенді алып тастаңыз.

Дәріс 15. Білімді бақылау

04. finally блогында «Өңдеу аяқталды» хабарламасын шығарыңыз. Қалыпты және қате орындалуда оның шақырылғанын тексеріңіз.

Дәріс 15. Білімді бақылау

05. Бастапқы $m[a-1] = 4/a$ кодын түзетіп, $a=1$ және $a=2$ жағдайларын қауіпсіз есептеңіз; қалған мәнге түсінікті хабарлама беріңіз.

Дәріс 15. Білімді бақылау

09 / ТЕРМИНДЕР

Глоссарий

Алгоритм

Есепті шешуге арналған анық әрі реттелген әрекеттер тізбегі.

Айнымалы

Бағдарламада аты мен типі бар, мәні өзгеруі мүмкін дерек сақтау орны.

Байт-код

Java компиляторы жасайтын және JVM орындайтын аралық нұсқаулар.

JVM — Java Virtual Machine

Java виртуалды машинасы; байт-кодты орындайтын орта.

JDK — Java Development Kit

Java әзірлеу құралдарының жиынтығы; компилятор және басқа құралдар кіреді.

JRE — Java Runtime Environment

Java бағдарламаларын орындауға қажетті орта ұғымы.

IDE — Integrated Development Environment

Код жазу, іске қосу және қатені іздеуді біріктіретін әзірлеу ортасы.

ООП / OOP — Object-Oriented Programming

Объектіге бағытталған бағдарламалау; дерек пен әрекетті объектке біріктіру тәсілі.

Класс

Объекттердің өрістері мен әдістерін сипаттайтын үлгі.

Объект

Класс негізінде құрылған, өз күйі мен әрекеттері бар экземпляр.

Конструктор

Объект құрылған кезде оны инициализациялайтын арнайы құрылым.

Инкапсуляция

Объекттің ішкі деректерін бақылауда ұстап, қатынауды әдістер арқылы ұйымдастыру.

Мұрагерлік

Бір кластың басқа кластың қасиеттері мен әрекеттерін кеңейтуі.

Полиморфизм

Бір ортақ тип арқылы әртүрлі объекттердің өз іске асыруын шақыру мүмкіндігі.

Әдіс

Класс ішінде хабарланған, белгілі әрекет орындайтын код бөлігі.

Параметр / аргумент

Параметр — әдіс анықтамасындағы атаулы кіріс; аргумент — шақырғанда берілетін нақты мән.

Рекурсия

Әдістің өзін шақыруы; тоқтайтын базалық жағдай болуы тиіс.

Массив

Бір типтегі элементтерді индекстермен сақтайтын құрылым.

Тізім / List

Реттілігі сақталатын және қайталанатын мәндерге жол беретін коллекция.

Жиын / Set

Қайталанбайтын элементтер коллекциясы.

Сөздік / Map

Кілт пен мән жұптарын сақтайтын құрылым; кілттер қайталанбайды.

Пакет

Өзара байланысты Java класстары мен интерфейстерінің атаулар кеңістігі.

Модуль

Тәуелділігі мен экспортталатын пакеттері анықталған кодты ұйымдастыру бірлігі; пакетпен бірдей ұғым емес.

API — Application Programming Interface

Бағдарламалық компоненттің қолжетімді әдістері мен пайдалану келісімі.

GUI — Graphical User Interface

Терезе, батырма және графика арқылы әрекеттесетін интерфейс.

MATLAB — Matrix Laboratory

Матрицалық есептеулерге арналған бөлек орта. Java Math класынан ажыратылады.

Exception

Орындалудың қалыпты ағынын бұзатын, өңдеуге болатын ерекше жағдай объектісі.

Throwable

Java-дағы тасталатын ерекше жағдайлар иерархиясының түбір класы.

Error

Әдетте қолданба деңгейінде қалпына келтіру қиын жүйелік ақаулар тармағы.

Checked / unchecked

Компилятор өңдеуді талап ететін және талап етпейтін ерекше жағдайлар санаттары.

try / catch / finally

Қауіпті кодты орындау, ерекше жағдайды ұстау және қорытынды әрекетті орындау блоктары.

throw / throws

throw — ерекше жағдайды тастайды; throws — әдістен шығуы мүмкін түрлерді хабарлайды.

InputMismatchException

Scanner сұралған типке сәйкес келмейтін токенді кездестіргенде туатын ерекше жағдай.

ArrayIndexOutOfBoundsException

Массивтің жарамды шекарасынан тыс индекс қолданылғанда туатын ерекше жағдай.

ArithmeticException

Бүтін санды нөлге бөлу тәрізді жарамсыз арифметикалық операциядағы ерекше жағдай.

10 / НЕГІЗГІ ТҰЖЫРЫМДАР

Қорытынды

Java бағдарламасын құру есепті талдаудан басталады: кіріс деректер, қажетті нәтиже және алгоритмнің шарттары анық болуы тиіс. Типтерді, операторларды және басқару құрылымдарын дұрыс таңдау есептеу нәтижесінің дұрыстығын қамтамасыз етеді.

Әдістер, массивтер және коллекциялар есепті бөліктерге бөліп шешуге мүмкіндік береді. Жол, тізім, жиын және сөздік әртүрлі міндет атқарады; деректер құрылымы есептің қасиетіне сай таңдалады.

Класс пен объект деректерді және сол деректерге қатысты әрекеттерді біріктіреді. Инкапсуляция, мұрагерлік және полиморфизм бағдарламаның түсініктілігін және кеңейтілуін арттырады.

Математикалық және графикалық құралдарды қолданғанда өлшемдер, координаттар және есептеу дәлдігі ескеріледі. MATLAB пен Java құралдарының міндеттерін ажырату дұрыс орта таңдауға көмектеседі.

Ерекше жағдайларды өңдеу, енгізуді тексеру және шекаралық сынақтар — сенімді бағдарламаның ажырамас бөлігі. №15 жұмыстың негізгі тұжырымы: қате себебін нақты анықтап, оны пайдаланушыға түсінікті түрде көрсету керек.

Пәнді меңгерудің нәтижесі — тек код жаза білу емес, шешімін негіздеу, тексеру хаттамасын дайындау және басқа адамның түсінуіне қолайлы бағдарлама ұсыну.

11 / ДЕРЕККӨЗДЕР

Пайдаланған әдебиеттер

Бастапқы оқу құралындағы әдебиеттер

1. Java:Объектно-ориентированное программирование Васильев А.Н. 2011, Питер
2. Turbo Pascal бағдарламалау тілі (зертханалық жұмыс):
оқу құралы – Астана Фоллиант, – 208б 2010
3. Шалыто А. А. Алгоритмизация и программирование для логического управления и «реактивных» систем//Автоматика и телемехани 2001. № .1 .-С . 3-39.
4. Демидович Е. М., Романчик В. С. Основы алгоритмизация программирования //Язык СИ/Е.М. Демидович - Минск: Бестпринт. – 2001
5. Шалыто А. А. Switch-технология. Алгоритмизаций программирование задач логического управления. - СПб: Наука, 1998. Мовшович С. М., Кравченко О.А. Алгоритмизаци программирование. - 2008. Электрондық ресурстар:
Мурат Йенер, Алекс Фидан JAVA паттерны проектирования для п рофессионалов 14,7 МБ Питер, 2016. -240 с.

Қосымша оқу және ресми анықтамалар

Oracle / Dev.java. Learn Java: тіл, объекттер, коллекциялар және модульдер
<https://dev.java/learn/>

Oracle. The Java Tutorials: Catching and Handling Exceptions
<https://docs.oracle.com/javase/tutorial/essential/exceptions/handling.html>

Oracle. The Java Tutorials: Collections

<https://docs.oracle.com/javase/tutorial/collections/>

MathWorks. MATLAB documentation

<https://www.mathworks.com/help/matlab/>

ҚарТУ. Объектіге бағытталған Java бағдарламалау. Қазақша ашық видеокурс

<https://edu.kstu.kz/course/view.php?id=3204>

Қолданылған қазақша видеосабақтар

Лекция 1 Объектіге бағытталған бағдарламалауға кіріспе

КиИИ_ҚарТУ имени Абылкаса Сагинова · Дәрістер: 1

<https://www.youtube.com/watch?v=0uzlT-woqPk>

Java программалау тілі / Сабақ #1.

Школа Программирования · Дәрістер: 2, 4

<https://www.youtube.com/watch?v=Oy8iUj-t0ME>

Лекция 8 Кіріс шығыс үрдістері

КиИИ_ҚарТУ имени Абылкаса Сагинова · Дәрістер: 3

<https://www.youtube.com/watch?v=J15ZmrZ0eKY>

Java программалау тілі / Сабақ #2. Шартты оператор

Школа Программирования · Дәрістер: 5

https://www.youtube.com/watch?v=QnXjS2__wol

Алгоритмдер қазақша | #2 — Сұрыптау алгоритмдері

CodeeKZ — ашық IT сабақтары · Дәрістер: 6

<https://www.youtube.com/watch?v=BwR8B-xyacl>

Лекция 6 Жолдармен жұмыс

КиИИ_ҚарТУ имени Абылкаса Сагинова · Дәрістер: 7

<https://www.youtube.com/watch?v=1cCErm7m9ul>

Java. ООП. Матрицаны құру және көбейту программасы

Aibol Maxotov · Дәрістер: 8

https://www.youtube.com/watch?v=_0wTQIUfyog

Пайдаланушы функциялары мен процедуралары. Рекурсия

Қашықтан үйрен / Информатика АКТ · Дәрістер: 9

<https://www.youtube.com/watch?v=l5Sm2Ps6AOg>

Лекция 10 Объектіге бағытталған бағдарламалау принциптерін жолдарға қолдану

КиИИ_КарТУ имени Абылкаса Сагинова · Дәрістер: 10

<https://www.youtube.com/watch?v=ymEk7l1pDXk>

Лекция 2 Объектіге бағытталған әдіс

КиИИ_КарТУ имени Абылкаса Сагинова · Дәрістер: 11

https://www.youtube.com/watch?v=e2_9PspS438

Лекция 3 Объектілер мен класстар туралы түсінік

КиИИ_КарТУ имени Абылкаса Сагинова · Дәрістер: 12

<https://www.youtube.com/watch?v=-FwgqBZ4pzig>

Лекция 7 Java тілінің стандартты кітапханалары және пакеттер

КиИИ_КарТУ имени Абылкаса Сагинова · Дәрістер: 13

<https://www.youtube.com/watch?v=Adfis0xegGY>

Лекция 9 Ерекшеліктерді өңдеу

КиИИ_КарТУ имени Абылкаса Сагинова · Дәрістер: 14, 15

<https://www.youtube.com/watch?v=5P1JN5l7sal>

Электрондық дереккөздер мен видеолекцияларға жүгіну күні: 05.10.2026. Жаңа сұрақтар мен тапсырмалар бастапқы оқу құралының нақты тақырыптары және осы бөлімдегі ресми анықтамалар негізінде құрастырылды.

12 / КОДТАР МЕН АЛГОРИТМДЕР

Қосымшалар

Бұл бөлімдегі Java үлгілері негізгі ұғымдарды қайталау және бастапқы мәтіндегі салыстырмалы тіл мысалдарын Java-ға бейімдеу үшін берілген.

ҚОСЫМША А / JAVA

А. Консольдік енгізу және есептеу

```
import java.util.Scanner;

public class Average {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            int a = sc.nextInt();
            int b = sc.nextInt();
            int c = sc.nextInt();
            double result = ((double) a + b + c) / 3;
            System.out.printf("%.2f%n", result);
        }
    }
}
```

Алгоритм: үш санды оқу → қосындыны нақты типке ауыстыру → 3-ке бөлу → нәтижені шығару. 1, 2, 2 енгізуінде 1.67 шығады.

ҚОСЫМША В / JAVA

В. Рекурсия және матрица

```
public class Algorithms {  
    static long factorial(int n) {  
        if (n < 0 || n > 20) {  
            throw new IllegalArgumentException("n: 0-20");  
        }  
        return n <= 1 ? 1 : n * factorial(n - 1);  
    }  
  
    static int diagonalSum(int[][] matrix) {  
        int sum = 0;  
        for (int i = 0; i < matrix.length; i++) {  
            if (matrix[i].length != matrix.length) {  
                throw new IllegalArgumentException(  
                    "Квадрат матрица қажет");  
            }  
            sum += matrix[i][i];  
        }  
        return sum;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorial(5));  
        int[][] m = {{1,2,3},{4,5,6},{7,8,9}};  
        System.out.println(diagonalSum(m));  
    }  
}
```

Күтілетін нәтиже: 120 және 15. Факториалға $n \leq 1$ базалық жағдайы берілген; long типінің шегіне сай $n \leq 20$ шарты қойылған.

ҚОСЫМША С / JAVA

С. Жолдар және коллекциялар

```
import java.util.*;

public class TextTools {
    public static void main(String[] args) {
        String text = "Айна Java тілін үйренеді";
        System.out.println(text.replace("Айна", "Асыл"));
        System.out.println(text.substring(0, 4));
        System.out.println(text.indexOf("Java"));
        StringBuffer buffer = new StringBuffer("Java");
        buffer.append(" 2026");
        System.out.println(buffer);

        List<Integer> list = new ArrayList<>(
            Arrays.asList(3, 1, 3, 2, 1));
        Set<Integer> unique = new LinkedHashSet<>(list);
        System.out.println(unique);

        Map<String, Integer> counts = new LinkedHashMap<>();
        for (String word : "java code java".split(" ")) {
            counts.merge(word, 1, Integer::sum);
        }
        System.out.println(counts);
    }
}
```

Нәтиже: Асыл Java тілін үйренеді; Айна; 5; Java 2026; [3, 1, 2]; {java=2, code=1}.
C++ assign() идеясына Java-да айнымалыға меншіктеу, find() идеясына indexOf(), салыстыруға equals() сәйкес келеді.

ҚОСЫМША D / JAVA

D. Объект және конструктор

```
public class Student {  
    private final String name;  
    private int grade;  
  
    public Student(String name, int grade) {  
        this.name = name;  
        setGrade(grade);  
    }  
  
    public void setGrade(int grade) {  
        if (grade < 0 || grade > 100) {  
            throw new IllegalArgumentException("Баға: 0-100");  
        }  
        this.grade = grade;  
    }  
  
    public String describe() {  
        return name + ": " + grade;  
    }  
  
    public static void main(String[] args) {  
        Student student = new Student("Азат", 85);  
        System.out.println(student.describe());  
    }  
}
```

Күтілетін нәтиже: Азат: 85. private өрістер инкапсуляцияны, конструктор объектін бастапқы күйге келтіруді көрсетеді.

ҚОСЫМША Е / БАҚЫЛАУ

Е. №15 жұмысқа тексеру хаттамасы

Енгізу	Күтілетін нәтиже
1	m[1] = 4
2	m[2] = 2
0	ArithmeticException өңделеді
3	ArrayIndexOutOfBoundsException өңделеді
abc	InputMismatchException өңделеді

Әр іске қосуда finally блогының «Өңдеу аяқталды» хабарламасын да тексеріңіз. Хаттаманы нақты нәтижемен, сәйкестік белгісімен және түзету түсіндірмесімен толықтырыңыз.

Практикалық жұмысты рәсімдеу үлгісі

1. Тақырып және мақсат.
2. Кіріс деректер, шектеулер және күтілетін нәтиже.
3. Алгоритм немесе блок-схема.
4. Бастапқы Java коды.
5. Қалыпты және шекаралық тексерулер кестесі.
6. Қорытынды және кездескен қателердің түсіндірмесі.